

Final Report

A Security Evaluation Framework for Distributed Retrieval-Augmented Generation Systems

by

Ifte Kharul Islam Nihal
22101104

Md. Rahim Bhuiyan
22101364

Md. Habibur Rahman
21201822

Nabel Ahmed
22101374

Tanzim Hossain Safin
22101118

A thesis submitted to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
BRAC University
August 2026

Declaration

To the best of our knowledge and belief, this thesis:

- Contains no material previously published or written by another person, except where due reference is made in the text of the thesis;
- Contains no material which has been accepted for the award of any other degree or diploma at any university or equivalent institution;
- Represents our own research work, and any collaborative contributions have been clearly indicated and acknowledged;
- Has been conducted in accordance with the ethical guidelines and regulations of BRAC University and relevant regulatory bodies;
- Contains no confidential or proprietary information that would violate privacy regulations, including GDPR and other applicable data protection frameworks;
- Uses publicly available datasets and acknowledges all sources appropriately.

Student's Full Name & Signature:

NABEL AHMED
22101374

IFTE KHARUL ISLAM NIHAL
22101104

TANZIM HOSSAIN SAFIN
22101118

MD. RAHIM BHUIYAN
22101364

MD. HABIBUR RAHMAN
21201822

Approval

The thesis titled “A Security Evaluation Framework for Distributed Retrieval-Augmented Generation Systems” submitted by

1. Ifte Kharul Islam Nihal (22101104)
2. Md. Habibur Rahman (21201822)
3. Md. Rahim Bhuiyan (22101364)
4. Nabel Ahmed (22101374)
5. Tanzim Hossain Safin (22101118)

of Spring 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Summer 2026.

Examining Committee:

Supervisor:
(Member)

Dr. Swakkhar Shatabda

Professor
Department of Computer Science and Engineering
BRAC University

Co-Supervisor:
(Member)

Mohammad Badrul Hossain

Lecturer
Department of Computer Science and Engineering
BRAC University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor and Chairperson
Department of Computer Science and Engineering
BRAC University

Ethics Statement

This thesis implements working attacks against retrieval-augmented generation systems, which carries a dual-use concern: code written to evaluate a defense could be repurposed against an undefended deployment. We address that concern in four ways.

Every experiment ran against sandboxed deployments on local and lab machines under our own control. We reached neither target system over a public network, and no production service or live user took part at any stage. Both DRAG and Reliable-dRAG are open-source research prototypes, and the modifications we made to them, including the write endpoint used by the data poisoning attack, exist only in our own working copies.

All corpora are public benchmark datasets: MMLU, SQuAD, PubMedQA, HuffPost News and a synthetic medical set. The work uses no personal data and involves no human subjects, so it required no clearance for human-subjects research.

Threat models that assume a compromised credential use test-only values. The Key-Forgery instantiation against Reliable-dRAG substitutes Hardhat’s documented default development keys for an adversary holding the orchestrator’s private key, and claims nothing about any deployed key.

We report a defense alongside each attack class rather than publishing offensive results alone. Chapter 5 pairs the six attack classes with six defense mechanisms and measures what each recovers, following the principle behind coordinated vulnerability disclosure: an exposure is worth reporting when the report also carries the means to close it.

Abstract

Traditional centralized RAG deployments place one administrator in charge of the whole knowledge base, which raises maintenance cost and denies data owners control of their documents. Distributed RAG systems emerged to remove that requirement, leaving each collection with its owner and routing queries across participants. Distribution carries its own cost. A system spreading knowledge across participants it does not control must decide which of them answers a query and trust what each reports about itself. Published security work on distributed RAG addresses one stage of that decision, the routing step, and no distributed RAG system published to date measures its own trust assumptions against an adversary. This thesis develops a security evaluation framework for such systems: a seven-step procedure that enumerates a target architecture’s trust assumptions and maps an attack onto the component carrying each, and six attack classes, two per CIA property, derived from the components distribution introduces beyond a centralized deployment. We implemented all six against DRAG, a peer-to-peer network routing by advertised topic expertise, and Reliable-dRAG, a three-source deployment reranking by a blockchain-held reliability score, then designed six defenses against our DRAG findings. The procedure produced Grounding-Farming, an attack on Reliable-dRAG’s reliability mechanism that needs no privileged access and that the system’s deployed defense cannot observe, since it inspects transaction properties the attack never exhibits. Knowledge extraction recovered 20 to 30 percent of a multiple-choice corpus against 1 to 6 percent of free-text corpora at identical topology and query budget. Two systems demonstrate that the procedure transfers rather than that it generalizes.

Keywords: Distributed/Decentralized Retrieval-Augmented Generation; Security Evaluation Framework; CIA Triad; Threat Modelling; Data Poisoning; Source Selection Manipulation; Knowledge Extraction; Membership Inference; Selective Forwarding; Peer-to-Peer Systems; Blockchain Reliability Scoring

Acknowledgement

We would like to express our sincere gratitude to all those who contributed to the successful completion of this thesis. Firstly, we extend our deepest appreciation to our thesis supervisor, Dr. Swakkhar Shatabda, and our co-supervisor, Mohammad Badrul Hossain, for providing invaluable guidance, continuous support, and insightful feedback throughout this research journey. We also acknowledge the support of our peers and colleagues who engaged in thoughtful discussions and provided constructive criticism that enhanced the quality of our work. We would also like to acknowledge the open-source community and researchers whose publicly available implementations and datasets made portions of this research possible.

Special thanks to our families for their unwavering support, patience, and encouragement throughout our academic journey. Their understanding during the demanding phases of this research has been invaluable.

Thank you all.

Contents

Declaration	i
Approval	iii
Ethics Statement	v
Abstract	vi
Acknowledgement	vii
Nomenclature	xv
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Methodology in Brief	4
1.6 Scope and Challenges	6
2 Literature Review	9
2.1 Preliminaries	9
2.2 Review of Existing Research	14
2.3 Summary of Key Findings	23
3 Requirements, Impacts and Constraints	25
3.1 Final Specifications and Requirements	25
3.2 Project Management Plan	26
3.3 Societal Impact	27
3.4 Environmental Impact	27
3.5 Ethical Issues	27
3.6 Standards	28
3.7 Risk Management	28
3.8 Economic Analysis	29
4 Proposed Methodology	30
4.1 Design Process or Methodology Overview	30
4.1.1 The Evaluation Procedure	30
4.1.2 Deriving the Attack Set	33
4.1.3 Scope of the Framework	34
4.1.4 Application of the Procedure: A Worked Example	34
4.1.5 From Procedure to Implementation	35
4.2 Preliminary Design or Design (Model) Specification	36
4.2.1 Data Poisoning Attack Design	36

4.2.2	Source Selection Manipulation Attack Design	39
4.2.3	Knowledge Extraction Attack Design	41
4.2.4	Membership Inference Attack Design	44
4.2.5	Selective Forwarding Attack Design	46
4.2.6	Denial of Service Attack Design	48
4.2.7	Defense Mechanism Design	50
4.3	Data Collection	54
4.3.1	Data Cleaning	55
4.3.2	Data Transformation	56
4.3.3	Data Integration	56
4.3.4	Data Reduction	57
4.3.5	Summary of Preprocessed Data	57
4.4	Implementation of Selected Design	58
4.4.1	DRAG Implementation	58
4.4.2	Reliable-dRAG Implementation	59
4.4.3	Experimental Environment	60
4.4.4	Execution Protocol	60
4.4.5	Implementation Provenance	61
5	Result Analysis	62
5.1	Performance Evaluation	62
5.1.1	Evaluation Criteria and Metrics	62
5.1.2	Severity Classification	63
5.1.3	Testing Methodology	63
5.2	Analysis of Design Solutions	63
5.2.1	Integrity: Data Poisoning	64
5.2.2	Integrity: Source Selection Manipulation	66
5.2.3	Confidentiality: Knowledge Extraction	69
5.2.4	Confidentiality: Membership Inference	72
5.2.5	Availability: Selective Forwarding	76
5.2.6	Availability: Denial of Service	78
5.3	Final Design Adjustments	81
5.3.1	Replication and Cross-Peer Validation	82
5.3.2	Topic-Claim Quota Enforcement	84
5.3.3	Layered Extraction Prevention	88
5.3.4	Privacy-Preserving Retrieval	89
5.3.5	Reputation-Based Forwarding Defense	91
5.3.6	Layered Congestion Defense	92
5.3.7	Reliable-dRAG Defenses	93
5.4	Statistical Analysis	93
5.4.1	Sample Sizes and Repetition	93
5.4.2	Sweep Design and Monotonicity	93
5.4.3	Held-Out Validation	94
5.4.4	Degenerate Thresholds	94
5.5	Comparisons and Relationships	94
5.5.1	Attack Classes Across Two Architectures	94
5.5.2	Comparison with Existing Work	97
5.6	Discussions	98

5.6.1	Interpretation	98
5.6.2	Implications for System Design	98
5.6.3	Limitations	98
6	Conclusion	101
6.1	Summary of Findings	101
6.2	Contributions to the Field	102
6.3	Recommendations for Future Work	103
6.3.1	Cross-Source Correlation Attacks	103
6.3.2	Closing the Defense Gap on Reliable-dRAG	103
6.3.3	Formal Privacy Accounting for Privacy-Preserving Retrieval . . .	103
6.3.4	Statistical Replication for DRAG	103
6.3.5	Live-Deployment Validation Beyond a Single Configuration	104
6.3.6	Extending the Framework to Federated Architectures	104
6.4	Closing Remarks	104

List of Figures

2.1	RAG System Workflow	10
4.1	The seven-step evaluation procedure	32
4.2	Derivation of the six attack classes	33
4.3	An overview of DRAG Architecture by Xu et al. Adapted from [9]	54
4.4	System overview of Reliable-dRAG by Lu et al. Adapted from [21]	55
5.1	Data poisoning on Reliable-dRAG: attacked accuracy against the clean baseline, and degradation against the 10% success threshold, by source-selection strategy and poison type (undefended, mean over seeds 0, 42 and 123).	66
5.2	SSM-Score on Reliable-dRAG: per-trial engagement for Grounding-Farming, which escalates in three of six independent trials, alongside Key-Forgery, which succeeds in every trial but requires the orchestrator’s private key. .	68
5.3	Extraction rate by peer count and connectivity, per dataset, with News topic coverage shown alongside (DRAG, undefended). Extraction axes are logarithmic and shared.	70
5.4	Knowledge extraction on Reliable-dRAG by access level: extraction rate per source across the unauthenticated, blind and gray-box conditions, and indirect leakage through the orchestrator against the 0.8 recovery threshold. .	72
5.5	Membership inference across five metrics by local model (DRAG, MMLU, undefended). Llama 3.2 3B and Qwen 2.5 3B overlap on every axis. . . .	73
5.6	Membership inference on Reliable-dRAG: candidate weightings compared across the search seeds, and the locked production weighting evaluated once against five seeds generated after selection.	75
5.7	Selective forwarding on Reliable-dRAG: simulated sweep across compromise ratio, with the two live measurements overlaid (hit rate and TTL exhaustion, undefended).	77
5.8	Query failure rate and answer quality by targeting strategy and intensity, across all three datasets (DRAG, undefended).	79
5.9	Mock sweep availability drop and answer-quality collapse under live flood. Query failure rate is annotated at each endpoint.	80
5.10	Data poisoning defence at both poison ratios: F1 recovery against the clean baseline, and the message cost each mechanism carries (DRAG, MMLU). .	83
5.11	Data poisoning under High-Degree targeting across all three datasets: clean baseline, undefended, and defended (DRAG).	84
5.12	Answer quality by dataset, undefended against topic-claim quota enforcement (DRAG). Recovery scales inversely with how much the attack took. .	86
5.13	Answer quality across the topic claim ratio, undefended against defended (DRAG, MMLU). The shaded region is the recovery, which widens as the attack grows more aggressive.	87

5.14	Extraction rate and chunk recovery, undefended against the layered defence, for every topology and dataset tested (DRAG). Horizontal axes are symmetric-logarithmic.	88
5.15	Privacy-utility frontier across five noise scales (DRAG, MMLU, Llama 3.2 3B). Each point is one measured setting; better lies down and to the right.	90
5.16	Selective forwarding on DRAG: dropped queries and hit rate, undefended against the reputation defence, by targeting strategy and compromise ratio.	91
5.17	Query failure rate by strategy and intensity, with and without layered defense (DRAG)	92
5.18	Severity by attack class and system, assigned with the rubric of Table 5.2. Cell annotations give the measurement behind each tier.	96

List of Tables

2.1	Comparative Summary of Reviewed Literature by Architecture, Attack Coverage, Defense Coverage, and Domain	21
4.1	Derivation of the Attack Set from the Components Introduced by Distribution	33
4.2	Corpora Used in the Evaluation, After Preparation	58
4.3	Implementation Modules by Attack and Defence Class (DRAG)	59
5.1	Framework Metric Assignment by Attack Class and CIA Property	62
5.2	Severity Rubric Applied Across Attack Classes	63
5.3	Data Poisoning Attack Results (DRAG, MMLU Dataset, No Defense)	64
5.4	Data Poisoning Attack Results Across Datasets (DRAG, High-Degree Targeting, No Defense)	64
5.5	Data Poisoning Attack Results (Reliable-dRAG, No Defense)	65
5.6	Routing Manipulation Attack Results Across Datasets (DRAG, 6 attacker peers, topic claim ratio 0.6, No Defense)	66
5.7	Routing Manipulation Attack: Topic Claim Ratio Sweep (DRAG, MMLU, No Defense)	67
5.8	SSM-Score Attack: Multi-Seed Results (Reliable-dRAG, No Defense)	67
5.9	Knowledge Extraction Attack Results by Network Topology and Dataset (DRAG, No Defense)	69
5.10	Knowledge Extraction Attack Results (Reliable-dRAG, Authenticated Attacker, No Defense)	71
5.11	Knowledge Extraction Attack Results, Indirect LLM Leakage (Reliable-dRAG, Phase C)	71
5.12	Membership Inference Attack Results by Model (DRAG, MMLU, No Defense)	72
5.13	Membership Inference Attack Results by Dataset (DRAG, Confidence-Based Inference, No Defense)	73
5.14	Membership Inference Attack Results (Reliable-dRAG, Production Multi-Seed, No Defense)	74
5.15	Membership Inference Composite Selection and Held-Out Confirmation (Reliable-dRAG)	74
5.16	Selective Forwarding Attack Results by Strategy and Ratio (DRAG, No Defense)	76
5.17	Selective Forwarding Attack Results by Strategy and Ratio (Reliable-dRAG, Mock Sweep, No Defense)	76
5.18	Selective Forwarding Attack: Live-Network Validation (Reliable-dRAG, Both Expressible Severities)	77
5.19	DoS Attack Results, Mid Intensity (DRAG, All Datasets, No Defense)	78
5.20	DoS Attack Results, High Intensity (DRAG, All Datasets, No Defense)	78
5.21	DoS Attack Results by Strategy and Ratio (Reliable-dRAG, Mock Sweep, No Defense)	80

5.22	DoS Attack Results: Live-Network Answer-Quality Degradation (Reliable-dRAG, No Defense)	80
5.23	Evaluation Finding, Design Adjustment, and Measured Result	81
5.24	Data Poisoning Defense Results (DRAG, MMLU Dataset)	82
5.25	Data Poisoning Defense Results Across Datasets (DRAG, High-Degree Targeting, Replication + Cross-Peer Validation)	82
5.26	Routing Manipulation Defense Results Across Datasets (DRAG, 6 attacker peers, topic claim ratio 0.6)	85
5.27	Routing Manipulation Defense: Topic Claim Ratio Sweep (DRAG, MMLU)	85
5.28	Knowledge Extraction Defense Results by Network Topology and Dataset (DRAG, Layered Defense)	88
5.29	Membership Inference Defense Results: Privacy-Preserving Retrieval Noise Sweep (DRAG, MMLU, Llama 3.2 3B)	89
5.30	Membership Inference Defense Results by Dataset (DRAG, Privacy-Preserving Retrieval, $b = 0.2$)	89
5.31	Selective Forwarding Defense Results by Strategy and Ratio (DRAG, Reputation-Based Defense)	91
5.32	DoS Defense Results, Mid Intensity (DRAG, All Datasets, Layered Defense)	92
5.33	DoS Defense Results, High Intensity (DRAG, All Datasets, Layered Defense)	92
5.34	Attack Class Instantiation and Severity Across Both Systems	95

Nomenclature

The following list describes several symbols & abbreviations that will be used within the body of the document:

AES-CBC	Advanced Encryption Standard - Cipher Block Chaining
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
ASR	Attack Success Rate
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BGE	BAAI General Embedding
CIA	Confidentiality, Integrity, Availability
CRR	Chunk Recovery Rate
DoS	Denial of Service
DDoS	Distributed Denial of Service
DP	Differential Privacy
DRAG	Distributed Retrieval-Augmented Generation (Xu et al.)
EED	Effective Edit Distance
EM	Exact Match
ER	Extraction Rate
F1	Harmonic mean of precision and recall
FAISS	Facebook AI Similarity Search
FPR	False Positive Rate
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
IVF	Inverted File Index
KV	Key-Value
LLM	Large Language Model
MAD	Median Absolute Deviation
MIA	Membership Inference Attack
MMLU	Massive Multitask Language Understanding
MRR	Mean Reciprocal Rank
PR	Poison Ratio
PRS	Privacy Risk Score
QFR	Query Failure Rate
RAG	Retrieval-Augmented Generation
Reliable-dRAG	Decentralized RAG with blockchain reliability scoring (Lu et al.)
SGX	Software Guard Extensions
SS	Semantic Similarity
TARW	Topic-Aware Random Walk
TC	Topic Coverage
TPR	True Positive Rate
TTL	Time-To-Live

Chapter 1

Introduction

This chapter proceeds as follows. Section 1.1 gives background on the field. Section 1.2 explains the motivation for the thesis. Section 1.3 states the problem. Section 1.4 lists the objectives, and Section 1.5 summarizes the methodology. Section 1.6 defines the scope of the completed work and the challenges encountered while carrying it out.

1.1 Background

The emergence of large language models (LLMs) has transformed how individuals and institutions engage with AI. Developed models such as GPT-5, Claude, and LLaMA excel at understanding, generating, and reasoning in natural language in a multitude of contexts. Applying them to a specific institution’s knowledge, however, requires giving the model access to documents that institution holds, and traditional approaches do this by collecting and storing that material centrally, which poses risks of compromise, unauthorized access, and misuse.

To address this, retrieval-augmented generation (RAG) systems have emerged, as they attempt to separate the knowledge storage from model parameters. These systems query external knowledge bases and combine the generated output with the retrieved content to generate contextually relevant and informed responses. However, traditional RAG systems still have underlying privacy issues as user queries and documents are retrieved in centralized systems which allow for query logging, membership inference attacks (MIAs) and unauthorized data access [3, 5, 13, 18]. Also, centralized knowledge bases create single points of failure and are targets for adversarial attacks [2, 8, 9, 15].

Distributed computing offers one route away from this single point of failure: rather than a single administrator managing one knowledge base, a distributed RAG system partitions knowledge across independently operated participants, each retaining control over their own data [8, 9]. Federated learning is a related but distinct paradigm in the broader literature, in which participants cooperatively improve a shared model through aggregated updates rather than answering queries directly against partitioned documents [16]. This thesis concentrates on the distributed case: both systems evaluated in this report, DRAG [9] and Reliable-dRAG [21], partition a knowledge base across independent participants and answer queries directly against that partitioned knowledge, rather than aggregating model updates, so federated architectures are discussed only as related background and are not themselves evaluated. Distributing a RAG deployment in this way does not remove security risk, it relocates it: the mechanisms a distributed system must introduce to select among sources, route queries, and establish trust between participants introduce attack surfaces that a centralized deployment does not possess [5].

1.2 Rationale of the Study or Motivation

The motivation for this research emerges from a critical gap between the growing deployment of distributed RAG systems and the absence of comprehensive security evaluation frameworks for them. While numerous studies have identified specific vulnerabilities in centralized systems, there exists no systematic methodology for evaluating the security posture of distributed alternatives.

Organizations seeking to deploy RAG over sensitive document collections face difficult decisions without frameworks for comparing security trade-offs between architectures. Healthcare, financial, educational, and commercial organizations handling sensitive data need evidence-based guidance on whether distributed RAG systems offer genuine privacy improvements over traditional centralized alternatives, and on what new risks that distribution introduces.

Current literature provides isolated attack demonstrations and point defenses for specific systems, but lacks comprehensive evaluation frameworks that characterize the full range of threats. The uniqueness of distributed approaches such as decentralized knowledge sharing, peer-to-peer communication, and blockchain-secured trust management introduces attack surfaces that are poorly understood. There is also a lack of standard benchmarks for comparing security claims across studies: different papers employ different threat models, evaluation metrics, and attack scenarios. As our literature review in Chapter 2 documents directly, almost none of the distributed or blockchain-secured systems published to date evaluate their own privacy exposures empirically, and third-party evaluation of a distributed architecture has so far addressed the routing decision alone.

This study addresses these gaps through systematic evaluation. By developing and applying a security evaluation framework to two architecturally distinct distributed RAG systems, DRAG and Reliable-dRAG, this research provides evidence-based architectural guidance for responsible AI deployment.

1.3 Problem Statement

Despite growing interest in distributed RAG architectures, fundamental questions about the security and privacy guarantees of these alternative architectures remain unanswered. The primary problem is that **there exists no systematic framework for evaluating and comparing the security posture of distributed RAG architectures**, making it impossible to provide evidence-based recommendations for secure deployment.

While centralized LLM vulnerabilities have been extensively documented, distributed alternatives introduce novel attack surfaces that are not well understood. Organizations need to understand the relative security benefits and trade-offs between centralized approaches and distributed alternatives, yet no standardized methodology exists for comparing these fundamentally different approaches under consistent threat models and evaluation metrics.

Key open questions include:

- How do integrity, confidentiality, and availability attacks behave in distributed RAG systems, and does the same attack class manifest differently across architecturally distinct systems?
- What are the privacy-utility and security-cost trade-offs across different architectural choices?
- How effective are defense mechanisms designed against these attacks, and does that effectiveness hold as attack severity changes?
- Which attacks pose the greatest threats to which architectural patterns, and which of those attacks have no equivalent in a centralized deployment?

This research addresses these questions by developing and applying a comprehensive security evaluation framework to two distributed RAG systems, DRAG and Reliable-dRAG, as representative instantiations of the architecture.

1.4 Objectives

The primary objective of this thesis was to **develop and apply a comprehensive security evaluation framework for distributed RAG systems, and to evaluate that framework against two architecturally distinct systems, DRAG [9] and Reliable-dRAG [21].**

Core Research Goals

1. **Framework Development:** We formulated a seven-step evaluation procedure (Section 4.1.1) that characterizes a target system’s architecture, enumerates its trust assumptions, and maps a derived set of attack classes onto the components that carry each assumption. We further derived six attack classes directly from the components a distributed deployment introduces beyond a centralized one, two per property of the CIA triad, confidentiality, integrity, and availability (Section 4.1.2).
2. **Empirical Security Assessment:** We implemented and evaluated all six attack classes, data poisoning and source selection manipulation (integrity), knowledge extraction and membership inference (confidentiality), and selective forwarding and denial of service (availability), against both DRAG and Reliable-dRAG.
3. **Comparative Architecture Analysis:** We compared the two systems’ security posture under a shared metric set and severity rubric, identifying where the same attack class manifests at markedly different severity, why architectural redundancy rather than defense quality explains some of those differences, and which attack classes have no equivalent in a centralized deployment.
4. **Defense Mechanism Design and Evaluation:** We designed six defense mechanisms, one per attack class, obtained by applying the evaluation procedure’s final step to our own findings against DRAG, and measured each defense’s recovery and cost.

5. **Methodological Contribution:** We documented a reproducible evaluation protocol, parameter sweeps and monotonicity checks on DRAG, multi-seed repetition on Reliable-dRAG, and demonstrated that the same seven-step procedure and six attack classes transfer across two systems that share little beyond both being distributed RAG deployments.

Among the findings this process produced, Section 4.1.4 documents the discovery of Grounding-Farming, an attack against Reliable-dRAG’s on-chain reliability mechanism that requires no privileged access and was not obtained by adapting any attack reported in the reviewed literature (Chapter 2). Reliable-dRAG’s own deployed defense against reliability-score manipulation proved structurally unable to detect it, since that defense validates transaction properties, sender identity, update magnitude, and caller authorization, that Grounding-Farming never violates.

1.5 Methodology in Brief

This thesis employed an empirical evaluation methodology combining architectural analysis, threat modeling, attack and defense implementation, and comparative assessment. Full methodological detail, including the seven-step evaluation procedure, the derivation of the six attack classes, and the design of each attack and defense, is presented in Chapter 4; this section summarizes the approach at the level needed to understand the objectives above.

Research Approach

System Setup: We evaluated two existing, publicly available distributed RAG systems rather than building a new one: DRAG [9], a peer-to-peer network in which peers advertise topic expertise and route queries through a Topic-Aware Random Walk, and Reliable-dRAG [21], which submits every query to three independent data sources and reranks their responses using a reliability score secured on an Ethereum-compatible smart contract. The two systems were selected for their architectural distance from one another, since a second target closely resembling the first would exercise our framework’s mapping step considerably less than one that did not.

Attack Implementation: We derived six attack classes from the architectural components that a distributed deployment introduces beyond a centralized one (Section 4.1.2), two attacking each property of the CIA triad:

- **Data Poisoning:** Injecting corrupted content into a participant’s knowledge store so that poisoned material is retrieved and degrades answer quality.
- **Source Selection Manipulation:** Falsifying the signal a system uses to decide which participant answers a query, directing queries toward a source holding no relevant knowledge.
- **Knowledge Extraction:** Reconstructing a system’s private document collection by systematically querying it through its own legitimate interface.

- **Membership Inference:** Determining whether a specific record exists in a system’s knowledge base by analyzing query responses, without retrieving the record directly.
- **Selective Forwarding:** Withholding answers at a compromised participant that otherwise appears healthy and reachable.
- **Denial of Service:** Exhausting the processing capacity distributed across a system’s participants.

Each class was instantiated separately against DRAG and against Reliable-dRAG, since the same class exploits a different mechanism in each architecture; Section 4.2 details every instantiation and the standardized interface each attack was implemented behind.

Evaluation Framework: The metrics applied across the evaluation fell into four categories:

- **Integrity and Availability Metrics:** Exact Match, F1, and Semantic Similarity degradation relative to a clean baseline, together with query hit rate and availability, for attacks that affect the answer a user receives.
- **Confidentiality Metrics:** Extraction Rate, Chunk Recovery Rate, Semantic Similarity, and Effective Edit Distance for knowledge extraction; Accuracy, TPR, FPR, Precision, and AUC-ROC, combined into a composite Privacy Risk Score, for membership inference.
- **Cost and Overhead Metrics:** Average hops and messages per query, TTL exhaustion rate, and the message and routing overhead each defense mechanism adds over its undefended baseline.
- **Severity Classification:** A shared five-tier rubric, Negligible through Critical (Section 5.1.2), mapping each metric above onto a common scale so that results from different attack classes and different systems could be compared directly.

Empirical Evaluation: DRAG’s [9] evaluation swept parameters, network size, peer connectivity, poison ratio, topic claim ratio, and attack intensity, at a fixed random seed, and treated a monotonic response across a sweep as its evidence standard. Reliable-dRAG’s [21] deployment exposes no comparable topology parameters, so its evaluation instead repeated each experiment across multiple random seeds (0, 42, and 123, with a further five held-out seeds for the membership inference composite) and reported the mean and spread. Every attack except selective forwarding was evaluated against both systems using the datasets each system’s own authors shipped with it: MMLU, Medical Extended, and HuffPost News for DRAG; SQuAD-derived and PubMedQA-derived corpora for Reliable-dRAG. Selective forwarding degrades retrieval before generation occurs, so we evaluated it against a network model with synthetic knowledge availability instead, and Section 4.4 records that choice and its consequence for the result.

Defense Design and Comparative Analysis: Following the seventh step of the evaluation procedure, we designed and evaluated six defense mechanisms against our own findings for DRAG. Three draw on established theoretical lineages rather than ad-hoc rules: cross-peer validation, used against both data poisoning and source selection

manipulation, is a Byzantine-quorum-style majority vote over independently polled respondents, the same consensus principle DeRAG applies at its blockchain layer [10] but implemented here directly at the query-routing layer; the reputation-based selective-forwarding defense maintains an exponentially weighted moving average of each peer’s response behaviour, in the trust-scoring spirit of the reputation systems the collaborative RAG literature has proposed for incentivizing honest participation [16], applied here to detect adversarial withholding rather than low contribution; and privacy-preserving retrieval injects calibrated Laplace-mechanism noise into retrieval similarity scores, in the same differential-privacy lineage as prior DP-RAG work [12], applied to a similarity score rather than to top- k document selection. The remaining three mechanisms, topic-claim quota enforcement, layered extraction prevention, and layered congestion defense, are described in full in Section 4.2.7. Defense work was scoped to DRAG; Reliable-dRAG’s results are reported undefended throughout, and Chapter 5 records this asymmetry as a limitation rather than an oversight. The two systems were then compared on degradation relative to each system’s own clean baseline and on the severity tier that degradation falls into, since their differing corpora make raw metric values incomparable directly.

Experimental Environment: DRAG experiments used a Docker-based simulated peer-to-peer network of ten to twenty peers running local Llama 3.2, Qwen 2.5, or Gemma 2 instances. Reliable-dRAG experiments used Flask-based data-source microservices, each backed by its own FAISS index, an LLM orchestrator, and a local Hardhat Ethereum-compatible node running the system’s smart contract. All datasets and models used were open-source and run on local or lab compute, so the evaluation incurred no per-query API cost or dataset licensing fee.

1.6 Scope and Challenges

What This Study Covers

This research focused on the security and privacy evaluation of distributed RAG architectures, evaluated concretely against two deployed systems, DRAG [9] and Reliable-dRAG [21]. The scope includes:

Technical Focus:

- Development of a seven-step evaluation procedure and a CIA-triad-derived set of six attack classes, applicable to distributed RAG architectures generally
- Security assessment of two architecturally distinct distributed RAG systems
- Implementation and evaluation of all six attack classes, spanning integrity, confidentiality, and availability
- Design and evaluation of six defense mechanisms, one per attack class, against DRAG
- Characterization of privacy-utility and security-cost trade-offs, including the routing and message overhead each defense mechanism imposes

Evaluation Scope:

- Attack classes: data poisoning, source selection manipulation, knowledge extraction, membership inference, selective forwarding, and denial of service
- A shared severity rubric and metric set, adapted per architecture but held fixed across both systems
- Performance evaluation (retrieval accuracy, response quality, routing and message overhead)
- Parameter sweeps on DRAG and multi-seed repetition on Reliable-dRAG, exploring how configuration affects security posture

Architectural Coverage:

- Empirical focus on two distributed RAG systems: DRAG (peer-to-peer, gossip-style routing) and Reliable-dRAG (blockchain-secured reliability scoring)
- Comparison against each system’s own clean, unattacked baseline
- Analysis of decentralized knowledge-sharing mechanisms and the trust assumptions each introduces

What Is Out of Scope

To maintain feasibility within thesis constraints, the following were explicitly excluded:

- Building novel distributed architectures from scratch; both DRAG and Reliable-dRAG were adopted as published by their respective authors
- Federated RAG systems: a third system, FedRAG, was considered during early planning and explicitly descope to protect the depth of evaluation on the two systems already integrated (Chapter 3, Risk Management); federated learning is discussed only as related background in Chapter 2
- Attacks directed at the language or retrieval model itself (prompt injection, jail-breaking, adversarial embedding perturbation, training-data extraction), which are not consequences of distribution and fall outside the framework’s scope by construction (Section 4.1.3)
- Sybil attacks, collusion between sources, and cross-source correlation, which compose over or extend the six evaluated classes rather than falling within them (Section 4.1.3)
- Production-grade system implementation and large-scale deployment testing with thousands of participants
- Hardware-level security considerations, legal and regulatory compliance analysis, and user studies or human-subjects research

Key Challenges

Technical Challenges

Adapting a single attack class to two architecturally distinct systems, one a peer-to-peer overlay, the other a set of microservices behind a blockchain, required re-deriving the mechanism each attack exploits rather than reusing an implementation. Open-source distributed RAG implementations were limited in availability; DRAG and Reliable-dRAG were the only two publicly available systems identified during the literature review, which is itself part of the gap this thesis addresses.

Practical Challenges

Resource constraints limited the extent of some experiments, most visibly the seven-question sample used for Reliable-dRAG’s data poisoning evaluation (Section 5.4.1). Establishing reproducibility in systems with non-deterministic behavior required repeated trials rather than single runs, since an identical seed and configuration produced opposite outcomes for the Grounding-Farming attack across two independent runs (Chapter 3, Risk Management).

Methodological Challenges

Establishing fair experimental conditions across two fundamentally different architectures required different evidence standards for each: parameter sweeps and monotonicity on DRAG, multi-seed repetition on Reliable-dRAG. Findings from specific configurations do not necessarily generalize to all settings, and Chapter 5 characterizes those boundaries explicitly rather than presenting a single generalized claim. Security evaluation depends heavily on threat-model assumptions, and where an attack’s privilege requirement exceeded what a class’s stated threat model describes, such as the Key-Forgery instantiation against Reliable-dRAG, we reported it as a secondary rather than primary finding.

Chapter 2

Literature Review

This chapter is organized as follows. Section 2.1 establishes the background concepts needed to follow the studies discussed afterward. Section 2.2 reviews those studies in detail and closes with a comparative table. Section 2.3 draws the review together and states the gap this thesis addresses.

2.1 Preliminaries

Large Language Models and Institutional Knowledge

Large Language Models (LLMs) are transformer-based neural networks trained on large volumes of natural text so that they can understand and generate human-like language. Models such as GPT-4, Claude, LLaMA, and Gemini already sit at the core of many production applications. Deploying one of them against a specific institution’s documents, however, means giving the model access to material that institution holds. The conventional way of doing this collects and centralizes that material, and centralization is precisely what introduces the risk: a single repository becomes both a more attractive target for unauthorized access and a single point of failure if it is compromised [1, 3, 6, 8–10].

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) enhances an LLM by pulling in external knowledge at inference time instead of depending only on what the model learned during training [3, 4]. This directly targets three well-known weaknesses of standalone LLMs: outdated information, hallucination, and the absence of domain-specific or personal knowledge that training data never covered [3]. A typical RAG pipeline is built from the following stages.

- **Document Vectorization:** Text documents are converted into dense vector embeddings by an embedding model such as Contriever, BGE, or a sentence-transformer variant. Because these embeddings capture semantic meaning rather than surface wording, similar items can be retrieved by comparing vectors directly [2, 6].
- **Vector Database Storage:** The resulting embeddings are held in a vector store, ChromaDB, FAISS, Pinecone, or Milvus among the common choices, that supports Approximate Nearest Neighbor (ANN) search. Indexing structures such as HNSW (Hierarchical Navigable Small World graphs) and IVF (Inverted File Index) let these stores run similarity search over millions of vectors quickly [2, 9].
- **Query Processing and Retrieval:** Incoming queries are embedded with the same model used to encode the documents, which keeps query and document vectors

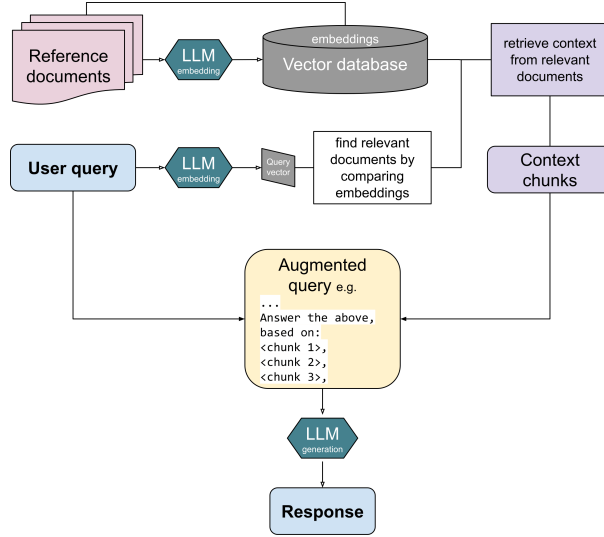


Figure 2.1: RAG System Workflow

semantically compatible. Relevant fragments are then retrieved through a similarity search, most often cosine similarity or inner product [5, 13].

- **Context Augmentation and Re-ranking:** The fragments a similarity search returns are often over-inclusive and unevenly relevant, which is why many pipelines add a second-stage re-ranker before generation. Where the initial search scores each candidate independently, a cross-encoder or a lightweight learned ranker scores every candidate jointly with the query, correcting the cases where a semantically close but tangential passage would otherwise outrank a shorter, more directly relevant one. Some re-ranking stages go further and enforce diversity, down-weighting redundant passages that repeat the same fact, or incorporate metadata such as recency and source trust. C-FedRAG reports a measured accuracy gain from adding such a stage on top of embedding-only ranking, while Reliable-dRAG blends it with an on-chain reliability score rather than relevance alone [2, 21].
- **Response Generation:** Finally, the LLM generates a response conditioned on the augmented prompt and the retrieved evidence. This grounding step is what curbs hallucination and lets the model surface recent, domain-specific, or personalized information it never saw during training [9, 18].

Together, these stages let a deployment update its knowledge dynamically, without re-training the underlying model, while also reducing hallucination [3].

Federated and Distributed Approaches

Federated Learning lets multiple participants train a shared model cooperatively while each keeps its own data local, exchanging only secure aggregates rather than raw records [2, 4]. Applied to RAG, this means organizations maintain their own local knowledge bases but combine them into a form of collective intelligence, governed by whatever information-sharing rules each participant sets for itself [2, 6].

Distributed Systems, by contrast, partition knowledge across participants that each

operate independently. The literature splits into two recognizable patterns here: a peer-to-peer overlay in which peers must discover one another and route queries across multiple hops [8, 9], and a fixed set of independent sources sitting behind a coordinating service that decides which of them to consult [21]. This thesis works with both patterns, and it is precisely the architectural distance between them that the evaluation in Chapters 4 and 5 is designed to exploit.

Attack Vectors

The literature converges on five broad attack vectors against RAG systems.

- **Membership Inference Attacks (MIA):** An adversary tries to determine whether a specific document sits inside a retrieval database, working from model behavior, statistical confidence scores, and retrieval patterns rather than direct access. The premise is the same one that underlies classical membership inference against trained models: a system that resolves a genuine match differently from a near-miss is, by construction, leaking a distinguishing signal through its retrieval confidence and routing behavior. In aggregate, such attacks can reveal whether sensitive documents exist in a knowledge base at all [3, 5, 14].
- **Knowledge Extraction Attacks:** Here the adversary attempts to reconstruct or extract verbatim content from a retrieval database, typically through carefully sequenced queries paired with close analysis of the generated responses. The attack is possible in the first place because the retrieval layer answers any well-formed query without checking whether the requester is authorized to see what comes back; RAG systems then compound the problem by tending to reproduce retrieved content verbatim, which can expose sensitive, proprietary, or confidential material [3, 7].
- **Data Poisoning:** A malicious actor inserts documents engineered with specific triggers, biases, or falsehoods into the knowledge base, hoping to skew the system’s behavior on selected queries. This works wherever the retrieval layer ranks candidates on embedding similarity alone, with no notion of provenance or trust weighting to flag a suspicious or duplicated entry. Once the LLM retrieves the poisoned material, it may respond to certain queries with harmful, biased, or simply wrong information, and can even leak information in the process [5, 7, 13].
- **Query Pattern Exploitation:** Some adversaries instead watch routing decisions, retrieval behavior, and response patterns over time, using them to infer information about the knowledge base, user preferences, or underlying data distributions [5, 9].
- **Adversarial Node Attacks:** Federated and distributed deployments introduce a further class specific to their multi-party structure: a malicious participant can feed corrupted information into an aggregation step, probe other nodes for private data through strategic queries, run Sybil attacks to manipulate aggregation mechanisms, or, in federated learning specifically, exploit gradient information to reconstruct another participant’s private training data [6, 17].

These five vectors describe the attack surface a RAG pipeline presents in general terms, and the literature largely reasons about them at the level of a single retrieval system. Distributing that pipeline across independently operated participants removes none of

them; what it does is change how two of them behave and give rise to two more that have no single-system equivalent. Chapter 4’s six-class taxonomy is derived directly from the components distribution introduces, rather than adopted wholesale from this list. Data Poisoning and Knowledge Extraction both survive distribution as recognizable classes, but neither survives unchanged: poisoning now scales its attack surface with the number of sources and rewards an adversary for choosing carefully which one to compromise, while extraction picks up a dependence on network topology that Section 5.2.3 measures directly. Membership Inference gains a second, network-level signal once queries must be routed across peers instead of answered from a single index. Query Pattern Exploitation and Adversarial Node Attacks, treated only qualitatively across the studies reviewed here, are the general precursors from which Chapter 4 derives two classes specific to the distribution layer itself: Source Selection Manipulation, which falsifies the routing-layer signal a system relies on to pick a source, and Selective Forwarding, in which a participant stays reachable while quietly declining to answer. Denial of Service, finally, extends the familiar threat of resource exhaustion to a setting where capacity is spread across many participants instead of concentrated in one. None of the twenty-one studies reviewed here carries out this derivation itself, and none evaluates the resulting classes against more than one architecture under a shared metric set, a gap the comparison table at the end of Section 2.2 returns to directly.

Defense Mechanisms

The privacy-preserving mechanisms proposed for RAG systems fall into five broad families.

- **Cryptographic Protections:** Homomorphic encryption, secure multi-party computation, and trusted execution environments such as Intel SGX or NVIDIA FLARE all appear in the literature as ways of protecting data in use, not just in transit or at rest [2, 6, 11].
- **Differential Privacy:** A mathematical framework that injects calibrated statistical noise, commonly via a Laplace or exponential mechanism, into retrieval scores, document selections, or generated responses, in exchange for a formal, provable privacy guarantee [12, 13].
- **Access Control and Filtering:** A layered set of controls: query validation to catch malicious inputs, input sanitization to strip harmful prompts, and output filtering to stop sensitive information from leaking back out in a response [3, 11, 13].
- **Data Protection Techniques:** Preprocessing methods applied before data ever reaches the pipeline, including anonymization, pseudonymization, synthetic data generation, and summarization [11, 13].
- **Architectural Defenses:** System-level measures: federated aggregation protocols that combine model updates or retrieval results without exposing individual contributions, decentralized mechanisms that split a query across multiple nodes, distributed knowledge bases that remove single points of failure, symbol substitution that swaps sensitive entities for placeholders before cloud processing, and consensus-style validation. This last mechanism draws on the same Byzantine-quorum princi-

ple underlying blockchain-secured RAG designs [10], trusting an answer only once a majority of independently polled participants agree on it [6, 8–10].

Each of these defenses targets one specific trust assumption that some attack exploits; none of them functions as a general-purpose safeguard on its own. That is also why Chapter 4 evaluates its own six defenses independently against the attack class each one answers: isolating a mechanism’s marginal effect this way is only possible if it is not bundled into a single security layer whose individual contributions cannot be disentangled afterward.

Foundations from Distributed Systems Security

Two of the classes Chapter 4 derives trace back further than the RAG literature itself. A participant that stays reachable while discarding the traffic routed through it is the selective forwarding attack Karlof and Wagner first characterised in sensor networks [22], and the reputation scoring this thesis applies against it follows EigenTrust, which infers trust from observed behaviour rather than declared identity [24]. Douceur’s analysis of identity fabrication [25] is why Chapter 4 treats Sybil attacks as a multiplier applied across the six classes rather than as a seventh class in its own right. Castro and Liskov’s quorum construction [26] supplies the majority-agreement principle behind the cross-peer validation defense this thesis applies at the query-routing layer. Distributed RAG inherits these problems together with the architecture that produces them, and reading them solely through the RAG literature would obscure where each one actually originates. These four are therefore treated as foundations rather than as reviewed studies in their own right; Section 2.2 and Table 2.1 are reserved for the RAG security literature this thesis positions itself against, and deliberately exclude work that predates it.

Evaluation Metrics and Datasets

Evaluations of privacy-preserving RAG systems tend to draw on a recurring set of metrics, grouped here by what each one measures.

Security Metrics: Attack Success Rate (ASR) captures how often an attack succeeds in breaching the system. Membership inference accuracy captures how reliably an attacker can tell whether a given document sits inside the retrieved set. Knowledge extraction rate captures how much of the knowledge base an outside adversary can reproduce. Detection rate captures how effectively the system catches and blocks malicious queries or actions before they succeed [3, 5, 7, 14, 15].

Privacy Metrics: Information leakage quantifies how much sensitive information escapes through any available channel. Query pattern exposure scores gauge how vulnerable a system is to traffic analysis and deanonymization [5, 9, 12, 13].

Utility Metrics: Retrieval quality is captured by the standard information-retrieval trio of precision, recall, and F1, together with Mean Reciprocal Rank (MRR). Generation accuracy captures the correctness of what the LLM outputs, and response relevance captures how well that output actually addresses the user’s query [1, 2, 6].

Performance Metrics: End-to-end latency is the total time from query submission

to delivered response. Throughput is the number of queries the system can process per unit time. Communication overhead tracks bandwidth consumption across the network, and scalability characteristics describe how performance degrades as the number of participants, documents, or queries grows [8–10, 18].

Privacy-Utility Trade-offs: These metrics quantify how much utility a system gives up for a given privacy guarantee, expressed through ϵ values, encryption strength, or noise level, and Pareto frontier analysis identifies the configurations where tightening privacy further would necessarily cost utility [1, 5, 12, 13].

The evaluation datasets recurring across this literature include medical QA benchmarks (BioASQ, PubMedQA, MIRAGE), general-knowledge benchmarks (Natural Questions, MMLU), enterprise datasets drawn from financial and legal documents, and assorted domain-specific datasets built for particular application contexts [1, 2, 9, 14, 17, 18].

2.2 Review of Existing Research

This section reviews the RAG security literature against which this thesis positions its contribution. Foundational work from distributed systems security, which predates RAG and supplies the lineage of two attack classes and two defenses, was already covered in Section 2.1 rather than here.

Chakraborty et al. [1] surveyed 18 primary studies on federated RAG systems published between 2020 and 2025 and found the field growing quickly, organized around three architectural categories: federated retrieval, federated generation, and privacy/security enhancements. Roughly half of the surveyed work still treats privacy and security as the primary concern. Healthcare was the most prominent application domain, ahead of finance, legal services, and enterprise deployments; notable systems included ones that answer clinical questions while keeping electronic health records in-hospital, enterprise assistants that avoid centralizing proprietary documents, and a routing system that cut redundant cross-silo queries by 75% for finance workloads. The review also surfaced four persistent open problems: scalable index synchronization across silos, meta-learned retriever adaptation that survives concept drift, privacy-utility evaluation protocols under realistic attacker models, and live benchmarks tracking cost and latency as documents evolve. Standardized datasets and metrics that account for client relevance, privacy constraints, and network cost are still missing, which makes comparing across studies difficult.

Iyengar et al. [4] conducted a systematic analysis of security and privacy vulnerabilities across LLMs more broadly, building a framework for understanding and mitigating deployment risk. The work argues that traditional AI security approaches are not well suited to LLM-specific challenges, and addresses this through a systematic literature review, an adaptation of the OWASP framework to LLM applications, and STRIDE-based threat modeling applied to a healthcare case study. Privacy-preserving techniques evaluated include differential privacy, federated learning, and RAG. Vulnerabilities identified include prompt injection, training-data poisoning, data leakage, and model denial of service, and the proposed mitigations span LLM firewalls that combine rule-based filtering with semantic analysis, differential privacy and federated learning to avoid centralizing

training data, human-in-the-loop validation for high-stakes applications, and OWASP-based guidance on output handling and access control. The healthcare case study showed these measures could be implemented while maintaining HIPAA and GDPR compliance. That said, the authors flag several open concerns: current strategies may not hold up against more sophisticated future attacks such as advanced adversarial techniques or model theft, universal standards for LLM transparency and security enforcement do not yet exist, bias evaluation at the semantic and contextual level remains underdeveloped, and adaptive security frameworks capable of tracking emerging threats are still needed, alongside further work on secure multi-party computation and homomorphic encryption for LLM training and on supply-chain vulnerabilities from third-party integrations.

Zeng et al. [3] examined RAG privacy from two angles at once: vulnerability of the retrieval data and exposure of the LLM’s own training data. They built a structured prompting-attack framework that combines an information component and a command component to extract sensitive records from a retrieval database, and tested it on the Enron Email dataset and HealthCareMagic medical dialogues against GPT-3.5-turbo and Llama, finding severe vulnerability: up to 50% of prompts extracted verbatim records. The more surprising finding was a paradox: while RAG opens a new vulnerability for retrieval data, it simultaneously shields training data, cutting successful training-data extractions from over 200 down to fewer than 40, because retrieval augmentation reduces the model’s reliance on memorized training data in the first place. Among mitigations tested, abstractive summarization cut privacy risk by roughly 50% against untargeted attacks, while re-ranking barely helped. The authors conclude that protecting retrieval data specifically remains an open problem.

Arzanipour et al. [5] built a formal threat model for RAG systems, laying down foundational security definitions and attack characterizations. Their taxonomy classifies adversaries along two axes, model access (black-box vs. white-box) and adversarial knowledge (normal vs. informed), yielding four adversary categories with distinct capabilities, and they give formal mathematical definitions for document-level membership inference, content leakage, and data-poisoning attacks specific to RAG architectures. Their formal analysis shows that different RAG pipeline stages are exploited by different attacks, and that context injection is the single most effective attack stage. Proposed defenses include retriever-level differential privacy with formal guarantees, elimination of position bias, adversarial training against content leakage, and scoring-based filtering against poisoning. The authors themselves caution that real-world effectiveness of these defenses remains uncertain, and that the formal definitions do not account for attack mechanisms that continue to evolve in a fast-moving RAG ecosystem.

Addison et al. [2] built C-FedRAG, a practical confidential federated RAG system aimed at healthcare. Its three-part architecture consists of data providers that vectorize and retrieve locally, an orchestrator running inside a confidential computing environment that aggregates contexts and generates responses, and security mechanisms, mutual TLS for encrypted communication and trusted execution enclaves that protect raw contexts during processing. On medical QA benchmarks (BioASQ and PubMedQA), C-FedRAG reached 72.51% average accuracy, beating centralized RAG baselines on BioASQ and matching them on PubMedQA while drawing securely on more diverse data sources. Re-ranking mattered: it added roughly 4% accuracy over embedding-only ranking. The system was

tested only in medical domains, though, and the authors flag several gaps: no threat modeling against data poisoning or adversarial attacks, unresolved key management for encrypted contexts, and no support for multi-modal data.

Mao et al. [6] built FedE4RAG to improve RAG retrieval models through federated learning without sacrificing privacy. Three components do the work: contrastive-learning-based RAG fine-tuning on synthetic query-chunk pairs, knowledge distillation between global and local embeddings to handle non-IID data, and homomorphic encryption to protect model parameters in transit. On financial-domain datasets, the system reached Hit@1 of 73.00% versus 70.00% for centralized training, and an MRR of 55.39, outperforming even a centralized baseline trained on aggregated data, an effect the authors attribute to the federated knowledge-distillation step. Integrating the retriever with different LLMs held up well downstream, with GPT-4o Mini reaching a 24.40 ChrF score. Still, the evaluation used only 5 clients, so scalability to hundreds or thousands remains unverified, the threat model assumes honest-but-curious participants without accounting for malicious adversaries or collusion, and questions around network heterogeneity, client dropout, and enterprise integration are left for future work.

An et al. [19] designed Guardian Angel, a secure RAG framework that uses cloud-based LLM inference while protecting sensitive data through symbol substitution: pattern-based placeholders stand in for sensitive entities, with a privacy database mapping placeholders back to their originals, layered on a hybrid RAG pipeline that pairs edge-side vector databases (improved HNSW plus product quantization) with a cost model balancing latency against monetary cost. On BoolQ and QuALITY, symbol substitution did not degrade accuracy while still protecting sensitive data during cloud inference, and the hybrid setup beat both pure-local and pure-cloud alternatives on cost while adapting well across hardware configurations. Privacy guarantees remain only partial, though, since advanced inference attacks are still a risk, the system depends heavily on a reliable local privacy database, and the evaluation itself was limited in scope. The authors position the framework for healthcare, finance, and government services that need strong security guarantees.

Ammann et al. [13] produced a systematic risk-assessment framework identifying 11 distinct risks across the RAG pipeline (indexing, retrieval, generation) and mapping them onto 13 mitigation strategies. Threats such as blind-spot vulnerabilities, retrieval data leakage, embedding inversion, membership inference, knowledge corruption, and indirect prompt injection are each paired with an appropriate defense, spanning data protection (anonymization, pseudonymization, synthetic data), access control, system hardening, secure retrieval techniques, and infrastructure safeguards. The framework ties RAG-specific measures to general IT standards, NIST, OWASP, and ISO, giving organizations structured guidance for securing RAG deployments. It lacks empirical validation of how well the mitigations actually work, some defenses affect performance in ways that are never quantified, and the guidance itself has not been implementation-tested. The authors expect new attack vectors to keep emerging as the field matures.

Xu et al. [9] built DRAG, a distributed RAG framework that runs over peer-to-peer networks with no central coordinator. It decomposes multi-document RAG into parallel cloud and device processes using the Law of Total Probability, guided by a Topic-Aware

Random Walk (TARW) algorithm that uses an LLM for intelligent peer discovery; each peer keeps its own local knowledge base and LLM instance, avoiding both transmission and recomputation of document key-value caches. Across MMLU, Medical Extended, and News, and multiple LLMs, DRAG reached 83.90% Exact Match versus 85.73% for centralized RAG on MMLU, while cutting network traffic by roughly 50% relative to flooding approaches (6.87 vs. 10.91 messages). It also outperformed centralized RAG when knowledge bases were incomplete, holding 83.92% F1 while centralized systems dropped to 61.54% at 70% data availability. The authors are candid about remaining vulnerabilities: user-filtered knowledge snippets can leak sensitive information, the TARW algorithm is exposed to deanonymization through network traffic observation, and query topics themselves can reveal sensitive patterns. Performance also degrades as the network grows, F1 falling from 83.92% to 79.72% between 20 and 100 peers, knowledge quality is harder to police without centralized curation, and the lack of an incentive mechanism raises sustainability concerns if peers free-ride instead of contributing.

Yu & Sato [10] introduced DeRAG, a decentralized RAG system that integrates multiple independent data providers behind blockchain-based validation. Its four-layer architecture pairs a RAG-Optimized Pyth Consensus protocol for multi-source data validation with domain-specific validator networks running two-tier consensus, and distributed index structures built on Kademlia-style DHTs and IPFS-inspired content-addressable storage. The system scaled to 10,000 nodes with acceptable consensus times while holding retrieval precision at 0.92, recall at 0.88, and F1 at 0.90; content-addressable storage cut storage requirements by 63% while keeping data integrity at 99.99%. Decentralization, in other words, bought fault tolerance and data sovereignty without a performance penalty. The remaining limitations are a cold-start problem for new nodes with low initial reputation, temporary inconsistency during network partitions, and privacy-preserving computation capabilities that are still underdeveloped.

Li et al. [8] proposed EACO-RAG, a distributed architecture that leans on collaboration between edge nodes and cloud resources for cost-efficient RAG. It uses hierarchical collaborative gating driven by Safe Online Bayesian Optimization for real-time routing decisions, GraphRAG for structured knowledge-graph retrieval in the cloud, and periodic edge-dataset updates keyed off local query terms and distributed knowledge chunks. On Wiki QA and Harry Potter, EACO-RAG matched the accuracy of a 72-billion-parameter cloud system while cutting costs by up to 84.6% under relaxed delay constraints, and its collaborative gating mechanism reliably told simple queries from complex ones and picked strategies accordingly; edge-assisted retrieval also made the system less sensitive to update interval and chunk size, easing the need for frequent updates. The prototype is limited to four retrieval strategies, though, and the gating mechanism relies on general heuristics that would likely need dataset-specific tuning for other industrial settings.

Liu et al. [17] introduced DRAGON, a distributed RAG system that splits document retrieval and output generation collaboratively between cloud and edge devices. It addresses the KV-cache dilemma by storing and loading caches locally rather than transmitting them, uses speculative aggregation via a draft-then-verify approach to cut down on synchronization, and applies adaptive scheduling that picks the aggregation location dynamically based on compute, bandwidth, and acceptance rate. DRAGON delivered $1.9\times$ and $1.4\times$ improvements over standalone SLMs relative to the next-best baseline,

and stayed robust under varying network conditions, cutting per-token latency by 42.4%-49.5% relative to synchronized methods at 300ms network latency, with negligible time-to-first-token overhead. On the downside, symbol substitution and database operations both add latency, evaluation was limited to BoolQ and QuALITY, and high-volume real-world workloads remain untested.

Grislain [12] brought differential privacy (DP) into RAG systems to give formal guarantees against sensitive information leakage. The approach has two parts: a DP-compliant retrieval mechanism using the exponential mechanism for differentially private top- k selection, and a DP generation mechanism that applies a novel token-level aggregation to add privacy-preserving noise to response tokens. On synthetic medical documents, DP-RAG kept answers meaningful while preserving privacy, and accuracy actually improved when knowledge was spread broadly rather than concentrated in individual records, giving regulated fields like healthcare and finance a formal privacy guarantee to build on. The trade-off is that DP-RAG tends to suppress highly specific knowledge for privacy’s sake, which hurts accuracy on rare information, and tuning the DP parameters plus handling the efficiency cost at large scale both remain open problems.

Zhao [18] proposed FRAG, a federated approach to vector-database management that enables secure RAG across distributed silos. It runs federated queries across multiple vector databases, shares embeddings rather than raw data to minimize leakage, and wraps routing, retrieval, and aggregation into a single pipeline. The evaluation found retrieval quality comparable to centralized methods, with moderate latency overhead, while cutting the risk of data leakage; the system scales and looks promising for large-scale multi-party collaboration where silos need to stay intact. Federated retrieval is inherently slower than a centralized system because of the added communication overhead, though, the trust model still assumes some baseline trust among participants, data heterogeneity across organizations affects results, and security guarantees against more sophisticated attacks like membership inference remain limited; most of the validation so far is experimental rather than field-tested.

Jiang et al. [7] built CopyBreakRAG, a black-box extraction attack sophisticated enough to recover over 70% of chunks verbatim from a RAG knowledge base. Its agent-based framework generates adversarial queries and refines them dynamically through feedback, combining curiosity-driven exploration to diversify queries with reasoning-based exploitation to pull in neighboring chunks through contextual reasoning. The attack held up across multiple datasets (healthcare conversations, email text, narrative text), multiple LLMs, and commercial platforms including OpenAI GPTs and ByteDance Coze, and the chunks it extracted showed near-perfect semantic similarity and very low edit distance from the originals. It performs better against continuous, cohesive knowledge bases than fragmented ones, since it leans on associative reasoning. Pattern-matching defenses only partially mitigate the leakage and are largely bypassed by more advanced prompt-injection techniques, which the authors take as evidence of a significant intellectual-property exposure in current RAG deployments.

Gao et al. [15] developed DCMI, a differential calibration membership-inference attack aimed specifically at RAG systems. It exploits the fact that member-retrieved documents are far more sensitive to query perturbation than non-member documents: by swapping

adjectives and adverbs for their antonyms and calibrating the resulting response differences, the attack isolates the contribution of member-retrieved documents specifically. Tested across three adversary models, gray-box with probabilities, distribution-aware, and black-box with binary responses, the gray-box setting reached 97.42% AUC and 94.35% accuracy on Flan-T5 RAG. Even in the more realistic black-box setting, it exceeded 60% success against two production RAG deployments, Dify and MaxKB, a 10-20% improvement over the baselines tested. Among the proposed defenses, entity-relation extraction cut this back the furthest, though only to roughly 50% attack success, while confidence masking and paraphrasing each managed a more modest 4-20% reduction.

Feng et al. [20] introduced RAGLeak, a stealthy membership-inference attack purpose-built for RAG-based LLMs that determines whether a sample exists in the knowledge database without touching the model’s internals. It works under two realistic access models, grey-box (output probabilities available) and black-box (only inputs and outputs available), and its core technique, query cropping, splits a user query into a cropped input segment and a remaining segment used as ground truth. Accuracy exceeded 0.8 across every dataset and LLM tested, and the attack bypassed few-shot-learning prompt defenses that had driven a baseline Inquiry Attack down to near zero (0.074), instead maintaining 0.879 accuracy and 0.911 AUC on GPT-3.5/NQ. Interestingly, the black-box setting outperformed grey-box overall, reaching 0.890 accuracy and 0.909 AUC on GPT-3.5/HealthCareMagic-100k against 0.792 accuracy and 0.813 AUC in grey-box mode. Against the same two production systems Dong et al. targeted, RAGLeak’s black-box attack reached 74.15% accuracy against MaxKB and 61.90% against Dify, a similar 10-20% margin over its own black-box baseline. That two independently designed attacks converge on comparable production-system numbers, and a comparable defense ordering (entity-relation extraction strongest, instruction- or confidence-based filtering weakest at roughly 4-7%, paraphrasing in between at roughly 20%), is itself worth noting, since it suggests these are structural properties of Dify and MaxKB rather than artifacts of either attack’s particular design. RAGLeak’s own limitations include a text-only focus with little evaluation on non-textual modalities, perturbation strategies that would need rework for images and video, and linear $O(n)$ complexity that raises scalability questions at large scale.

Zhou et al. [11] presented Privacy-Aware RAG, which applies advanced encryption to protect both text content and embeddings across their entire lifecycle. Two complementary schemes do the work: AES-CBC-based encryption for efficient protection of document chunks and embeddings, and chained dynamic key derivation, which organizes encrypted data into linked lists with dynamically derived keys and trapdoor mechanisms for secure authentication. The authors provide formal security proofs showing the methods meet adaptive-chosen-plaintext-attack security and preserve integrity guarantees, and experiments confirm the encryption adds no significant overhead while blocking unauthorized access effectively. Scaling to very large datasets still needs more exploration, though, and the system currently assumes secure key management without examining broader ecosystem risks such as endpoint vulnerabilities.

Liang et al. [14] produced SafeRAG, introducing novel attacks that bypass modern RAG defenses and a Chinese-language RAG security benchmark built around realistic attack scenarios. Four attack types are identified: silver noise (partial answers that slip past

filters), inter-context conflict (conflicting external knowledge), soft advertising (implicit toxicity), and white DoS (false safety warnings). Testing across 14 RAG components showed every attack type degrading performance, even against advanced systems, with effectiveness ranked filtered context > retrieved context > knowledge base by stage. The benchmark uses attack-specific metrics, and shows Hybrid-Rerank retrievers holding up better against noise but becoming more vulnerable to conflict attacks; among the LLMs tested, Baichuan 13B was most resistant to DoS attacks, and some lighter models actually showed better safety than the GPT series in certain scenarios. The study is scoped to data-injection attacks only, limited to single-modal Chinese text, and uses a relatively small base sample of 100.

Muhamed, Diab & Smith [16] extended RAG into collaborative learning with CoRAG, where multiple clients jointly train a shared model over a collaboratively built passage store. Low-resource clients with little labeled data benefit from pooled collective knowledge while still keeping their valuable training data local. On the CRAB collaborative-QA benchmark, CoRAG consistently beat both local RAG models and parametric baselines, improving 33.8% over local RAG at 16-shot settings, and the passage-store analysis turned up some counterintuitive findings: relevant passages matter most for generalization, hard negatives hurt performance by confusing the retriever, and irrelevant passages are actually helpful, acting as easy negatives that make the retriever more robust. A design tension remains, though, between maximizing individual client benefit and contributing high-quality data to the shared pool; the authors formalize this as the CoRAG Participation Game and sketch potential incentive mechanisms, contribution-based rewards, tiered access levels, and reputation systems among them.

Lu et al. [21] developed dRAG (called Reliable-dRAG throughout this report to distinguish it from Xu et al.’s DRAG [9]), a decentralized RAG system that secures per-source reliability scores via blockchain-based smart contracts. Three components are kept separate: independently owned data sources exposing a standard retrieval API, an LLM service that samples sources by a learned usefulness score before reranking retrieved documents by reliability, and a smart contract, DragScores, maintaining a tamper-proof scoreboard signed by each data source. Sentence-level importance is estimated with Monte-Carlo Shapley values when no ground truth is available, or with a conditional V-information formulation trained via RORA when it is; document-level scores aggregate these values and update a source’s cumulative reliability and usefulness additively, rewarding sources that produced a correct, grounded answer and penalizing those that did not. Retrieval samples N sources by usefulness and reranks what comes back by blending semantic relevance with reliability. On Natural Questions with Llama-3.2-3B and Llama-3.1-8B across six data sources of injected, varying reliability, dRAG improved answer accuracy by 10.7% over a centralized baseline in unreliable data environments, closing in on the accuracy ceiling of a centralized system running on fully reliable data, and batching feedback transactions on-chain cut per-update gas cost by roughly 55.8%, from 2.6 cents at $n = 1$ to 1.1 cents at $n = 20$. Two limitations remain: performance falls well short of the centralized ceiling under document-level (rather than token-level) pollution, since the most reliable source may simply lack the passage a query needs, and the paper leaves unmeasured how quickly reliability scores converge as a function of query volume and source count.

Mu & Li [23] introduced Routing Hijacking, which targets the source-selection stage of federated RAG rather than its retrieval corpus. Because a federated server cannot inspect client corpora directly, it routes each query using a semantic profile the client uploads about itself; a client that builds that profile from target-domain proxy passages can therefore attract queries it has no real evidence for, and then return missing, harmful, or fabricated content to the generator. The authors tested the attack against three router families: an embedding-based router of the kind FRAG [18] and FedE4RAG [6] deploy, the RAGRoute neural router, and the ReSLLM natural-language source selector. With one malicious client among twenty, hijack rate at $K = 3$ ranged from 70.0% to 91.0% across three target domains, rising to 86.0%-95.0% with three malicious clients; against ReSLLM, three malicious sources registering a broad competence description reached 97.0%. Proxy ablations trace the effect to target-domain semantic signal rather than to the mere presence of malicious clients, since random non-target profiles never entered the routed set at all. A MedQA-USMLE case study showed forged medical profiles displacing honest medical sources, cutting honest-source access at $K = 1$ from 100% to 67.7%, after which every one of the five generator models tested produced incorrect, hallucinated, or sycophantic answers on the poisoned evidence, a 30-billion-parameter model still failing in 44% of cases. Defenses fared poorly too: CKKS-encrypted similarity search preserved the exploited ranking and offered no real protection, and Byzantine-robust aggregation rules borrowed from federated learning (Krum, coordinate-wise median, trimmed mean) transferred poorly to heterogeneous routing profiles. The authors’ own proposal, Trust-Aware Secure Routing (TASR), reweights each client by the relevance of the evidence it returns, that evidence’s consistency with its registered profile, and its agreement with other selected clients, bringing post-warmup hijack rate down to 0.0% on embedding routing and HR@3 down from 96.8% to 49.7% on RAGRoute. Three limitations bound the result: the evaluation covers a single attack class at a single pipeline stage; TASR needs a warmup window before its trust variables can separate honest from malicious clients, leaving early queries exposed; and the authors flag one adversary their design cannot address, a client returning evidence that is semantically relevant but false, for which external fact verification or provenance checks would be needed.

Comparative Summary of Reviewed Studies

Read one after another, the twenty-two studies above are hard to compare directly: they target different architectures, evaluate different attacks, and report defenses at very different levels of detail. Table 2.1 lays the same twenty-two studies out along four axes, architecture, attacks studied, defenses proposed, and application domain, so that the pattern running through the individual summaries becomes visible at a glance rather than only in hindsight.

Table 2.1: Comparative Summary of Reviewed Literature by Architecture, Attack Coverage, Defense Coverage, and Domain

Study	Architecture	Attacks Studied	Defenses Proposed	Domain
Chakraborty et al. [1]	Federated (mapping study, no single system)	— (secondary review)	— (secondary review)	Cross-domain (healthcare, finance, legal, enterprise)

Continued on next page

Table 2.1 – continued from previous page

Study	Architecture	Attacks Studied	Defenses Proposed	Domain
Iyengar et al. [4]	Generic LLM deployment (not RAG-specific)	Prompt injection, training-data poisoning, data leakage, model DoS	LLM firewalls, DP, federated learning, HITL validation	Healthcare (STRIDE case study)
Zeng et al. [3]	Centralized RAG	Structured-prompt extraction; training-data exposure	Abstractive summarization; re-ranking (limited effect)	Email, medical dialogue
Arzanipour et al. [5]	Centralized RAG (formal, architecture-agnostic)	MIA, content leakage, poisoning (formal taxonomy)	Retriever-level DP, adversarial training, scoring-based filtering	Generic
Addison et al. [2]	Federated (confidential computing)	None evaluated (threat modeling flagged as a gap)	TEE, mutual TLS, re-ranking	Healthcare
Mao et al. [6]	Federated	None evaluated (honest-but-curious only)	Homomorphic encryption, knowledge distillation	Finance
An et al. [19]	Hybrid edge-cloud	None evaluated (privacy guarantees only partial)	Symbol substitution, placeholder mapping	General (health-care/finance/gov. suitability claimed)
Ammann et al. [13]	Generic RAG pipeline (framework, no single system)	11-risk taxonomy (leakage, embedding inversion, MIA, corruption, injection)	13 mapped mitigations (no empirical validation)	Generic
Xu et al. [9] (DRAG)	Distributed P2P	None evaluated (vulnerabilities noted qualitatively)	None evaluated	MMLU, medical, news
Yu & Sato [10]	Decentralized, blockchain-validated	None evaluated	Consensus validation, DHT indexing	General
Li et al. [8]	Distributed edge-cloud	None evaluated (cost/accuracy focus)	None evaluated	Wiki QA, Harry Potter
Liu et al. [17] (DRAGON)	Distributed cloud-edge	None evaluated (latency focus)	None evaluated	BoolQ, QuALITY
Grislain [12]	Centralized RAG	None evaluated (formal DP mechanism only)	Differential privacy (exponential mechanism, token-level noise)	Synthetic medical
Zhao [18] (FRAG)	Federated vector database	None evaluated (leakage risk discussed qualitatively)	Embedding sharing instead of raw data	General, multi-party
Jiang et al. [7]	Centralized RAG (commercial platforms)	Agent-based knowledge extraction	Pattern matching (partially bypassed)	Healthcare, email, commercial platforms
Gao et al. [15] (DCMI)	Centralized RAG (production systems)	Membership inference (differential calibration)	Confidence masking, paraphrasing, entity-relation extraction	General, production (Dify, MaxKB)
Muhammed, Diab & Smith [16] (CoRAG)	Collaborative / federated-like	None evaluated (incentive design only)	N/A (participation-game incentive proposal)	QA benchmark (CRAB)
Feng et al. [20] (RAGLeak)	Centralized RAG (production systems)	Membership inference (query cropping)	Instruction-based filtering, paraphrasing, entity-relation extraction	General, production (Dify, MaxKB)
Zhou et al. [11]	Centralized RAG	None evaluated (encryption mechanism only)	AES-CBC encryption, chained key derivation	General
Liang et al. [14] (SafeRAG)	Centralized RAG (benchmark)	Silver noise, inter-context conflict, soft advertising, white DoS	Evaluated across 14 RAG components (no standalone defense proposed)	Chinese-language, general

Continued on next page

Table 2.1 – continued from previous page

Study	Architecture	Attacks Studied	Defenses Proposed	Domain
Lu et al. [21] (Reliable-dRAG)	Decentralized, blockchain- secured reliability	None evaluated (this thesis’s contribution)	Reliability-weighted retrieval (not attack-facing)	Wikipedia / Natural Questions
Mu & Li [23]	Federated RAG (three router families)	Routing Hijacking: profile forgery at the source-selection stage	TASR trust reweighting; HE routing and Byzantine-robust rules evaluated and found insufficient	StackExchange, FedWeb-2013, MedQA-USMLE

Three patterns stand out in Table 2.1 that are easy to miss when reading the studies in sequence. First, empirical attack evaluation against a running system is concentrated almost entirely on centralized deployments: four of the twenty-two studies attack production centralized systems (Zeng et al., Jiang et al., Gao et al., Feng et al.), and one attacks a purpose-built centralized benchmark (Liang et al.). Mu & Li [23] are the only study attacking a distributed architecture, and even their evaluation covers just one attack class at one pipeline stage. Every distributed, federated, or blockchain-secured system reviewed here, DRAG [9] and Reliable-dRAG [21] included, either discusses its own privacy exposures only qualitatively or leaves them for future work rather than measuring them directly. Second, not one study evaluates more than one attack class against more than one architecture under a single shared metric set, so nothing in the table tells us whether a confidentiality or availability exposure transfers across architectures the way a routing-stage integrity exposure evidently does. Third, the studies that do propose defenses either test them against a single attack on a single system (Gao et al., Feng et al.), list them without empirical validation at all (Ammann et al.), or evaluate them against just one attack class at one pipeline stage (Mu & Li). This intersection, six attack classes spanning all three properties of the CIA triad, evaluated under one framework against two architecturally distinct distributed systems, DRAG and Reliable-dRAG, neither of which received any attack evaluation of its own in the studies reviewed above, is precisely the gap this thesis closes.

2.3 Summary of Key Findings

Taken together, the literature shows federated and distributed RAG systems moving from conceptual proofs of concept to genuinely usable alternatives that match centralized performance while improving privacy, and doing so effectively across several sensitive domains. DRAG cuts network traffic by roughly 50% and still performs well even with incomplete knowledge bases. Reliable-dRAG takes a different route to a similar destination: rather than routing around a central authority the way DRAG does, it keeps data ownership distributed but secures per-source reliability scores on a blockchain, and its reliability-weighted retrieval beats a centralized baseline by 10.7% in unreliable data environments while approaching the accuracy of a fully reliable centralized system. That trust layer is not free, however; batching its on-chain feedback updates still costs roughly 1.1 to 2.6 cents per update depending on batch size, and its reliability scoring degrades under document-level (rather than token-level) data pollution, since the most reliable source may simply lack the passage a query needs. Edge-cloud collaboration claims an 84.6% cost reduction while preserving accuracy, and privacy-preserving RAG more broadly draws on confidential computing, homomorphic encryption, and differential pri-

vacy in combination.

Security, meanwhile, remains an unresolved and moving target across all of these architectures. Membership inference attacks reach success rates as high as 97% against black-box models, knowledge extraction attacks recover more than 70% of content verbatim, and query pattern analysis alone can enable deanonymization through nothing more than observed network traffic. What these vulnerabilities show, collectively, is that privacy protection needs integrated, multilayered strategies rather than isolated point fixes. The field still lacks standardized evaluation frameworks and benchmarks, and real-world deployment questions, incentive mechanisms, reputation systems, long-term sustainability, remain poorly understood. Architectural defenses help, but they can themselves introduce adversarial nodes or leak query patterns. Cryptographic protections are strong in principle but run into scalability and key-management issues in practice. Differential privacy offers a mathematically grounded guarantee, but at the cost of added operational complexity and reduced utility. Table 2.1 in Section 2.2 lays out this one-attack-one-system pattern architecture by architecture.

What the field needs next, then, is standardized benchmarks that account for privacy constraints, scalable index synchronization for federated systems, and comprehensive security evaluation frameworks built around realistic adversary models that hold across architecturally different systems, which is exactly the gap this thesis works to close by evaluating both DRAG and Reliable-dRAG under one shared framework. Closing it is what should ultimately provide the architectural insight organizations need to deploy AI and related technologies safely.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

Building and evaluating a security framework across two architecturally different distributed RAG systems imposes requirements at three levels: what the framework itself must do, what software and infrastructure each target system needs, and what data the experiments may draw on.

Functional requirements. The framework must cover all three legs of the CIA triad: data poisoning and source selection manipulation for integrity, knowledge extraction and membership inference for confidentiality, and selective forwarding and denial of service for availability. Metrics have to stay comparable across both systems even though every attack is re-derived per architecture. Membership inference on DRAG, for instance, uses network routing metrics as a discriminative signal, since queries traverse a multi-hop peer network differently depending on whether the target record is present; Reliable-dRAG offers no equivalent routing signal, so a decision-match signal takes its place instead. Availability attacks follow the same logic: DRAG’s evaluation targets a simulated peer network, while Reliable-dRAG’s containerized microservices are small enough to permit a real HTTP-level flood. What stays fixed across both adaptations is the framework’s core metric set and reporting format, so that the results remain comparable even where the underlying mechanism differs.

Software and infrastructure requirements.

- **DRAG:** A Docker-based simulated peer-to-peer network (10-20 peers) using a Barabási-Albert topology generator, the Topic-Aware Random Walk (TARW) routing implementation, and a local LLM instance per peer (Llama 3.2, Qwen 2.5, or Gemma 2).
- **Reliable-dRAG:** Flask-based data source microservices, each backed by its own FAISS vector index, a central LLM orchestrator service, and a local Hardhat Ethereum-compatible node running the `DragScores` smart contract.
- **Shared:** A Python-based attack/defense implementation layer, sentence-embedding models for semantic similarity metrics (used in Semantic Similarity and Effective Edit Distance calculations), and JSON-based logging of every attack and defense run to support reproducibility.

Data requirements. Every experiment draws only on public, non-proprietary benchmark data: MMLU, Medical Extended, and HuffPost News for DRAG; SQuAD and

PubMedQA for Reliable-dRAG. No real user data or personally identifiable information enters the work at any stage, in keeping with the declaration accompanying this report.

Reproducibility requirement. Several of our own experiments turned up measurable run-to-run variance, most visibly a Reliable-dRAG source-manipulation attack that produced opposite outcomes across repeated trials at the same configuration. We therefore require that every result rest on more than a single observation, with the specific form of that requirement depending on what the target system actually exposes. Where a system offers no controllable topology parameters, as with Reliable-dRAG’s fixed three-source deployment, we require a minimum of three independent seeds with the observed spread reported. Where a system exposes a parameter space, as DRAG does, we instead require a monotonic response across a swept range, since testing a claim at several operating points is a stronger check than repeating it once. Section 5.4 reports both standards and identifies the two sweeps that show reversals under them.

3.2 Project Management Plan

The project follows a phased schedule, with the output of each phase feeding directly into the requirements of the next.

Phase 1 – System Setup and Literature Synthesis: Set up the DRAG and Reliable-dRAG environments, pull evaluation metrics out of the literature review, prepare datasets, and define the threat model around the CIA triad together with each system’s access assumptions.

Phase 2 – Attack Implementation and Characterization: Implement the six attack categories against DRAG first, then adapt each to Reliable-dRAG’s architecture, documenting results and any methodological corrections as they come up.

Phase 3 – Framework Integration and Defense Design: Fold the attack results into a single evaluation framework spanning both systems, and design defense mechanisms against the attacks implemented in the previous phase, detailed further in Chapter 4.

Phase 4 – Comparative Analysis and Documentation: Finish the comparative analysis between DRAG and Reliable-dRAG to establish the security, utility and performance trade-offs between them, evaluate the remaining defenses, and write up the thesis. The phases carry planned overlap between them so progress stays continuous rather than strictly sequential.

Resource management. The project team consists of five students. Computational demand is centered on running local LLM inference (Llama 3.2/3.1, Qwen 2.5, Gemma 2) across repeated multi-seed trials on local and lab compute; since every model and dataset used is open-source, the project incurs no per-query API cost and no dataset licensing fee.

3.3 Societal Impact

Distributed RAG systems are proposed for exactly the domains where centralizing data is undesirable or legally constrained: healthcare, finance, legal services, and multi-tenant enterprise platforms, all of which turn up repeatedly among the systems surveyed in our literature review. Decentralization solves the privacy and compliance problems a centralized RAG deployment creates, but it does not remove risk so much as relocate it: this thesis’s own results show that the same query interface meant to serve legitimate users can be used to reconstruct a large share of a system’s private knowledge. Our Reliable-dRAG extraction results, run against a PubMedQA-derived corpus representative of the kind of medical knowledge base a hospital consortium might deploy, recovered 37–42% of the underlying document collection through nothing more than the system’s own legitimate query API. Surfacing this before deployment gives adopters in these sectors a way to audit a system’s actual security posture rather than discover it after a breach.

3.4 Environmental Impact

The project’s computational footprint comes from repeated local LLM inference across three model families, several seeds, and a range of attack and defense configurations, run on local and lab hardware rather than large-scale cloud training. This is bounded but not negligible, and we report it here rather than leave it out. The blockchain component in Reliable-dRAG runs on a local Hardhat node rather than a public chain, so it carries none of the energy cost associated with public proof-of-work networks; even a real-world deployment on a modern proof-of-stake chain like Ethereum would not incur that cost either. We do not attempt to quantify whether decentralized RAG reduces net energy use relative to the redundant central data centers a centralized alternative requires, since that comparison sits outside this thesis’s scope; we flag it here as an open question rather than an unsupported claim.

3.5 Ethical Issues

Implementing working attacks against real, functioning systems carries a dual-use concern: the same code that evaluates a defense could, in principle, be repurposed to attack an undefended deployment. We address this in four ways. First, every attack in this thesis runs only against sandboxed, locally-hosted Docker deployments under our own control; no production system is ever a target. Second, all corpora are public benchmark datasets (MMLU, SQuAD, PubMedQA, HuffPost News, and a synthetic medical dataset), so no real user data or personally identifiable information enters the work at any stage. Third, threat models that assume a compromised credential use test-only values rather than anything resembling a real secret. Reliable-dRAG’s Key-Forgery attack uses Hardhat’s documented default development keys as a deliberate stand-in for “attacker has compromised the orchestrator’s key,” not a claim that any production key is exposed. Fourth, we pair every attack class with a defense mechanism rather than publishing offensive results in isolation, following the principle that motivates coordinated vulnerability disclosure in industry practice.

3.6 Standards

No part of this project requires formal compliance certification, but our threat model and metric design deliberately align with existing security standards rather than inventing terminology from scratch. Ammann et al. [13], covered in our literature review, maps RAG-specific security measures onto NIST, OWASP, and ISO guidance; our CIA-triad-based attack categorization follows the same standard security-engineering decomposition used there, and several of our metric names, extraction rate, membership inference accuracy, privacy risk score, correspond to concepts defined in the OWASP Top 10 for LLM Applications, specifically its treatment of sensitive information disclosure and excessive agency. This alignment is a design choice meant to keep our results comparable to standards-informed work elsewhere in the field, not a certification claim.

3.7 Risk Management

Several risks materialized during this project rather than remaining hypothetical, which gives this section concrete grounding instead of a generic list.

- **Silent dataset drift between components.** Reliable-dRAG’s `data-source-0` was migrated from a SQuAD-derived corpus to a PubMedQA-derived one for an unrelated fix, while the knowledge extraction attack script still assumed all three sources served SQuAD. This made the attack 100% non-functional, failing before sending a single probe, until we caught it during analysis. *Mitigation:* verify every attack script’s assumptions against the live system’s actual served content rather than trusting an earlier configuration to still hold.
- **Non-deterministic attack outcomes.** An identical seed and configuration produced opposite outcomes for the Grounding-Farming attack on two independent runs (full escalation on one run, marginal on the other). A single-run result would have misrepresented the attack’s real success rate. *Mitigation:* require multiple independent trials per configuration and report the observed spread, not a single point estimate.
- **A theoretically appealing signal that broke in practice.** An early Reliable-dRAG targeting strategy selected sources by their blockchain reliability score, but that score never updates during a normal query, so every source tied at zero and the strategy always selected the same source regardless of actual reliability. *Mitigation:* replace it with a directly observable signal (live document count) once the failure was measured rather than assumed.
- **Schedule risk from architectural divergence.** Adapting all six attack categories across two structurally different systems, one peer-to-peer, one microservice-plus-blockchain, risks exceeding the time available before each milestone. *Mitigation:* the phased plan in Section 3.2 implements and validates each attack on DRAG first, then ports it to Reliable-dRAG, with overlap built into the schedule.
- **Scope creep from a third target system.** Early planning considered adding FedRAG as a third evaluated system. *Mitigation:* explicitly descope for the current phase to protect depth of evaluation on the two systems already integrated; revisited only as future work in Chapter 6.

3.8 Economic Analysis

The clearest quantifiable cost in this project is Reliable-dRAG’s own trust mechanism: securing reliability scores on-chain is not free, and Lu et al. [21] report a per-update gas cost that falls from 2.6 cents at a batch size of one to 1.1 cents at a batch size of twenty. This is the direct economic trade-off between the two systems we evaluate: DRAG’s peer-level trust model carries no comparable per-query fee but offers no cryptographic guarantee against a dishonest peer, whereas Reliable-dRAG’s blockchain layer adds a small, bounded, and shrinking-with-scale cost in exchange for a tamper-proof, auditable reliability record. On our own project’s cost side, every model and dataset used is open-source and run on local or lab compute, so the research itself carries no per-query API spend or dataset licensing fee. The value on the other side of that ledger, catching an extraction, poisoning, or availability vulnerability before a real deployment ships, is not something we can attach a specific dollar figure to without relying on an unverified industry estimate, so we state it qualitatively instead: the marginal cost of running this evaluation is close to zero next to the cost of discovering the same vulnerability after a real system is already in production.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The security evaluation framework proposed in this thesis is a structured methodology rather than a software artefact. It specifies a procedure by which an evaluator can assess the security posture of a distributed RAG system without prior familiarity with that system’s internals, in the same methodological sense in which STRIDE specifies a procedure for threat modelling without supplying an implementation of one. We use the term framework in this sense throughout the thesis.

The motivation for specifying a procedure, rather than a fixed suite of attacks, follows from the observation that distributed RAG systems share a family of security exposures that arise from the decision to distribute rather than from any individual design choice. Partitioning a knowledge base across independently operated participants yields benefits in privacy, cost and scalability, and it also introduces architectural components that a centralised deployment does not possess. Each of those components rests on an assumption about the honest behaviour of the participants it depends upon. The framework provides a systematic means of identifying those assumptions, testing each of them under adversarial conditions, and reporting which of them fail.

4.1.1 The Evaluation Procedure

Application of the framework to a target system proceeds in seven steps. The first three constitute the analytical stage and yield a concrete attack instantiation for each of the six attack classes; the remaining four convert those instantiations into comparable and actionable results.

Figure 4.1 sets out the procedure and the feedback loop its final step closes.

1. **Characterise the architecture.** The evaluator identifies four properties of the target system: the scheme by which knowledge is partitioned across participants, the mechanism by which the system determines which participant answers a given query, the signal on which that mechanism bases its judgement of participant trustworthiness, and the boundary separating the system’s internals from the interface exposed to an ordinary user.
2. **Enumerate the trust assumptions.** For each component identified in the preceding step, the evaluator states what the system must assume to operate correctly. A routing mechanism that consults self-declared topic expertise, for example, assumes that participants represent their own capabilities honestly.

3. **Map the attack classes onto the identified mechanisms.** Each of the six attack classes is defined independently of any particular architecture. Instantiating a class against a target system requires locating the component that carries the corresponding trust assumption in that system and constructing the attack against that component. A single class therefore yields materially different attacks in different architectures, which is the reason the framework specifies classes rather than implementations.
4. **Specify the threat model.** The evaluator states the privilege level each instantiated attack requires, distinguishing between an external client holding only query access, a participant operating one legitimate source, a compromised source under adversarial control, and an adversary in possession of stolen credentials. Attacks requiring insider access and attacks requiring none carry substantially different weight, and reporting them without this distinction overstates the severity of the weaker case.
5. **Measure.** The evaluator applies the metric set for the CIA property the attack targets. Section 4.2 defines each metric alongside the attack it scores, and Table 5.1 lists the assignment of metrics to classes.
6. **Assign severity.** Each measured outcome is mapped onto the shared severity rubric of Section 5.1, so that results obtained from different attack classes and different target systems may be compared on a common scale.
7. **Derive recommendations.** The evaluator identifies which trust assumption the successful attack violated, and what modification to the system’s design would close it. The six defence mechanisms reported in Chapter 5 were obtained by following this step against our findings for DRAG.

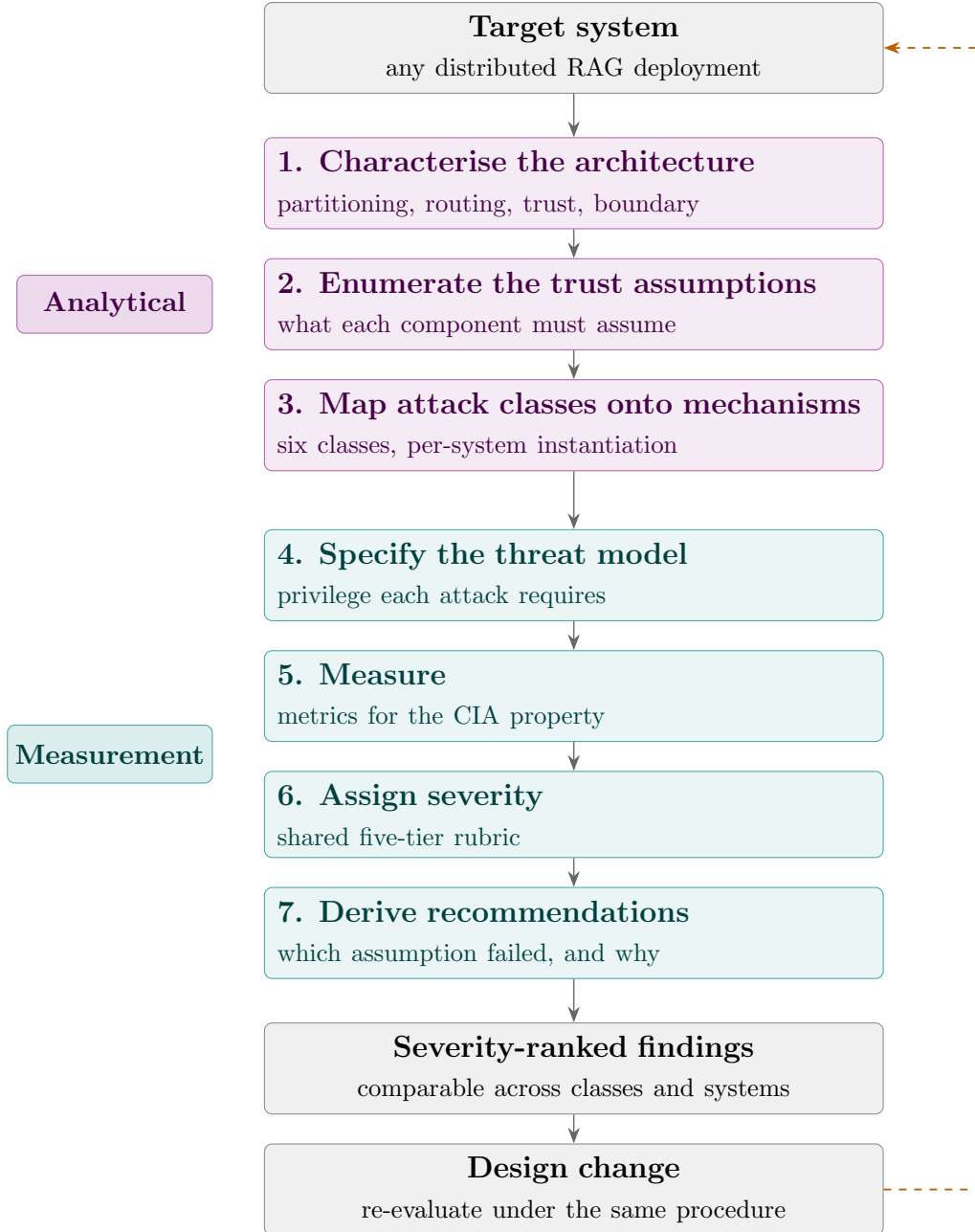


Figure 4.1: The seven-step evaluation procedure of Section 4.1.1. Steps 1 to 3 form the analytical stage and yield one concrete attack instantiation for each of the six classes; steps 4 to 7 convert those instantiations into comparable, severity-ranked results. The dashed return path is the evaluation-to-design feedback loop, whose output Chapter 5 reports as the six defence mechanisms of Section 5.3.

We applied the full seven-step procedure to two target systems. DRAG resolves queries by routing them through a peer-to-peer overlay in which peers advertise topic expertise, whereas Reliable-dRAG submits every query to three independent data sources and ranks their responses using a reliability score maintained on a blockchain. The two systems share little beyond the property of being distributed RAG deployments, and this architectural distance was a deliberate selection criterion: a second target closely resembling the first would have exercised the mapping stage of the procedure considerably less than one that did not.

4.1.2 Deriving the Attack Set

The six attack classes are derived by asking which architectural components a distributed RAG system possesses that a centralised one does not. A centralised deployment maintains a single knowledge base, answers queries from it, and requires no mechanism for selecting among sources because no such choice arises. Distributing that deployment introduces six components, each of which rests on a trust assumption available to an adversary. Table 4.1 and Figure 4.2 presents the resulting mapping.

Table 4.1: Derivation of the Attack Set from the Components Introduced by Distribution

Component Introduced by Distribution	Trust Assumption Carried	Attack Class	CIA Property
Multiple independent knowledge stores	The contents of each store are accurate	Data Poisoning	Integrity
A mechanism selecting which source responds	Participants represent themselves honestly	Source Selection Manipulation	Integrity
A shared query interface over partitioned knowledge	Answering a query does not disclose the corpus	Knowledge Extraction	Confidentiality
The partitioning of knowledge itself	The source holding a given record remains unobservable	Membership Inference	Confidentiality
Dependence on participants to respond when queried	A reachable participant responds when able	Selective Forwarding	Availability
Processing capacity distributed across participants	Each participant can serve the load directed to it	Denial of Service	Availability

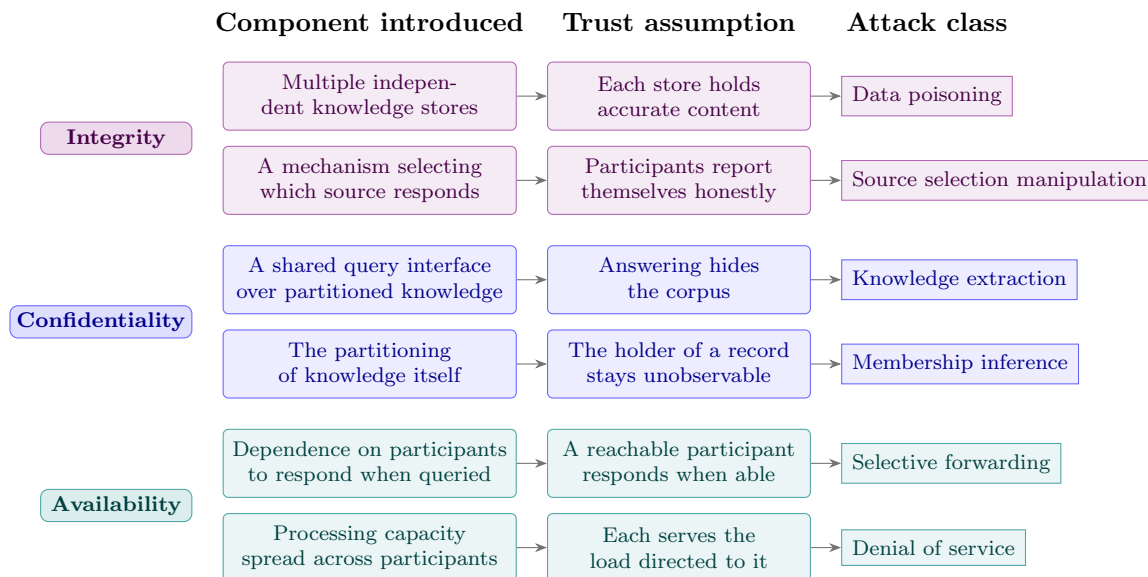


Figure 4.2: Derivation of the six attack classes of Table 4.1. The six distribute evenly across the CIA triad as an outcome of the derivation rather than as a constraint imposed on it.

The six classes distribute evenly across the three properties of the CIA triad, at two per property. This balance is an outcome of the derivation rather than a constraint imposed upon it, and it affords the framework even coverage of confidentiality, integrity and availability without requiring that attacks be selected to populate predetermined categories.

Three of the six classes have no counterpart in a centralised deployment. Source Selection Manipulation presupposes multiple sources between which the system must choose. Selective Forwarding presupposes a participant capable of remaining reachable while declining to respond honestly. Membership inference acquires a discriminative signal under distribution that a centralised deployment does not emit: routing behaviour. Our DRAG composite draws 30% of its weight from hop and message counts, quantities that exist only because queries traverse a partitioned network, and Section 5.2.4 shows that share is large enough to leave two different language models with identical confusion matrices. A centralised retriever answers from one index and produces no such signal. The three remaining classes exist in centralised systems and acquire additional properties under distribution: the attack surface available to data poisoning scales with the number of sources and rewards strategic selection of which source to compromise, knowledge extraction acquires a dependence on network topology that we quantify in Section 5.2.3, and denial of service acquires a redundancy dimension that a single-server deployment does not exhibit.

4.1.3 Scope of the Framework

The framework evaluates the distribution layer of a target system. Attacks directed at the language model or at the retrieval model fall outside its scope by construction rather than by omission. Prompt injection and jailbreaking target the instruction-handling behaviour of the generator, adversarial perturbation targets the embedding model, and training-data extraction targets model parameters. Each is a legitimate threat to a RAG deployment, none arises as a consequence of distributing such a deployment, and each has been studied extensively in the context of centralised architectures.

Three further attack classes fall within the distribution layer and outside the six we evaluate. Sybil attacks, in which a single adversary fabricates multiple participant identities, and collusion, in which several malicious sources coordinate their behaviour, both compose over the six classes rather than introducing trust assumptions those classes do not already address: a Sybil adversary multiplies identities to amplify data poisoning or source selection manipulation, and colluding sources multiply adversaries acting against the same assumptions. Cross-source correlation, by which an adversary infers from the combined responses of several sources what none of them discloses individually, constitutes the class our framework does not presently address, and Section 5.6 identifies it as the most immediate candidate for extension. Its derivation would follow the same seven steps set out above.

4.1.4 Application of the Procedure: A Worked Example

The analytical stage of the procedure is set out below as applied to a specific case. We take the derivation of our Source Selection Manipulation attack against Reliable-dRAG, since that attack was not obtained by adapting any attack reported in the literature

reviewed in Chapter 2, and was not ported from its DRAG counterpart.

Step 1, characterise the architecture. Reliable-dRAG partitions its document collection across three data sources. Every incoming query is submitted to all three, and an orchestrator collects the passages they return, ranks the combined set, and supplies the highest-ranked passages to a language model for answer generation. The ranking function weights each source’s reliability score, a value maintained on a blockchain and updated following each query according to whether the generated answer was correct and grounded in the text that source contributed.

Step 2, enumerate the trust assumptions. The ranking function assumes that a source’s reliability score reflects the quality of the contributions that source has made. That assumption depends in turn upon a second: that a source cannot satisfy the grounding check without having contributed genuinely relevant material. The grounding check determines whether the wording of the generated answer appears within the passage the source returned.

Step 3, map the class onto the mechanism. Source Selection Manipulation prescribes manipulation of whatever signal the selection mechanism treats as evidence of trustworthiness. In this architecture that signal is the grounding check. A source returning broad filler content spanning a wide range of possible phrasings will satisfy that check with sufficient frequency to accumulate reliability credit, notwithstanding that it holds no knowledge relevant to the questions being asked. We designate the resulting attack Grounding-Farming, and Section 4.2.2 presents its design.

Two observations follow from this example, both bearing on whether the procedure performs substantive work. The attack was obtained from the mapping stage rather than from a survey of previously reported attacks, and the same class applied to DRAG yields a different attack, since DRAG’s routing mechanism treats advertised topic expertise rather than a grounding check as its trust signal, and the corresponding instantiation falsifies those advertisements. Furthermore, Reliable-dRAG already incorporates a defence against manipulation of its reliability scores, and that defence proves structurally incapable of observing Grounding-Farming: it examines submitted transactions for a forged sender, an excessive update magnitude or an unauthorised caller, and Grounding-Farming submits no transaction exhibiting any of these properties. The framework therefore identified an exposure against which the system’s own designers had already deployed a countermeasure aimed elsewhere. Section 5.5 discusses this finding further.

4.1.5 From Procedure to Implementation

Section 4.2 presents the design of each attack and defence mechanism, treating the six classes in turn and describing the instantiation of each against both target systems. Section 4.3 documents the corpora used throughout the evaluation, and Section 4.4 the implementation and deployment of both systems together with the modules we added to them.

4.2 Preliminary Design or Design (Model) Specification

This section presents the design of each attack and defence obtained by applying the mapping stage of Section 4.1 to both target systems. The six attack classes derived in Table 4.1 are treated in turn, grouped by the property of the CIA triad each violates:

- **Integrity:** This property requires that data held by the system remains accurate during both storage and transmission. Data Poisoning corrupts the contents of a participant’s knowledge store, and Source Selection Manipulation corrupts the signal by which the system decides which participant to consult.
- **Confidentiality:** This property requires that data remain accessible only to those authorised to access it. Knowledge Extraction reconstructs the corpus through the system’s own query interface, and Membership Inference establishes whether a specific record is present without retrieving it.
- **Availability:** This property requires that data remain accessible to authorised users when needed. Selective Forwarding withholds answers from queries that reach a compromised participant, and Denial of Service exhausts the capacity of the participants serving them.

Each subsection describes the instantiation of one class against DRAG and against Reliable-dRAG, and states the metrics by which its effect is measured. The measurements themselves appear in Chapter 5. Integrity and availability attacks are scored by the degradation they cause in an answer-quality metric relative to that configuration’s own clean baseline. For a metric M with clean value M_{clean} and attacked value M_{atk} ,

$$\Delta(M) = \frac{M_{\text{clean}} - M_{\text{atk}}}{M_{\text{clean}}} \times 100\%,$$

so a configuration with a weak baseline is not penalised twice. Every severity assignment in Section 5.5 uses this relative form rather than an absolute difference in metric points.

4.2.1 Data Poisoning Attack Design

Data poisoning attacks aim to degrade the quality of answers a system provides by injecting malicious or incorrect content into its knowledge base, rather than exploiting the retrieval or generation logic directly – the attack succeeds only if the poisoned material is later retrieved and makes it into the model’s final prompt context. Both DRAG and Reliable-dRAG were evaluated under this attack; the point at which poison is introduced, and what ”success” costs the attacker, differ with how each system exposes its knowledge base to write access.

DRAG Implementation

Attack Methodology

The attack operates in two different phases:

1. **Peer Selection:** We designed and implemented three distinct peer selection strategies, and each strategy selects a subset of peers to compromise based on a configurable poisoning ratio (e.g., 10-30% of total peers):
 - **Random Peer Selection:** This baseline strategy randomly selects peers to compromise based on a specified poisoning ratio. The random approach simulates opportunistic attackers who compromise peers without specific targeting.
 - **Data-Rich Peer Targeting:** This strategy targets peers with the largest knowledge bases, as they are more likely to be relevant for a wider range of queries. By poisoning data-rich peers, attackers can affect diverse query topics. The process starts with counting the number of data points in each peer’s local knowledge base and ranking peers by knowledge base size, followed by selecting peers with the most data points for poisoning.
 - **High-Degree Node Targeting:** This strategy exploits the scale-free nature of peer-to-peer networks by targeting hub nodes, or peers with the highest number of connections. Since highly connected peers are more likely to be queried during information retrieval, compromising them has a disproportionate impact on system integrity. The process first begins by computing the degree (number of connections) for each peer in the network, then sorting peers by degree in descending order, and then selecting the top-k peers (where k is determined by poisoning ratio) for compromise.
2. **Data Injection:** For each compromised peer, the attack injects poisoned data points into selected peers. During this, an amplification factor determines how many copies of each poisoned data point will be created to overwhelm legitimate data, and a question variant count determines how many semantically similar questions with wrong answers will be generated to increase attack surface. The primary method of poisoning is answer swapping, where correct answers are replaced with incorrect ones from different topics in the knowledge base.

Evaluation Metrics

The effectiveness of data poisoning is measured through performance degradation across several previously established metrics:

- **Exact Match (EM):** Binary measure of whether the system’s answer exactly matches the expected answer (0 or 1)
- **F1 Score:** Harmonic mean of precision and recall, providing a balanced measure of answer quality
- **Semantic Similarity:** Cosine similarity between embeddings of the system’s answer and expected answer, measuring semantic closeness
- **Average Messages:** Network efficiency metrics showing how many peer-to-peer communications were required to answer queries

Success is measured by comparing these metrics with the clean baseline after the attack, with degradation indicating successful poisoning.

Reliable-dRAG Implementation

The DRAG baseline injects poisoned data directly into a compromised peer’s in-memory knowledge base, since data ownership and routing are the same peer-level concept in that architecture. Reliable-dRAG separates document storage from query routing entirely: each of the three data sources is an independent microservice holding its own FAISS vector index behind a live HTTP interface, so a comparable attack instead requires a write path into that running index. A `/poison` endpoint – alongside `/reset` and `/info` – was added to each data-source service as a controlled research extension, modelling an adversary who has gained the ability to insert or modify documents at a data source, whether through a compromised provider, a malicious contributor, or an ingested-but-unverified upload, rather than an attacker who can directly rewrite the LLM’s generation logic.

Attack Methodology

1. **Source Selection:** Three targeting strategies were implemented:
 - **Random:** Selects a subset of data sources to poison with no targeting logic, simulating an opportunistic attacker.
 - **Targeted:** Poisons a specific, named set of sources, passed explicitly, modelling an attacker with prior knowledge of which source to compromise.
 - **Data-Rich:** Targets the source(s) holding the most documents, measured via each source’s live document count. This replaced an earlier reliability-based targeting strategy that selected the source with the highest blockchain reliability score; that strategy was found to always select the same source regardless of actual reliability, since the blockchain feedback loop does not update during a normal query and every source tied at a reliability of zero. Data-rich targeting uses a real, directly observable signal instead.
2. **Poison Injection:** For each targeted source, a sample of clean documents is used as templates to generate poisoned variants, using one of four poison types – **wrong_answer** (replaces the text with an explicit corrupted-document message), **misleading** (generates plausible-looking but false text derived from the original), **noise** (appends random tokens to the original text), or **answer_swap** (replaces the document with content from a semantically unrelated document, to maximize model confusion). Each poisoned document is duplicated according to a configurable amplification factor and expanded into a configurable number of question-variant copies, so that the poisoned material occupies a disproportionate share of the source’s semantic neighborhood around the target query, then submitted through `/poison`, which triggers a full rebuild of that source’s FAISS index over the combined clean-plus-poisoned document set.

Evaluation proceeds across five phases: a clean baseline run, execution of the poisoning injection, a second evaluation run against the now-compromised sources, a direct comparison between the two accuracy figures, and a reset of every source back to its original clean state.

Evaluation Metrics

- **Clean Accuracy:** The proportion of correct answers before poisoning, using substring matching between the model’s response and a known expected-answer string.
- **Attacked Accuracy:** The same measure, computed after poisoned documents have been injected and re-indexed.
- **Accuracy Degradation:** The fall from clean to attacked accuracy relative to the clean baseline, computed as $\Delta(M)$ in Section 4.2.
- **Attack Success:** An attack counts as successful when $\Delta(M)$ exceeds 10%.

A caveat applies to the accuracy metric itself: correctness is judged by substring containment between the model’s response and the expected answer, which is a reasonable simplification for short-answer QA but can misjudge cases such as a negated response that happens to contain the expected substring, and is reported here as a named limitation of the measurement rather than as an unqualified ground truth. It is also worth noting that this injection strategy is not stealthy by design – a single run can increase a targeted source’s document count by 60-150%, well within range of a basic ingestion-volume monitor – so the measured degradation figures below characterize an undefended, unmonitored deployment specifically.

4.2.2 Source Selection Manipulation Attack Design

Source Selection Manipulation attacks target the mechanism by which a distributed RAG system decides which participant answers a given query. Every such system implements some form of this mechanism, and every implementation must derive its decision from a signal that participants themselves influence. The attack falsifies that signal so that queries are directed toward a source holding no relevant knowledge, degrading answer quality while displacing legitimate sources from the selection. Both DRAG and Reliable-dRAG were evaluated under this class. The signal each system trusts differs, and the instantiation differs accordingly. What transfers is the class and the question it asks, which component the selection mechanism treats as evidence of trustworthiness, not the attack or the damage it causes.

DRAG Implementation

DRAG’s Topic-Aware Random Walk consults two routing structures when resolving a query: a mapping from each peer to the topics it claims to serve, and the inverse mapping from each topic to the peers claiming it. Peers write their own entries in both structures, and the protocol authenticates neither. A peer may therefore claim expertise in any topic at no cost and without detection. Mu & Li [23] document the server-mediated form of this exposure, in which a federated router scores clients on semantic profiles it cannot check against their corpora. DRAG has no server, so no single point exists at which a profile could be checked, and a false claim propagates across the overlay through gossip rather than sitting in one place for a defender to audit.

Attack Methodology

The attack proceeds in two stages.

1. **Attacker Selection and Claim Injection:** A subset of peers is designated adversarial according to a configurable attacker ratio, selected either at random or by descending node degree so that the attackers occupy hub positions in the scale-free topology. Each attacker then claims a configurable fraction of the network’s topics, writing itself into both routing structures rather than one. Writing to both matters, because the two are consulted by different code paths during resolution, and an attacker present in only one remains reachable through fewer of them.
2. **Response Degradation:** Having attracted queries it cannot answer, the attacker returns degraded content. Two response types were implemented. Under the empty response type the attacker clears its own knowledge base, so every query routed to it terminates without an answer. Under the fabricated response type the attacker constructs plausible but incorrect entries by taking questions from its own holdings and pairing them with answers drawn from different questions on the same topic. The fabricated type is the more damaging of the two, since a wrong answer satisfies the routing algorithm’s termination condition where an absent answer does not.

The attack requires no privilege beyond normal participation. An attacker mutates only its own entries, through the write path the protocol provides for that purpose, and holds no access to any other peer’s knowledge base. Its cost is a single pass over the routing structures at setup, after which the corrupted state persists and degrades every subsequent query without further attacker action.

Evaluation Metrics

- **Exact Match, F1 and Semantic Similarity:** Answer quality relative to the clean baseline, measuring the degradation the misdirected queries produce.
- **Query Hit Rate:** The proportion of queries that reached a peer holding relevant knowledge, capturing displacement of legitimate sources rather than answer corruption.
- **Topics Hijacked:** The count of topics falsely claimed, which follows from the claim ratio and confirms the attack executed as configured.
- **Average Hops and Messages:** Routing cost, recorded so that any subsequent defence can be assessed against the overhead it adds.

Reliable-dRAG Implementation

Reliable-dRAG selects sources by a different route. Its orchestrator submits every query to all three data sources and reranks the combined candidate passages by a weighted blend of semantic relevance and each source’s on-chain reliability score. Two properties of that arrangement shape the attack. The reliability score is normalised across only the three participating sources, so a small absolute gain translates into a large relative reranking advantage. The score itself is updated after each query according to a grounding check that tests whether the generated answer’s wording appears within the passage a source returned, without establishing that the passage caused the answer.

Two attacks were implemented against this mechanism, at different privilege levels.

Attack Methodology: Grounding-Farming

The flagship instantiation requires no privilege beyond operating one data source. The attacker injects broad filler documents into its own store through the same write path used by the data poisoning attack, constructing one keyword cluster per article title in the target corpus so that the filler remains on topic without reproducing any real answer. Because the grounding check tests only for the presence of the answer’s wording somewhere within a returned passage, filler spanning a wide range of phrasings satisfies the check often enough to accumulate reliability credit the source has not earned.

The escalation dynamic was established empirically rather than assumed. A single early grounding credit raises the farming source’s score slightly, the normalisation across three sources amplifies that increase into a substantial reranking advantage, and the source then wins a share of the fixed top- k candidate budget in most subsequent rounds. The result is a self-reinforcing loop rather than a stable per-query success rate, which is why the attack either escalates fully within a run or fails to catch at all. Answer quality falls through displacement rather than through collision: inspection of individual regressions found no case in which the farming source’s filler displaced its own correct content, and every regression traced to gold context held by an untouched comparison source that lost its slot in the shared budget.

Attack Methodology: Key-Forgery

The secondary instantiation assumes the adversary holds the orchestrator’s private key, a materially higher privilege than operating a source. The contract restricts score updates to the orchestrator, and the signature accompanying each update covers the query and the list of selected sources but not the score values themselves. An adversary holding the key may therefore attach a source’s previously valid signature to fabricated reliability and usefulness figures. We report this instantiation as a secondary finding precisely because its privilege requirement exceeds what the framework’s threat model for this class describes, and Grounding-Farming supersedes it as the primary result.

Evaluation Metrics

- **Engagement Rate:** The proportion of rounds in which the attacking source’s content entered the orchestrator’s selected context, measuring whether the manipulation achieved selection.
- **Reliability and Usefulness Delta:** The absolute change in each on-chain score over a run, measuring how much undeserved credit accumulated.
- **Accuracy Drop:** The percentage-point fall in answer correctness against the clean baseline, measuring the consequence of that credit for end users.

4.2.3 Knowledge Extraction Attack Design

Knowledge extraction attacks aim to reconstruct a system’s private document collection by systematically querying it through the system’s own legitimate-looking interface,

without exploiting any implementation vulnerability. Both DRAG and Reliable-dRAG were evaluated under this attack; the specific mechanism each attacker exploits, however, follows directly from how each system exposes its retrieval layer.

DRAG Implementation

We implemented an external adversary model where the attacker operates as a network participant but must rely on the system’s normal query interface rather than direct access to peer internals.

Attack Methodology

The attack proceeds in several phases:

- **Attacker Peer Selection:** A randomly selected peer acts as the attacker, participating in the network with legitimate data while attempting extraction.
- **Query Selection:** The attacker uses assigned dataset questions to probe the network. Questions are selected using topic-balanced sampling to ensure coverage across different knowledge domains while excluding questions the attacker peer already possesses.
- **Knowledge Extraction:** The attacker sends queries through the network’s routing protocol. When responses contain relevant knowledge, this data is collected. The attack counts only knowledge extracted from other peers (answers requiring > 0 network hops), preventing ”self-extraction” from the attacker’s own knowledge base.
- **Deduplication:** Extracted data points are deduplicated based on question-answer pairs to identify unique recovered knowledge.

Evaluation Metrics

The success of knowledge extraction was quantified with primarily these metrics:

- **Extraction Rate (ER):** The proportion of the network’s knowledge recovered by the attacker. Let D_{net} be the union of all peers’ knowledge bases excluding the attacker’s own, and D_{ext} the set of unique question-answer pairs the attacker recovered at more than zero network hops. Then

$$\text{ER} = \frac{|D_{\text{ext}}|}{|D_{\text{net}}|}.$$

Issuing B queries across T topics bounds this above at $\text{ER} \leq BT/|D_{\text{net}}|$, which for the budget used here caps the reachable rate at 50%.

- **Chunk Recovery Rate (CRR):** The proportion of ground-truth chunks recovered at sufficient quality. For a chunk set C , a chunk c counts as recovered when any one of the three conditions holds, a deliberately attacker-favourable disjunction:

$$\text{rec}(c) = 1[\text{EM}(c) \vee \text{SS}(c) \geq 0.8 \vee \text{EED}(c) \leq 0.2], \quad \text{CRR} = \frac{1}{|C|} \sum_{c \in C} \text{rec}(c).$$

[7]

- **Semantic Similarity (SS):** Measures how semantically close extracted chunks are to original chunks using cosine similarity of sentence embeddings, calculated as:

$$SS(\text{extracted}, \text{target}) = \frac{E_{\text{extracted}} \cdot E_{\text{target}}}{\|E_{\text{extracted}}\| \times \|E_{\text{target}}\|}$$

where E represents embedding vectors. A threshold of $SS \geq 0.8$ (or 80%) indicates successful semantic recovery. [7]

- **Effective Edit Distance (EED):** Normalized Levenshtein distance measuring character level similarity, calculated as:

$$EED(\text{extracted}, \text{target}) = \frac{\text{Levenshtein_distance}(\text{extracted}, \text{target})}{\max(\text{len}(\text{extracted}), \text{len}(\text{target}))}$$

where a threshold of $EED \leq 0.2$ indicates successful recovery based on textual similarity. [7]

- **Query Efficiency:** Ratio of successful extractions to total queries sent.
- **Topic Coverage:** Proportion of knowledge topics successfully extracted from – meaningful here because reaching a topic requires the routing protocol to actually traverse to a peer holding it.

ER measures overall data extraction rate, while CRR measures data extracted with a sufficient quality only. SS and EED are calculated only for extracted chunks (excluding non-extracted chunks) to provide a true measure of extraction quality. Query efficiency and topic coverage measure the overall efficiency of the attack simulation.

Reliable-dRAG Implementation

Where the DRAG baseline attack extracts knowledge through multi-hop peer routing – an attacker peer issuing queries that traverse the network via TARW – Reliable-dRAG’s three data sources are reached directly over HTTP, so extraction here targets the `/query` API surface itself rather than a routing protocol. With no multi-hop traversal to exploit, the attack instead models escalating credential access.

Attack Methodology

The attack proceeds through three sequential phases:

- **Phase A – Unauthenticated Access:** Every data source is probed with no API key, to establish whether the credential gate is enforced before any extraction is attempted.
- **Phase B – Authenticated Extraction:** A fixed set of realistic probe questions is sent to each source with the shared API key, modelling an attacker holding a valid (or stolen) credential. Returned documents are deduplicated and compared against that source’s true document set.
- **Phase C – Indirect LLM Leakage:** A subset of probes is routed through the LLM orchestrator rather than the raw retrieval endpoint, to measure whether private content leaks indirectly through the model’s generated, paraphrased answers rather than through verbatim retrieval.

Evaluation Metrics

- **Extraction Rate (ER):** The proportion of a single source’s document collection recovered, computed per source rather than across the deployment. For source s holding documents D_s , with $D_{\text{ext},s}$ the deduplicated set returned to the attacker,

$$\text{ER}_s = \frac{|D_{\text{ext},s}|}{|D_s|}.$$

- **Extraction Accuracy:** The proportion of recovered documents that are genuine rather than fabricated or noise; since retrieval returns only stored passages verbatim, this is structurally bounded at 1.0.
- **Query Efficiency:** The average number of documents recovered per probe query sent.
- **Chunk Recovery Rate (CRR):** Applied to Phase C only, using the same $SS \geq 0.8$, $EED \leq 0.2$ thresholds defined above, since generation can restate private content without quoting it exactly.

4.2.4 Membership Inference Attack Design

Membership inference attacks aim to determine whether a specific data point exists in a system’s private knowledge base by analyzing system behavior during query processing, without ever seeing the knowledge base directly. Both DRAG and Reliable-dRAG were evaluated as black-box adversaries with standard query access; the discriminative signals available to each attacker, however, depend on what the underlying architecture actually exposes.

DRAG Implementation

Attack Methodology

- **Data Preparation:** The dataset is partitioned into members (70% – data points present in knowledge bases) and non-members (30% – data points temporarily removed from all peers). Non-members are completely removed from knowledge bases and embeddings are rebuilt to ensure authentic non-member status.
- **Query-Based Feature Extraction:** For each test sample, the attack queries the network and extracts discriminative features from the responses:
 - Semantic Similarity (weighted 60%): Cosine similarity between generated answer and ground-truth answer using sentence embeddings
 - Network Routing Metrics (weighted 30%): Number of hops and messages required to retrieve the answer – meaningful because member and non-member queries traverse the multi-hop network differently
 - Answer Length Ratio (weighted 10%): Ratio of generated answer length to expected length

- **Membership Score Calculation:** Features are combined via weighted aggregation to produce a composite membership score for each query.
- **Threshold Classification:** A percentile-based threshold (50th percentile/median) separates member from non-member predictions without requiring model training.

The attack exploits the observation that member queries retrieve relevant context and generate semantically similar answers, while non-member queries produce generic responses due to absent knowledge.

Evaluation Metrics

- **Attack Accuracy:** Overall correctness of membership predictions [20]
- **True Positive Rate (TPR/Recall):** Percentage of actual members correctly identified [20]
- **False Positive Rate (FPR):** Percentage of non-members incorrectly classified as members [20]
- **Precision:** Accuracy of positive membership predictions
- **AUC-ROC:** Area under ROC curve measuring discrimination capability across all thresholds [20]
- **Privacy Risk Score:** Composite metric combining accuracy (40%), TPR (30%), and AUC-ROC (30%), calculated as:

$$(\text{Accuracy} \times 0.4) + (\text{TPR} \times 0.3) + (\text{AUC-ROC} \times 0.3)$$

Risk levels are categorized as: CRITICAL (> 0.9), HIGH (0.75-0.9), MEDIUM (0.6-0.75), LOW (0.5-0.6), NEGLIGIBLE (≤ 0.5).

Reliable-dRAG Implementation

The DRAG baseline composite includes Network Routing Metrics as a discriminative signal, since a multi-hop peer-to-peer network produces routing behavior that varies systematically between member and non-member queries. Reliable-dRAG’s LLM service is reached directly rather than through multi-hop routing, so no equivalent routing signal exists; in its place, whether the response commits to the correct categorical decision is used instead.

Attack Methodology

The attack queries the system with a mix of member (in-corpus) and non-member (held-out) question-answer pairs drawn from a PubMedQA-derived medical corpus, and extracts four candidate discriminative signals from each response:

- **Embedding Similarity:** Cosine similarity between the generated answer and the ground-truth answer.

- **Certainty Proxy:** A measure of how confidently the response commits to an answer, versus hedging.
- **Answer Length Ratio:** Ratio of generated answer length to expected answer length.
- **Decision Match:** Whether the response commits to the same categorical decision (yes/no/maybe) as the gold answer, matched positionally or by scanning the full response for known synonyms.

Rather than fixing composite weights by inspection, the weighting of these four signals was empirically tuned under a train/held-out split: candidate weightings were searched for using signal data pooled across thirteen seeds, then locked and evaluated exactly once against five fresh seeds never queried during tuning, to obtain an honest generalization estimate rather than a search-fit number.

Evaluation Metrics

The same classification metrics as the baseline evaluation – Attack Accuracy, Precision, Recall (TPR), False Positive Rate (FPR), and AUC-ROC – together with the same Privacy Risk classification scale defined above.

4.2.5 Selective Forwarding Attack Design

Selective forwarding attacks target a system’s dependence on its own participants to respond when queried. A compromised participant stays connected, passes any liveness check put to it, and retains whatever routing priority it has legitimately accumulated, yet returns nothing when a query arrives. To the system the participant appears healthy and merely lacking relevant knowledge, which is behaviour an honest participant also exhibits. Both DRAG and Reliable-dRAG were evaluated under this class. The damage a withheld response causes depends on how each system’s routing recovers from it, and the two systems recover differently.

DRAG Implementation

The attack intercepts the response a compromised peer returns, leaving the overlay graph, that peer’s connections and its degree unmodified. This is what distinguishes selective forwarding from an availability attack that removes participants outright: the topology stays intact and the network’s view of itself stays accurate, so no structural measurement reveals the compromise. Only the statistical pattern of a peer’s responses across many queries separates it from an honest peer serving topics it does not hold.

Attack Methodology

Two targeting strategies were implemented, matching those used for data poisoning so that the effect of targeting sophistication can be compared across attack classes:

- **Random Selection:** Compromises peers drawn uniformly at random according to a configurable ratio, modelling an adversary holding no knowledge of network structure.

- **High-Connectivity Targeting:** Compromises the peers of highest degree, exploiting the concentration of routing traffic through the small number of hubs that a scale-free topology produces.

The compromise ratio was swept from 10% to 50% of peers under both strategies. The attack requires no privilege beyond control of the peers it compromises, and no interaction with any other peer’s knowledge base.

Evaluation Metrics

- **Dropped Queries:** The count of queries a compromised peer discarded, measuring the attack’s reach independently of whether the router later recovered.
- **Query Hit Rate:** The proportion of queries that still reached a peer holding relevant knowledge, measuring whether the network’s redundancy absorbed those drops within the hop budget.
- **Average Hops per Query:** Routing depth, recorded so that wasted traversal can be separated from outright failure.

Reliable-dRAG Implementation

Reliable-dRAG’s deployment resolves every query against three sources behind one orchestrator, so withholding by a single source removes a third of the candidate pool without any traversal to recover through. To vary compromise ratio and targeting strategy across a range the three-source deployment cannot express, we evaluated this class against a network model that routes by breadth-first search under a fixed time-to-live budget, in which each visit to a compromised peer consumes one hop without returning an answer. The sweep results in Section 5.2.5 therefore characterise that model rather than the deployed system, and we report a live single-configuration run beside them. Section 5.6 records the gap between the two as unresolved.

Attack Methodology

A compromised peer drops a configurable fraction of the queries routed through it, a gray-hole behaviour closer to a cooperative insider than to an overwhelmed or disconnected node. Two targeting strategies were implemented, uniform random selection and degree-ranked targeting of high-connectivity peers, posing the same targeting-sophistication question tested against DRAG but on an architecture where a wasted visit costs part of a fixed budget.

Evaluation Metrics

- **Hit Rate:** The proportion of queries that received a correct answer within the time-to-live budget, sensitive to budget exhaustion rather than to peer unavailability alone.
- **Average Hops per Query:** Mean routing depth, which rises as the router is forced past compromised peers before reaching a responsive one.

- **TTL Exhaustion Rate:** The proportion of queries that exhausted their hop budget without reaching an answer. This is distinct from an outright failure, since the peer responsible is by design indistinguishable from a slow but honest one until the metric is examined in aggregate.
- **Dropped Queries:** The raw count of queries silently discarded, used to confirm the attack behaved as configured independently of its effect on hit rate.

4.2.6 Denial of Service Attack Design

Denial of service attacks target the finite processing capacity that a distributed deployment spreads across its participants. Queries arrive in excess of what those participants can serve, and the system falls behind demand while remaining, by its own account, fully operational. Both DRAG and Reliable-dRAG were evaluated under this class. The scale at which each system can be tested differs, and Reliable-dRAG’s container deployment permits a form of evaluation that DRAG’s simulator does not.

DRAG Implementation

Distributed denial-of-service attacks flood the network with queries in excess of what its peers can process, leaving those peers online and reachable while they fall behind demand. The failure mode is subtler than an outright loss of participants, since the network’s view of its own topology stays accurate throughout. A congested peer silently drops a fraction of the queries routed to it, imposes additional hop and message overhead on the queries that survive, propagates part of its congestion to its immediate neighbours, and recovers once its attack window expires. The model operates at the application layer over the peer topology rather than at the packet or socket level, matching the granularity at which the routing algorithm itself operates.

Attack Methodology

The attack unfolds in discrete waves. At the start of each wave, any peer whose attack window has elapsed is restored to full health, modelling the way rate limiting or firewall protection lets a peer drain its backlog between bursts. The attack then selects a fresh set of targets, samples a load intensity for each from a configured range, and derives from that intensity both a drop probability, capped so that a saturated peer never becomes a complete black hole, and a load penalty quantifying how heavily the peer is congested. Each query routed to a congested peer is subjected to an independent probabilistic drop decision against that peer’s drop probability, and queries that survive still pay overhead in additional hops and messages proportional to the peer’s intensity.

Three targeting strategies were implemented:

- **Random Targeting:** Selects target peers by uniform random sampling each wave, modelling an unsophisticated attacker holding no knowledge of network structure.
- **Targeted Selection:** Preferentially selects peers not already overloaded, maximising newly inflicted damage per wave.

- **Sequential Targeting:** Sweeps through peers in deterministic round-robin order, modelling a systematic scan across the whole network.

Congestion is not confined to the peers directly targeted. An overloaded peer spills load onto its immediate neighbours at a reduced intensity, capped so that neighbours remain less affected than direct targets, reflecting the way routing pressure bleeds into adjacent nodes. Because each targeted peer recovers independently once its attack duration expires, network health follows a ramp-up-and-recover trajectory across successive waves rather than collapsing monotonically. All randomness is drawn from independently seeded streams, with wave generation separated from per-query drop decisions, so that repeated runs against the same seed reproduce an identical damage trajectory and the undefended network can be compared against a defended variant under identical attack conditions.

The attack was evaluated at two intensity profiles, each run against all three targeting strategies, giving six configurations per dataset:

- **Medium-Intensity Attack:** A 40% attack ratio applied over five waves at load intensities between 0.6 and 0.9.
- **High-Intensity Attack:** A 60% attack ratio applied over five waves at load intensities between 0.8 and 1.0.

Evaluation Metrics

- **Query Failure Rate:** Percentage of queries dropped by a congested peer and scored as a failed retrieval.
- **Availability:** The proportion of peers still serving at the end of the final attack wave. Taking P as the peer set and d_p as peer p 's drop probability,

$$\text{Avail} = \frac{|\{p \in P : d_p < 0.5\}|}{|P|} \times 100\%.$$

This measures network health independently of query outcomes.

- **Exact Match:** Proportion of queries, including drops scored as zero, whose answer matched the expected answer exactly.
- **F1 Score:** Answer correctness across all queries, including drops scored as zero.
- **Semantic Similarity:** Embedding-based similarity between returned and expected answers, capturing correct but differently worded responses that exact match and F1 under-credit, which matters most on the open-generative News and Medical datasets.
- **Average Hops and Messages:** Mean routing steps and peer-to-peer communications per query, including retransmission overhead from congested peers.

Reliable-dRAG Implementation

DRAG evaluates availability entirely in simulation, across networks of ten to twenty

peers, where failure is a modelled routing outcome. Reliable-dRAG’s deployment consists of three data-source containers behind a live orchestrator, small enough to attack directly rather than only in simulation. That changes what an availability result can establish: not merely whether a simulated router believes a peer is reachable, but whether a generated answer returns correctly, or returns at all.

Attack Methodology

The attack was implemented in two complementary forms, addressing the gap between a controllable simulation and a realistic network condition. An application-layer congestion simulation marks a subset of peers overloaded during each discrete wave, with every query routed to an overloaded peer facing a calibrated probability of being dropped, modelled as a Bernoulli trial per query whose probability rises with a randomly sampled load intensity rather than a single fixed value, so that overloaded peers do not fail identically. A second implementation launches real concurrent HTTP request floods against the data-source containers and measures the effect on generated answers rather than on a simulated routing outcome. It operates at three severities, corresponding to one, two or all three data sources being flooded at once, so that the marginal cost of losing each additional source can be observed directly rather than inferred.

Evaluation Metrics

The Exact Match, F1, Semantic Similarity and Average Messages metrics defined above were retained, computed against generated answers under the live-flood condition rather than against a simulated routing outcome. The simulation form additionally reports Hit Rate, Availability and TTL Exhaustion Rate as defined for the selective forwarding attack above, since both attacks degrade the same TTL-bounded routing process.

- **TTL Exhaustion Rate:** The proportion of queries that spent their full hop budget without reaching an answer, $TER = |\{q \in Q : h(q) = \text{TTL}_{\max} \wedge \neg \text{hit}(q)\}|/|Q|$, where $h(q)$ is the hops query q consumed.

4.2.7 Defense Mechanism Design

The six defences described below were designed by following step 7 of the evaluation procedure against our findings for DRAG. Each answers a specific trust assumption that Section 5.2 records as having failed, and each was implemented on DRAG alone, which serves as the framework’s demonstration that a system can act on the evaluation’s output. The measured effect of each defence is reported in Section 5.3.

Defence Against Data Poisoning: Replication and Cross-Peer Validation

The poisoning attack succeeds because the routing algorithm accepts the first answer it retrieves and holds no means of distinguishing a fabricated answer from a legitimate one. Two mechanisms address this, and we evaluated them independently and in combination.

Knowledge Base Replication stores each data point on a configurable number of peers rather than one. Redundancy raises the probability that an uncompromised copy of any

given record survives a partial compromise, gives legitimate answers multiple possible sources, and supplies the diversity of respondents that the second mechanism requires. Its cost is storage overhead at each peer.

Cross-Peer Validation makes agreement a precondition for acceptance. Rather than returning the first answer found, the system continues routing until it has collected responses from a minimum number of peers, groups those responses by fuzzy string matching so that legitimate answers differing in phrasing cluster together, and accepts an answer only if it belongs to the largest group and that group is a configured majority of respondents. Where no clear majority forms, the system accepts the original candidate rather than rejecting it, preserving availability at the cost of protection in ambiguous cases. A poisoned source is by construction a minority, so an adversary must compromise more than the majority threshold of the peers consulted for a given query to pass a fabricated answer through validation.

The mechanism’s cost is communication. Collecting enough responses to validate meaningfully requires additional routing hops beyond those a single answer would need, and Section 5.3 reports the resulting overhead alongside the recovery.

Defence Against Source Selection Manipulation: Topic-Claim Quota Enforcement

Cross-peer validation addresses fabricated answers after they have been retrieved. The attack described above can be addressed earlier, at the point where a peer advertises topics it does not serve, because the advertisement itself carries a detectable signature: an attacker claiming a large share of the network’s topics departs sharply from the distribution honest peers produce, and does so before answering a single query.

The defence audits the claimed-topic count of every peer and computes a robust quota from the median count across the network. Peers whose counts fall outside that quota are identified as statistical outliers using a median absolute deviation test, chosen over a mean-based test because the attacker’s own extreme values would otherwise inflate the statistic used to detect them. A flagged peer has its excess claims struck from both routing structures, removing it from topic-matched routing priority. Should a quarantined peer’s answer nonetheless surface, a validation step rejects it and substitutes the majority answer among peers not under quarantine.

The mechanism’s effectiveness depends on the attacker’s claims being distinguishable from honest variance, which links its performance to attack severity rather than making it uniform. An attacker claiming few topics remains closer to the honest distribution than one claiming many, and Section 5.3 reports how the recovery varies across the claim ratios we tested. The cost is validation traffic rather than routing depth.

Defence Against Knowledge Extraction: Layered Extraction Prevention

Extraction succeeds through the breadth of an attacker’s queries rather than through any single request, and no one barrier stops a patient adversary. The defence therefore

applies three layers, each addressing a different property of extraction behaviour, so that circumventing one leaves the others in force.

Query Rate Limiting caps the number of queries a peer may submit within a rolling time window and blocks a peer that exceeds the cap for a fixed cooldown. The layer is inexpensive and takes effect before any data moves. Its weakness is an adversary who distributes queries across several identities or paces them below the cap.

Extraction Anomaly Detection addresses the behavioural pattern the rate limiter cannot see. Legitimate users query a small number of related topics, whereas an extraction adversary spreads queries across as many topics as possible to maximise coverage. The detector tracks query volume, distinct topics queried, and the ratio between them, and flags a peer that crosses a threshold on any of the three after a short grace period. Queries from a flagged peer are then blocked.

Response Perturbation acts on data that passes both gates. The layer injects character-level noise into the raw structured record returned to the requester, after the answer presented to a human reader has been finalised from the unmodified context. An automated extraction pipeline parsing the structured output therefore receives corrupted content while a genuine user’s answer remains correct. The layer degrades exact recovery in particular, since a single altered character defeats an exact match.

The three layers gate the query lifecycle at different points: rate limiting and anomaly detection stop queries before retrieval, in that order, and perturbation acts on what is returned. All three were active in every defended run.

Defence Against Membership Inference: Privacy-Preserving Retrieval

The membership inference composite draws the majority of its discriminative power from retrieval similarity, so degrading that signal at its source collapses the adversary’s ability to separate member from non-member queries irrespective of what happens downstream. The defence adds calibrated Laplace noise to each peer’s retrieval similarity scores before those scores are clipped back into range, rather than filtering or hedging the language model’s output.

The noise scale governs a trade between privacy and utility, and the trade is not linear: the variance the mechanism introduces grows quadratically in the scale parameter, so the utility cost escalates faster than the privacy gain as the parameter rises. We therefore evaluated the defence across the range of noise scales its configuration documents rather than at a single setting, and Section 5.3 reports the resulting frontier together with the operating point we recommend.

Defence Against Selective Forwarding: Reputation and Routing Bypass

A compromised peer that drops queries silently passes every liveness check available to the router, so no single interaction distinguishes it from an honest peer holding no

relevant knowledge. Evidence accumulates only across repeated interactions, which is what the defence tracks.

Each peer carries an exponential moving average of its response rate, weighting recent behaviour more heavily than older behaviour while remaining stable against isolated failures. A peer whose response rate stays below a threshold across a minimum number of observations is blacklisted, and the routing algorithm then bypasses it rather than down-weighting it, so a blacklisted peer consumes no part of a query’s hop budget. A bounded number of redundant probes against untried, trusted peers is permitted once the primary budget is exhausted, which is what converts avoidance of a compromised peer into recovery of the query it would have absorbed.

Two properties of the design bear on how its results should be read. The evidence requirement that makes the mechanism robust against isolated failures also delays detection, so a peer dropping a small fraction of queries takes longer to identify than one dropping all of them. Separately, where the network already replicates topics widely enough for the undefended router to find an alternative path within its hop budget, the mechanism’s contribution is a reduction in wasted routing rather than an increase in answered queries. Section 5.3 reports both halves of that outcome.

Defence Against Denial of Service: Layered Congestion Defence

Congestion differs from the failure modes above in that an affected peer degrades gradually and probabilistically, spreads part of its load to its neighbours, and recovers once the pressure against it ends. A defence that permanently excludes a congested peer would discard capacity the network regains shortly afterward. Four layers therefore operate together, each on the congestion state the attack itself produces.

Adaptive routing weight reduces a peer’s selection probability in proportion to its measured load, so influence degrades smoothly rather than by a binary decision. **A circuit breaker** tracks each peer through closed, open and half-open states, excluding a peer whose failure probability crosses a threshold, retrying it at a reduced rate after a cooldown measured in attack waves, and extending that cooldown if it fails again.

Query hedging issues a second attempt at a healthier peer whenever the primary peer’s load exceeds a threshold, counting a query as lost only if both attempts fail.

Load-shedding mitigation decays the congestion state of any peer the breaker currently excludes, on the principle that withholding traffic allows a peer to clear its backlog. The fourth layer is the only one that reduces the damage the attack causes rather than routing around it.

All four depend on capacity remaining somewhere in the network for traffic to be redirected toward, which ties their effectiveness to how evenly the attacker spreads its load. Section 5.3 reports how the recovery varies across the targeting strategies and intensities we tested.

4.3 Data Collection

Neither target system was built for this thesis. DRAG, proposed by Xu et al. [9], was selected from our literature review as the only publicly available implementation of a distributed RAG architecture at the time the work began. Reliable-dRAG by Lu et al. [21] was added subsequently to test whether the framework transfers across architectures, and was chosen for its distance from DRAG rather than its similarity to it. Each system arrives with its own corpora, and we retained those corpora rather than substituting our own, so that our baseline measurements remain comparable with the results each system’s authors published.

DRAG distributes a question-answer dataset across peers according to topic assignment. Each peer holds a private shard, observes no other peer’s holdings, and serves a locally hosted language model for answer formatting. Three datasets ship with the implementation:

- **MMLU (Massive Multitask Language Understanding):** A multiple-choice question answering dataset spanning academic subjects, in which each answer is a single option symbol.
- **Medical Extended:** A synthetic collection of patient questions paired with clinician responses across a range of conditions and treatments, answered in free text.
- **News:** Headlines drawn from HuffPost between 2012 and 2022, configured for an author identification task in which the headline serves as the question and the article’s byline as the answer.

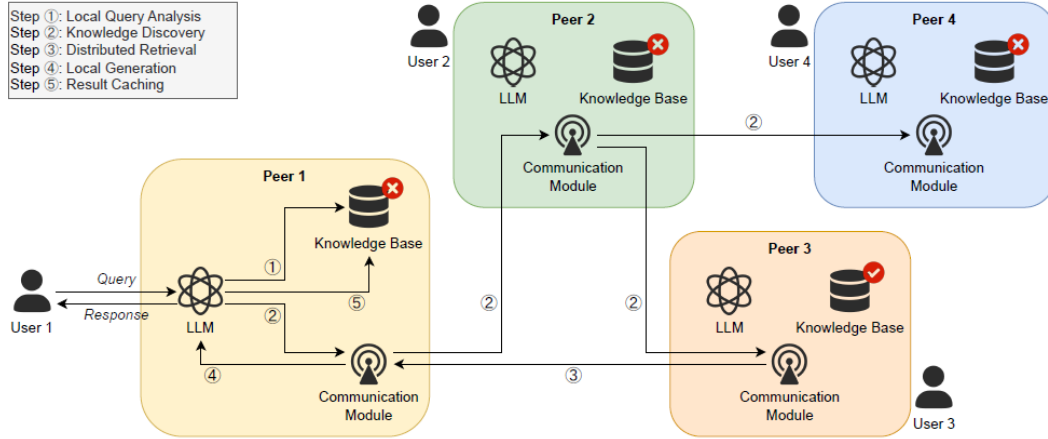


Figure 4.3: An overview of DRAG Architecture by Xu et al. Adapted from [9]

Reliable-dRAG organises its data differently. Rather than partitioning by topic across many peers, it maintains three independent data sources, each holding its own document collection behind a retrieval service, with an orchestrator querying all three for every request. The distribution supplied by its authors derives from SQuAD and assigns each source a different proportion of deliberately corrupted documents, which the system uses to demonstrate its reliability scoring. That corruption is part of the system’s own design

rather than an artefact of our attacks, and it forms the baseline against which our integrity results are measured.

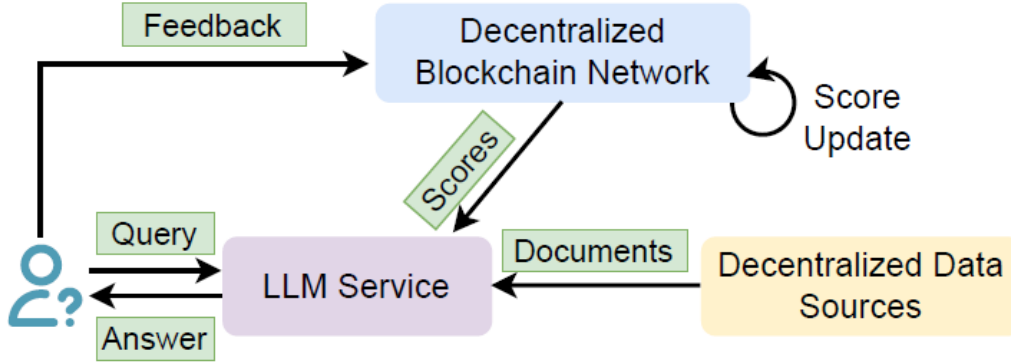


Figure 4.4: System overview of Reliable-dRAG by Lu et al. Adapted from [21]

4.3.1 Data Cleaning

Four preparation steps were required before the corpora could support the evaluation, each addressing an inconsistency that would otherwise have distorted a measurement.

Verification of corruption levels. Reliable-dRAG names its three sources after the proportion of corrupted documents each is intended to hold. Comparing the source files directly against one another established that the names do not describe the distribution accurately: the first source is clean as labelled, but the second holds roughly eleven percent altered documents and the third roughly fifty-nine percent, rather than the twenty and one hundred percent the names imply. We report corruption levels from the measured values throughout, since attributing an integrity result to a source’s labelled corruption rather than its actual one would misstate the conditions under which that result was obtained.

Corpus substitution for membership inference. The membership inference attack requires non-member records drawn from the same distribution as the members it tests against. An initial configuration drew members from one Reliable-dRAG source and non-members from the others, which placed medical abstracts against encyclopaedic trivia and produced a membership signal reflecting the difference between two domains rather than the presence or absence of any record. We replaced the first source’s contents with the labelled subset of PubMedQA and drew non-members from held-out rows of that same corpus, so that the only property distinguishing a member from a non-member is whether the system holds it.

Genuine removal of non-member records. For the equivalent attack against DRAG, records designated non-members were removed from every peer’s knowledge base and the retrieval embeddings rebuilt afterward, rather than being excluded at evaluation time alone. Without the rebuild, a removed record’s embedding would remain available to retrieval and the non-member set would not be non-member in any meaningful sense.

Baseline equalisation for poisoning. The data poisoning evaluation against Reliable-dRAG used a corpus state in which all three sources hold identical content. Any degradation observed under that condition is therefore attributable to the injected material rather than to pre-existing differences between the sources, at the cost that the data-rich targeting strategy reduces to an arbitrary tie-break when every source holds the same number of documents. Section 5.2.1 reports that consequence alongside the result.

4.3.2 Data Transformation

Both systems encode their text for retrieval before any query is served, and our instrumentation operates on the encoded form rather than replacing it.

Each question-answer record is converted to a dense vector by a sentence-transformer model, and retrieval scores queries against those vectors by cosine similarity. Reliable-dRAG maintains a vector index per data source, and any write to a source’s document store triggers a rebuild of that index before the source serves further queries. The poisoning attack described in Section 4.2.1 depends on this rebuild, since injected documents become retrievable only once they have been encoded and indexed.

DRAG additionally derives a topic label for each record and for each incoming query, and the routing algorithm matches the two when selecting which peer to consult. Topic assignment is therefore both a data property and a routing input, which is what makes the source selection manipulation attack of Section 4.2.2 possible against this architecture.

Two transformations introduced by our attack instrumentation are worth recording here because they alter the corpus rather than the query path. Poisoned records are duplicated according to an amplification factor and expanded into semantically similar question variants, so that corrupted material occupies a larger share of the retrieval neighbourhood around a target query than a single insertion would achieve. The response perturbation defence of Section 4.2.6 substitutes characters within the record returned to a requester, leaving the record held in the knowledge base unmodified.

One property of MMLU shapes how its results must be read. Its answers are single option symbols, which causes token-overlap F1 to coincide exactly with exact match in every run. We verified this identity numerically across all poisoning runs rather than inferring it from the tables, and Chapter 5 reports a single column for MMLU in consequence.

4.3.3 Data Integration

DRAG assembles its network by distributing records across peers according to topic, with each topic assigned to a set of peers and each peer consequently holding a subset of topics. The replication mechanism described in Section 4.2.6 modifies this distribution by placing each record on a configurable number of peers rather than one, which changes the integration step itself rather than the query path, and is why replication must be configured before a network is built rather than enabled at query time.

Reliable-dRAG performs no comparable integration at the data layer, since its three

sources are independent services holding separate collections. Integration occurs instead at query time, where the orchestrator merges candidate passages returned by all three sources into a single ranked list before selecting the passages a language model will read. That merge step is the point the source selection manipulation attack targets.

The two systems consume different corpora, and we did not attempt to align them. Comparing absolute scores between DRAG and Reliable-dRAG would measure the difference between MMLU and SQuAD as much as any difference between the architectures. Chapter 5 therefore compares the two systems on degradation relative to each system’s own clean baseline, and on the severity tier that degradation falls into, rather than on raw metric values.

4.3.4 Data Reduction

Evaluating every attack against a full corpus was neither necessary nor affordable, since each query in DRAG invokes a locally hosted language model and each attack was measured across a parameter sweep. We reduced the working set in three ways.

Evaluation sampling. Each DRAG run evaluates one hundred questions drawn from the configured dataset. This figure was chosen so that a single question moves a percentage metric by one point, which is fine enough to separate the conditions in our sweeps without requiring a full-corpus pass for every cell.

Probe budgets for the confidentiality attacks. The knowledge extraction attack against DRAG issues five queries per topic, sampled to balance coverage across topics and excluding questions the attacker peer already holds. The budget bounds the reachable extraction rate at half the corpus, which Chapter 5 states alongside the results so that the figures are read against the correct ceiling. The equivalent attack against Reliable-dRAG issues fifty-four probes per source per seed.

Membership partitions. The membership inference attack against DRAG partitions its evaluation sample into seventy percent members and thirty percent non-members. Against Reliable-dRAG the partition is even, at twenty-five records on each side, drawn from the PubMedQA corpus described above.

The data poisoning evaluation against Reliable-dRAG is the smallest sample in the study, at seven evaluation questions, which places each question at slightly over fourteen accuracy points. Section 5.2.1 reports the resulting standard deviations without adjustment, and Section 5.6 records the sample size as a limitation of that particular result.

4.3.5 Summary of Preprocessed Data

Table 4.2 lists the corpora in the state in which the evaluation consumed them.

Table 4.2: Corpora Used in the Evaluation, After Preparation

System	Corpus	Answer Format	Preparation Applied	Attacks Evaluated Against It
DRAG	MMLU	Single option symbol	Topic assignment; 100-question evaluation sample	All six classes
	Medical Extended	Free text	Topic assignment; 100-question evaluation sample	Poisoning, extraction, routing, DoS
	News	Free text (byline)	Topic assignment; 100-question evaluation sample	Poisoning, extraction, routing, DoS
Reliable-dRAG	SQuAD-derived, corrupted	Free text	Corruption levels measured directly rather than taken from source names	Extraction, forwarding, DoS, source selection
	PubMedQA (labelled subset)	Categorical decision	Substituted into the first source; non-members held out from the same corpus	Membership inference
	Equalised baseline corpus	Free text	All three sources set to identical content	Data poisoning

Note: Reliable-dRAG’s corpus state differs between attacks because the membership inference and poisoning evaluations each required a preparation the others did not. The document counts consequently differ between passes, and results from one pass should not be read as a baseline for another. Section 4.4 records which corpus state each experiment ran against.

Two properties of this table bear on how Chapter 5 should be read. The News dataset pairs a headline with an article byline, so semantic similarity between a question and its answer is low by construction on that dataset and does not indicate a weaker attack. Reliable-dRAG’s corpus state is not constant across attacks, since the membership inference and poisoning evaluations each required a substitution the other attacks did not, which means absolute figures from those two evaluations are not directly comparable with the remaining four.

4.4 Implementation of Selected Design

The framework required no modification to either target system’s core logic. Both systems were deployed as their authors published them, and our attacks and defences were added as separate modules that attach to the existing query path. This constraint was deliberate: an evaluation that alters the system it measures cannot claim its findings apply to the system as deployed, and a defence that requires rewriting the retrieval pipeline is a redesign rather than a mitigation.

4.4.1 DRAG Implementation

DRAG runs as a single-process simulator. A network object constructs a Barabási–Albert overlay of peer objects, assigns each peer a shard of the configured dataset, and resolves

queries by a breadth-first traversal bounded by a hop budget. We added attack and defence modules alongside the existing network, retrieval and evaluation components, and activated them through the configuration files the simulator already reads rather than through code changes at each experiment.

Every defense implements a shared interface providing a validation hook, a statistics accessor and an enable flag, so that defenses may be combined or disabled independently within a single run. Attacks attach at whichever layer the corresponding class targets: data poisoning and knowledge extraction operate on peer knowledge bases, routing manipulation writes to the shared routing structures, membership inference and knowledge extraction issue queries through the ordinary interface, and selective forwarding intercepts a peer’s query response without altering the overlay graph or that peer’s degree. Table 4.3 lists the mapping between each attack or defence and the module implementing it.

Table 4.3: Implementation Modules by Attack and Defence Class (DRAG)

Class	Attack Module	Defence Module
Data Poisoning	attacks/data_poisoning.py	defenses/cross_peer_validation.py
Source Selection Manipulation	attacks/routing_manipulation.py	defenses/routing_manipulation_defense.py
Knowledge Extraction	attacks/kb_extraction.py	defenses/query_rate_limiter.py, response_perturbation.py, extraction_anomaly_detector.py
Membership Inference	attacks/membership_inference.py	defenses/privacy_preserving_retrieval.py
Selective Forwarding	Select.Attack/selectiveForwardAttack.py	defenses/selective_forwarding_defense.py
Denial of Service	attacks/ddos_attack.py	defenses/ddos_defense.py

Note: Replication is not listed as a defence module, since it modifies how the network distributes records at construction time rather than intercepting a query. Section 4.4.5 records which of these modules reside outside the main working copy.

4.4.2 Reliable-dRAG Implementation

Reliable-dRAG runs as a set of containers rather than a single process, which changes what an attack can reach and what a measurement can observe. Three data-source services each hold a document collection behind a retrieval endpoint, an orchestrator service queries all three and generates answers, and a local Ethereum node hosts the smart contract that records each source’s reliability and usefulness scores. Services communicate over HTTP, and a single shared credential gates the retrieval endpoints.

Our additions to this system were smaller in number and larger in consequence than those to DRAG. Each data source gained three administrative endpoints: one accepting document insertions, one restoring a source to its original contents, and one reporting a source’s current document count. The insertion endpoint models an adversary who has obtained write access to a source, whether through a compromised operator, a malicious

contributor or an unverified ingestion path, and it triggers a rebuild of that source’s vector index so that inserted documents become retrievable in the same way legitimate ones do. The restoration endpoint exists so that each experimental condition begins from a known state, which matters more here than in DRAG because a container’s document store persists between runs where a simulator’s memory does not.

The attacks against this system reach it through the interfaces an ordinary client or participant would use. Membership inference and knowledge extraction issue HTTP queries. Source selection manipulation writes filler content through the insertion endpoint in its primary form, and submits blockchain transactions in its secondary form. Availability attacks direct concurrent request load at the retrieval services.

4.4.3 Experimental Environment

DRAG experiments ran on networks of ten to twenty peers, generated with two to four attachments per node, two to four neighbours queried per hop, and a hop budget between four and six. Each peer served one of three locally hosted models, Llama 3.2 3B, Gemma 2 2B or Qwen 2.5 3B, through a local inference runtime. Unless a result states otherwise, the reported configuration is twenty peers with four attachments, four query neighbours, a hop budget of six, and Llama 3.2 3B.

Reliable-dRAG offers no comparable topology parameters, since its deployment is fixed at three data sources behind one orchestrator. Its language model is either a locally hosted Qwen 2.5 1.5B Instruct or a hosted model reached by API, and the blockchain component runs as a local development node rather than a public network. This difference in available configuration is what led us to vary parameters on DRAG and random seeds on Reliable-dRAG, as set out in Section 5.1.3.

4.4.4 Execution Protocol

Three practices governed how experiments were run, each addressing a way in which a result could otherwise have been produced by something other than the condition under test.

Isolation of the condition under test. Several attacks and defences are enabled by default in the shipped configuration, and leaving them active would confound any experiment targeting a different class. Poisoning runs therefore disabled membership inference and privacy-preserving retrieval explicitly, since the latter activates automatically whenever defences are enabled and would otherwise have added retrieval noise to the defended condition alone.

Matched attack and defence parameters. Each defended run was configured from the saved configuration of its undefended counterpart rather than from the defence module’s own defaults, which differ from the attack-side defaults in sample size, probe budget and replication factor. Reading parameters from the saved configuration rather than assuming them ensures that a defended and undefended cell differ in the defence alone.

Verification that a run executed as configured. Run logs were checked for the

parameters each condition was intended to carry rather than for successful completion alone. This practice identified a configuration fault in which the replication defence had been recorded as active while its replication factor remained at one, meaning that the condition labelled as replicated had never replicated any record. Every affected experiment was re-run after the fault was corrected, and Chapter 5 reports the corrected results.

4.4.5 Implementation Provenance

The framework’s modules were developed across more than one working copy, and three defences do not reside in the main DRAG repository. We record their locations here because a reader assessing a result is entitled to know which artefact produced it.

The **layered congestion defense** and its corresponding attack were implemented together in a separate repository, which is also the source of every denial of service figure reported in Chapter 5. The **layered extraction prevention** defence resides on a separate branch, whose configuration loader resolves its path relative to the working directory, so that experiments against it were executed from that branch’s directory using the shared runtime environment. Neither arrangement affects the validity of the measurements, and both are recorded so that the results can be reproduced from the correct source.

The **reputation and routing bypass defense** against selective forwarding was evaluated differently, and the difference is methodological rather than administrative. Selective forwarding degrades retrieval before any answer is generated, so answer-quality metrics carry no information about whether the defence succeeded; what matters is whether the router located an alternative peer within its hop budget. We therefore evaluated this defence against a network model with synthetic knowledge availability rather than through the full generation pipeline, which isolates the routing behaviour under test and avoids invoking a language model for every query across a parameter sweep that would not consult its output. The consequence is that the hit rate this evaluation reports depends on the modelled probability that an alternative peer holds relevant knowledge, rather than on retrieval measured against a real corpus, and Section 5.6 records that dependence as a limitation of the result.

The same modelling choice applies to the selective forwarding attack results on both systems. Neither sweep drives the full generation pipeline: DRAG’s sweep runs against the same network model used for its defence, and Reliable-dRAG’s runs against a breadth-first-search model whose topology its three-source deployment does not exhibit. Both are reported as model results in Section 5.2.5, alongside the single live configuration we ran against the real Reliable-dRAG deployment.

Chapter 5

Result Analysis

This chapter reports what our evaluation framework measured when we applied it to DRAG and to Reliable-dRAG. Section 5.1 fixes the metrics, the severity rubric and the testing protocol. Section 5.2 gives the undefended results for all six attack classes on both systems. Section 5.3 reports the defences we added to DRAG in response, and what each one recovered. Sections 5.4 to 5.6 cover the statistical treatment, the cross-architecture comparison that supports our generalisability claim, and the limitations we accept.

5.1 Performance Evaluation

5.1.1 Evaluation Criteria and Metrics

Each attack class carries one primary metric that decides severity, plus secondary metrics that characterise cost or mechanism. Table 5.1 lists the assignment. Answer-quality metrics (Exact Match, F1, Semantic Similarity) serve integrity and availability attacks, since both degrade the answer a user receives. Confidentiality attacks need adversary-side metrics instead, because a successful extraction or inference leaves answer quality untouched.

Table 5.1: Framework Metric Assignment by Attack Class and CIA Property

CIA Property	Attack Class	Primary Metric	Secondary Metrics
Integrity	Data Poisoning	F1 degradation vs. clean baseline	EM, SS, messages per query
	Source Selection Manipulation	F1 degradation, hit rate	Topics hijacked, hops, messages
Confidentiality	Knowledge Extraction	Extraction Rate	CRR, SS, EED, query efficiency, topic coverage
	Membership Inference	AUC-ROC	Accuracy, TPR, FPR, precision, Privacy Risk Score
Availability	Selective Forwarding	Hit rate, dropped queries	TTL exhaustion, hops per query
	Denial of Service	Query failure rate, availability	EM, F1, SS, hops per query

Note: CRR=Chunk Recovery Rate, EED=Effective Edit Distance, SS=Semantic Similarity, TPR/FPR=True/False Positive Rate. Metric definitions and formulae appear in Section 4.2 alongside each attack’s design.

5.1.2 Severity Classification

Comparing an extraction rate against an AUC-ROC requires a shared scale. We map each primary metric onto five tiers, extending the Privacy Risk Score bands defined in Section 4.2.3 to the remaining attack classes. Table 5.2 gives the thresholds. Section 5.5 uses these tiers to place both systems side by side.

Table 5.2: Severity Rubric Applied Across Attack Classes

Tier	Integrity / Availability (quality degradation)	Knowledge Extraction (extraction rate)	Membership Inference (Privacy Risk Score)
Critical	> 75%	> 75%	> 0.90
High	50%–75%	50%–75%	0.75–0.90
Medium	25%–50%	25%–50%	0.60–0.75
Low	10%–25%	10%–25%	0.50–0.60
Negligible	< 10%	< 10%	≤ 0.50

Note: Degradation is measured against each configuration’s own clean baseline rather than against an absolute floor, so a system with a weak baseline is not penalised twice. Privacy Risk Score combines Accuracy (40%), TPR (30%) and AUC-ROC (30%) as defined in Section 4.2.3.

5.1.3 Testing Methodology

The two target systems admit different forms of variation, and we varied each along the axis it exposes.

DRAG runs as a parameterised simulator. Network size, peer attachments, query neighbours, hop budget, attack ratio and per-attack parameters such as poison ratio and topic claim ratio are all controllable, so we swept those parameters and held the random seed fixed at 0. Sweep position carries the information here: a defence that holds across a monotonic parameter range has been tested more severely than one measured once with error bars.

Reliable-dRAG runs as a fixed deployment of three data-source containers behind an LLM orchestrator and a local Hardhat node. None of DRAG’s topology parameters exist, so stochastic variation across seeds is the only axis available, and we report the mean and standard deviation over seeds 0, 42 and 123 for every attack that permits repetition.

Unless a table states otherwise, DRAG results use a 20-peer Barabási–Albert network with 4 peer attachments, 4 query neighbours, a hop budget of 6, 100 evaluation questions per run, and Llama 3.2 3B as the local model. We disabled membership inference and privacy-preserving retrieval during poisoning runs so that neither confounded the measurement, and matched every defended run’s parameters cell by cell against its undefended counterpart from the saved run configuration.

5.2 Analysis of Design Solutions

This section reports each attack against both systems with no defence active. Results are grouped by CIA property so that the two architectures stay adjacent throughout.

5.2.1 Integrity: Data Poisoning

Table 5.3: Data Poisoning Attack Results (DRAG, MMLU Dataset, No Defense)

PR	Scenario	F1 (%)	SS	# Messages
0	Base	82.00	0.909	8.41
20%	Data-Rich	72.00	0.880	6.55
	High-Degree	66.00	0.854	6.41
	Random	75.00	0.900	6.81
30%	Data-Rich	63.00	0.832	5.99
	High-Degree	68.00	0.858	6.35
	Random	63.00	0.837	5.92

Note: PR=Poisson Ratio, SS=Semantic Similarity. Exact Match is omitted because MMLU answers are single multiple-choice symbols, which forces F1 to coincide with Exact Match in every run; we verified the identity numerically against the raw metrics across all 19 runs. Defended results appear in Table 5.24.

Table 5.4: Data Poisoning Attack Results Across Datasets (DRAG, High-Degree Targeting, No Defense)

Dataset	PR	EM (%)	F1 (%)	SS	# Messages
MMLU	0	82.00	82.00	0.909	8.41
	10%	69.00	69.00	0.848	6.07
	20%	66.00	66.00	0.854	6.41
	30%	68.00	68.00	0.858	6.35
News	0	74.00	78.30	0.791	9.21
	10%	48.00	54.28	0.616	7.83
	20%	60.00	64.68	0.683	7.04
	30%	54.00	60.42	0.661	7.53
Medical	0	87.00	96.45	0.969	10.05
	10%	65.00	83.63	0.887	7.10
	20%	65.00	81.49	0.871	7.92
	30%	62.00	82.22	0.880	7.47

Note: High-Degree targeting throughout. EM and F1 coincide on MMLU and diverge on News and Medical, both free-text tasks. Defended counterparts appear in Table 5.25.

Table 5.3 places High-Degree targeting as the most damaging strategy at a 20% poison ratio, dropping F1 from a clean 82.00% to 66.00% against Random’s 75.00%. That ad-

vantage does not survive the move to 30%, where Data-Rich and Random both reach 63.00% while High-Degree recovers to 68.00%. Inspecting the saved poisoned-peer sets explains the crossover: selection is deterministic and nested within a strategy, so the 30% sets extend the 20% sets, and Data-Rich’s two additional peers at 30% include one that also sits inside High-Degree’s own top four by connectivity. Data-Rich therefore picks up genuine hubs on top of being data-rich, while High-Degree’s marginal additions rank fifth and sixth by degree in a 20-peer graph and carry little extra routing weight. We report High-Degree as the strongest strategy at lower poison ratios rather than across the sweep.

Table 5.4 extends the attack to News and Medical. Severity tracks the task. Medical starts at 96.45% F1 and loses at most 15 points, News starts at 78.30% and loses up to 24 points, and MMLU sits between them. News’s undefended sequence is not monotonic: 10% poisoning (54.28% F1) damages the system more than 20% (64.68%). Poisoned-peer selection is deterministic for a given seed, strategy and ratio but not nested across ratios under this dataset’s topic assignment, so which specific topics a selection happens to cover can outweigh the ratio itself in a 20-peer network. We report the reversal rather than smoothing it. Medical’s undefended sequence (83.63, 81.49, 82.22) sits close enough to flat that the same sampling effect plausibly covers both cases.

Reliable-dRAG

Table 5.5: Data Poisoning Attack Results (Reliable-dRAG, No Defense)

Attack	Poison Type	Attacked Acc. (mean $\pm$ std)	Degradation % (mean $\pm$ std)	Success Rate
Random	Noise	52.4% $\pm$ 21.8%	8.3 $\pm$ 38.2	1/3 (33%)
Random	Answer-Swap	42.9% $\pm$ 14.3%	25.0 $\pm$ 25.0	2/3 (67%)
Targeted (s_100)	Wrong-Answer	66.7% $\pm$ 8.2%	-16.7 $\pm$ 14.4	0/3 (0%)
Targeted (s_0, s_20)	Misleading	19.0% $\pm$ 8.2%	66.7 $\pm$ 14.4	3/3 (100%)
Data-Rich	Wrong-Answer	66.7% $\pm$ 8.2%	-16.7 $\pm$ 14.4	0/3 (0%)
Data-Rich	Noise	47.6% $\pm$ 16.5%	16.7 $\pm$ 28.9	1/3 (33%)

Note: Clean-baseline corpus of 3,197 documents per source, clean accuracy 57.1% (4/7 eval questions), averaged over seeds 0, 42 and 123. Success means degradation above 10.0%. Negative degradation reflects a single eval question that triggers a context-length overflow on the clean baseline in most runs rather than a defensive effect of the attack. With $n=7$ eval questions, each question carries 14.3 accuracy points.

Data Poisoning Attack Results (Reliable-dRAG, No Defense)

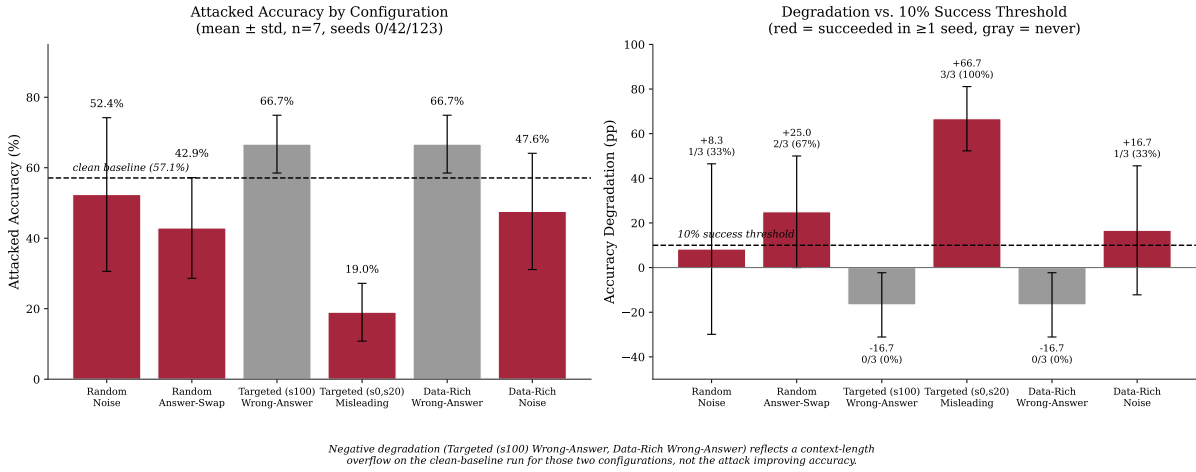


Figure 5.1: Data poisoning on Reliable-dRAG: attacked accuracy against the clean baseline, and degradation against the 10% success threshold, by source-selection strategy and poison type (undefended, mean over seeds 0, 42 and 123).

Table 5.5 and Figure 5.1 records four of the six configurations succeeding in at least one seed, so poisoning is not a narrow single-configuration threat against Reliable-dRAG. One configuration succeeds in every seed: Targeted against `sources_0` and `sources_20` with misleading poison, at 66.7% mean degradation. The two configurations that never succeed both use the generic `wrong_answer` text, a fixed content-free string with nothing question-specific to win a retrieval slot with, unlike misleading, answer-swap and noise, which retain or repurpose real corpus text. Data-Rich selection reduces to a random tie-break here, since all three deployed sources currently hold an identical corpus.

5.2.2 Integrity: Source Selection Manipulation

Table 5.6: Routing Manipulation Attack Results Across Datasets (DRAG, 6 attacker peers, topic claim ratio 0.6, No Defense)

Dataset	EM	F1	Hit Rate	SS	Avg Hops	Avg Msgs
Medical	0.82	0.9060	0.94	0.9168	2.10	9.03
MMLU	0.68	0.6800	0.82	0.7615	2.48	7.13
News	0.56	0.5232	0.62	0.5226	3.30	7.19

Note: Topics hijacked is fixed by the attack configuration at 34, 24 and 19 for Medical, MMLU and News respectively. Hop and message counts come from each run’s standard evaluation output. Defended counterparts appear in Table 5.26.

Table 5.7: Routing Manipulation Attack: Topic Claim Ratio Sweep (DRAG, MMLU, No Defense)

Claim Ratio	Topics Hijacked	EM / F1	SS	Hit Rate	Avg Msgs
0.2	8	0.77	0.859	0.93	7.43
0.4	16	0.73	0.817	0.88	7.53
0.6	24	0.68	0.762	0.82	7.13
0.9	36	0.59	0.657	0.71	6.25

Note: Attacker ratio, targeting strategy and response type held at the values used in Table 5.6; only the topic claim ratio varies. The MMLU network carries approximately 41 topics. Defended counterparts appear in Table 5.27.

Undefended degradation in Table 5.7 scales with the claim ratio without exception. F1 falls 0.77, 0.73, 0.68, 0.59 and hit rate falls 0.93, 0.88, 0.82, 0.71 as the ratio rises from 0.2 to 0.9. Topics hijacked tracks the ratio at almost exactly $\text{ratio} \times 40$ against the network’s 41 topics, which confirms the sweep measures the intended mechanism rather than sampling noise. Message count falls slightly as the ratio rises, from 7.43 to 6.25, consistent with more queries exhausting the hop budget and terminating early once a large share of topics has been hijacked.

Reliable-dRAG

Table 5.8: SSM-Score Attack: Multi-Seed Results (Reliable-dRAG, No Defense)

Attack	Seed	Engagement Rate (%)	R Delta	U Delta	Accuracy Drop (pp)	Outcome
GF	0	99.67	+680	+1,574	9.0	Full escalation
GF	42	0.33	-20	+20	-2.0	No catch
GF	123	11.00	-4	+216	-1.0	No catch (marginal)
KF	0	100.00	+20,000	+20,000	8.0	Full success
KF	42	100.00	+20,000	+20,000	12.0	Full success
KF	123	100.00	+20,000	+20,000	0.0	Full success

Note: GF=Grounding-Farming, KF=Key-Forger. Engagement Rate is `farm_source_in_importance_score` for Grounding-Farming ($n=100$ questions, 300 rounds/seed) and `rounds_accepted_fraction` for Key-Forger ($n=50$ questions, 5 rounds/seed). R/U Delta are absolute changes in the source’s reliability and usefulness scores over the run.

SSM-Score Attack: Multi-Seed Results (Reliable-dRAG, No Defense)

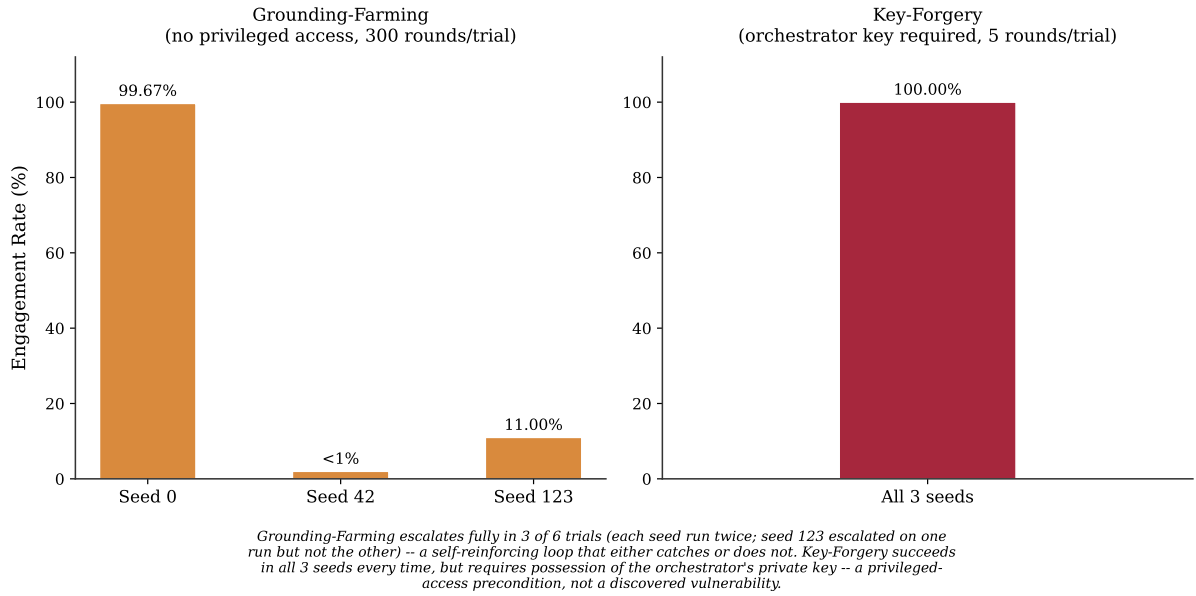


Figure 5.2: SSM-Score on Reliable-dRAG: per-trial engagement for Grounding-Farming, which escalates in three of six independent trials, alongside Key-Forgery, which succeeds in every trial but requires the orchestrator’s private key.

The two instantiations in Table 5.8 and Figure 5.2 trade privilege against reliability. Key-Forgery requires the orchestrator’s private key and succeeds in every seed, driving both on-chain scores to their +20,000 ceiling. Grounding-Farming requires no privileged access at all, only control of one participating source. Its success depends on tripping a self-reinforcing loop in the reranker’s own reliability weighting, which either catches early in a run or does not catch at all. Section 5.5 returns to this pair, since the mechanism Grounding-Farming exploits has no counterpart in DRAG and the mechanism DRAG’s routing manipulation exploits has no counterpart here.

5.2.3 Confidentiality: Knowledge Extraction

Table 5.9: Knowledge Extraction Attack Results by Network Topology and Dataset (DRAG, No Defense)

Dataset	Peers	Conn.	ER (%)	CRR (%)	SS	EED	TC (%)
MMLU	10	Low	25.00	28.00	0.7900	0.5000	63.41
		High	30.00	33.00	0.8300	0.4000	73.17
	20	Low	20.00	23.00	0.7300	0.6000	68.29
		High	20.00	22.00	0.7400	0.6000	63.41
News	10	Low	13.00	14.00	0.4566	0.6085	72.73
		High	7.00	10.00	0.3388	0.7142	81.82
	20	Low	5.00	8.00	0.2969	0.7531	81.82
		High	6.00	10.00	0.3283	0.7221	75.76
Medical	10	Low	2.00	12.00	0.6033	0.5604	66.67
		High	5.00	21.00	0.6921	0.4240	61.40
	20	Low	1.00	3.00	0.5554	0.6571	68.42
		High	2.00	10.00	0.6089	0.5628	66.67

Note: Conn.=Connectivity, ER=Extraction Rate, CRR=Chunk Recovery Rate, SS=Semantic Similarity and EED=Effective Edit Distance, both computed over extracted chunks only, TC=Topic Coverage. Low connectivity uses 2 peer attachments, 2 query neighbours and TTL 4; High uses 4, 4 and 6. Query Efficiency is omitted because it reads 100.00% in every row: the attack issues its network queries with `query_confidence_threshold=0.0`, so any non-empty response registers as a hit regardless of relevance. A budget of 5 queries per topic over 10 topics caps the reachable extraction rate at 50%. Defended results appear in Table 5.28.

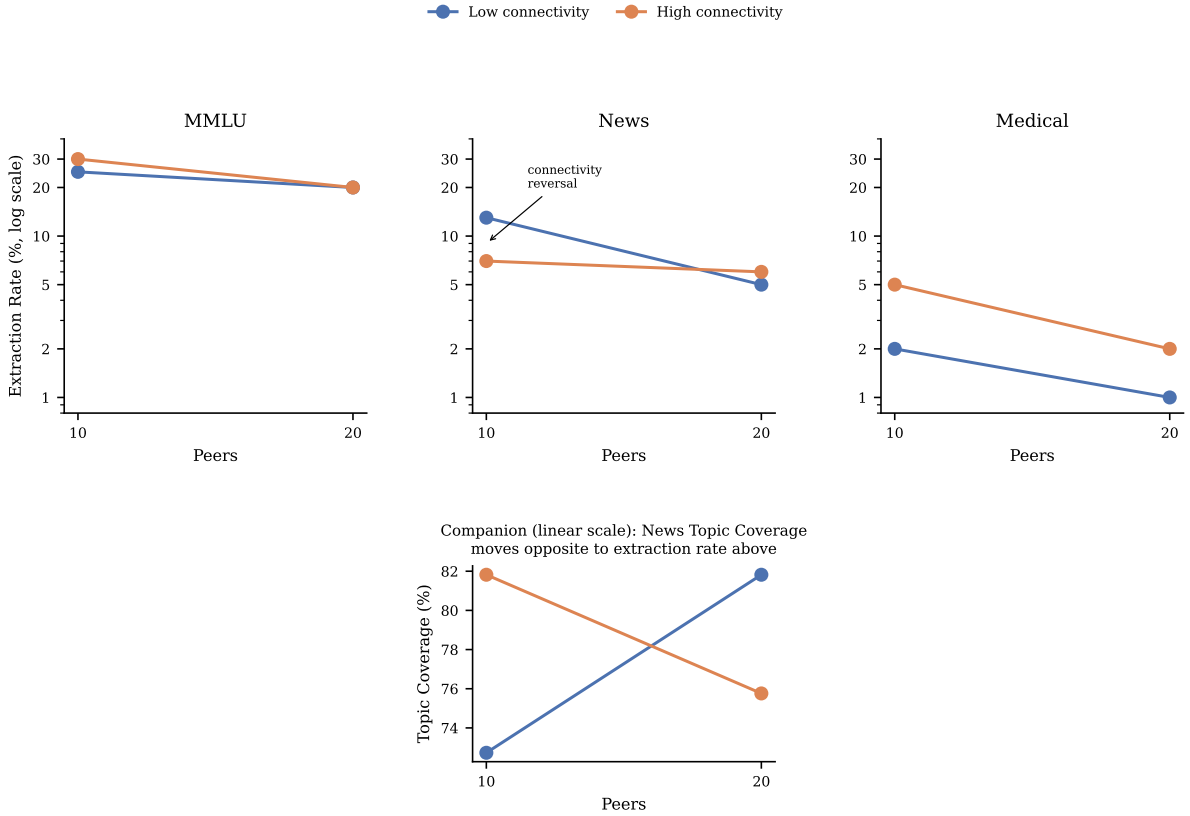


Figure 5.3: Extraction rate by peer count and connectivity, per dataset, with News topic coverage shown alongside (DRAG, undefended). Extraction axes are logarithmic and shared.

Table 5.9 separates two network parameters that a deployment can control. Network size behaves consistently: extraction falls when the same corpus is spread across twice as many peers, in all six matched comparisons and on every dataset. The effect is largest on News, where extraction drops from 13.00% to 5.00% at low connectivity, and smallest on Medical, where it falls from 2.00% to 1.00% against an already low base. Distributing a corpus more widely hands an attacker fewer complete records for a given query budget.

Connectivity behaves inconsistently. Raising it helps the attacker on MMLU at 10 peers, lifting extraction from 25.00% to 30.00%, and on Medical, from 2.00% to 5.00%, but reverses on News, where extraction falls from 13.00% to 7.00%. The reversal comes with a rise in topic coverage, from 72.73% to 81.82%, and the two moving in opposite directions identifies the mechanism: a fixed budget of five queries per topic spread across more topics recovers fewer records within each one. On MMLU a single option symbol is recovered from shallow coverage, whereas a News answer is an article byline that must be reproduced close to verbatim, so thin coverage yields little. A deployment tuning for extraction resistance should therefore treat network size as the reliable parameter and connectivity as one whose effect depends on how its corpus is structured.

Dataset structure dominates both. Extraction reaches 20–30% on MMLU and 1–13%

on News and Medical across every topology tested. MMLU answers are one of roughly four single-character options, so an attacker recovers a record by matching one symbol, whereas News and Medical answers are free text requiring near-verbatim reproduction before the metric counts them. Extraction rates in the 20–30% range are a property of multiple-choice corpora rather than of distributed RAG retrieval, and a deployment estimating its own exposure from published extraction figures should establish what answer format produced them.

Reliable-dRAG

Table 5.10: Knowledge Extraction Attack Results (Reliable-dRAG, Authenticated Attacker, No Defense)

Phase	Source (Dataset)	Extraction Rate	Extraction Acc.	Topic Coverage	Query Efficiency
Gray-Box (Authorized)	source_0 (pubmedqa)	0.4193	1.000	n/a	3.8827
	source_1 (squad)	0.3927	1.000	1.00	3.6358
	source_2 (squad)	0.3727	1.000	0.97	3.4506
Blind / Cold-Start	source_0 (pubmedqa)	0.2633	1.000	n/a	2.4383
	source_1 (squad)	0.3820	1.000	1.00	3.5370
	source_2 (squad)	0.3733	1.000	1.00	3.4568

Note: Mean of 3 seeds (0, 42, 123), 54 probes per seed per source. Extraction Accuracy is structurally bounded at 1.0, since the raw retrieval endpoint returns stored passages verbatim and never fabricates.

Gray-box probes are pre-filtered to guarantee a hit and give an upper bound on post-reconnaissance efficiency; blind probes are sampled without that filter and give the floor for an attacker with no prior knowledge of the deployment. A third unauthenticated condition is not tabulated because the shared API-key gate rejected all 54 probes per source with HTTP 401.

Table 5.11: Knowledge Extraction Attack Results, Indirect LLM Leakage (Reliable-dRAG, Phase C)

Seed	Exact Match Rate	Semantic Similarity	Edit Similarity	Chunk Recovery Rate
0	0.300	0.5479	0.3250	0.300
42	0.150	0.5432	0.1986	0.200
123	0.300	0.5347	0.3000	0.300
Mean (n=3)	0.250	0.5419	0.2745	0.267

Note: 20 probes routed through the orchestrator’s /query endpoint rather than the raw retrieval endpoint. A recovery counts if Exact Match, Semantic Similarity ≥ 0.8 , or Edit Similarity ≥ 0.8 holds, a deliberately attacker-favourable disjunction.

Reliable-dRAG Knowledge Extraction: Access-Level Escalation

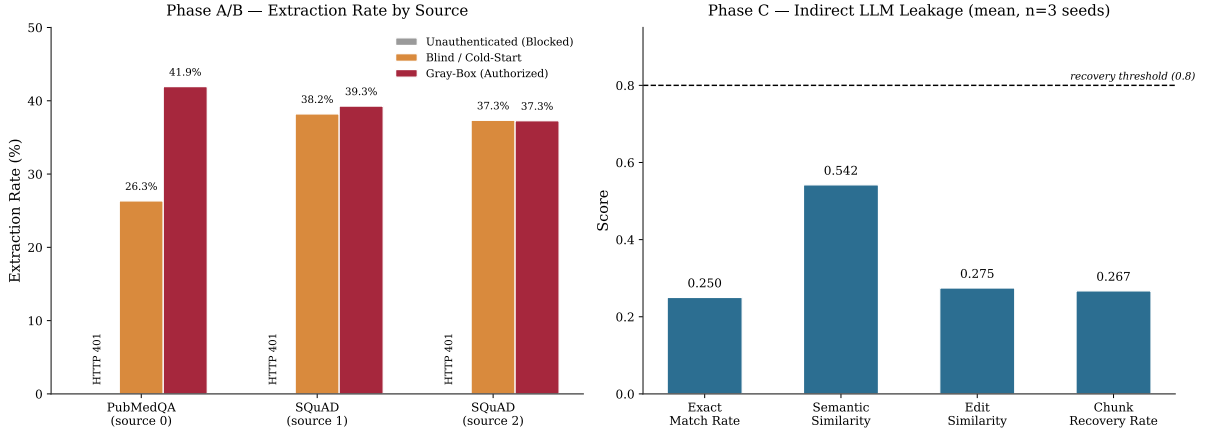


Figure 5.4: Knowledge extraction on Reliable-dRAG by access level: extraction rate per source across the unauthenticated, blind and gray-box conditions, and indirect leakage through the orchestrator against the 0.8 recovery threshold.

Reliable-dRAG’s gray-box extraction in Table 5.10 and Figure 5.4 reaches 37.3–41.9% across sources against DRAG’s 20–30% on MMLU, so both systems leak a substantial share of private content through the legitimate query interface despite the architectural distance between multi-hop peer routing and direct per-source HTTP querying. The blind condition tracks gray-box on the two SQuAD sources (37.3–38.2%) and falls to 26.3% on the PubMedQA source, which puts the cost of reconnaissance at roughly 16 percentage points on that corpus and near zero on the others. Indirect leakage through the orchestrator (Table 5.11) recovers content loosely, at mean semantic similarity 0.5419, a different mechanism from DRAG’s retrieval-based extraction, since the LLM paraphrases retrieved passages rather than returning them.

5.2.4 Confidentiality: Membership Inference

Table 5.12: Membership Inference Attack Results by Model (DRAG, MMLU, No Defense)

Model	Acc.	TPR	FPR	Precision	AUC-ROC	PRS	Risk Level
Llama 3.2 3B	0.78	0.700	0.0333	0.980	0.8795	0.786	High
Gemma 2 2B	0.74	0.6857	0.1333	0.923	0.8676	0.762	High
Qwen 2.5 3B	0.78	0.700	0.0333	0.980	0.8750	0.7845	High

Note: PRS=Privacy Risk Score. $n=100$ per model, split 70 members and 30 non-members.

Table 5.13: Membership Inference Attack Results by Dataset (DRAG, Confidence-Based Inference, No Defense)

Dataset	Acc.	TPR	FPR	Precision	AUC-ROC	Privacy Risk
Medical	0.58	0.5571	0.3667	0.780	0.6605	Low
MMLU	0.80	0.7143	0.0000	1.0000	0.9019	High
News	0.70	1.0000	1.0000	0.700	0.8324	High

Note: News has $TPR=FPR=1.0$, meaning the classifier’s operating threshold labelled every query a member; its 70% accuracy tracks the 70/30 base rate rather than genuine discrimination, so AUC-ROC is the reliable measure there. Defended counterparts appear in Table 5.30.

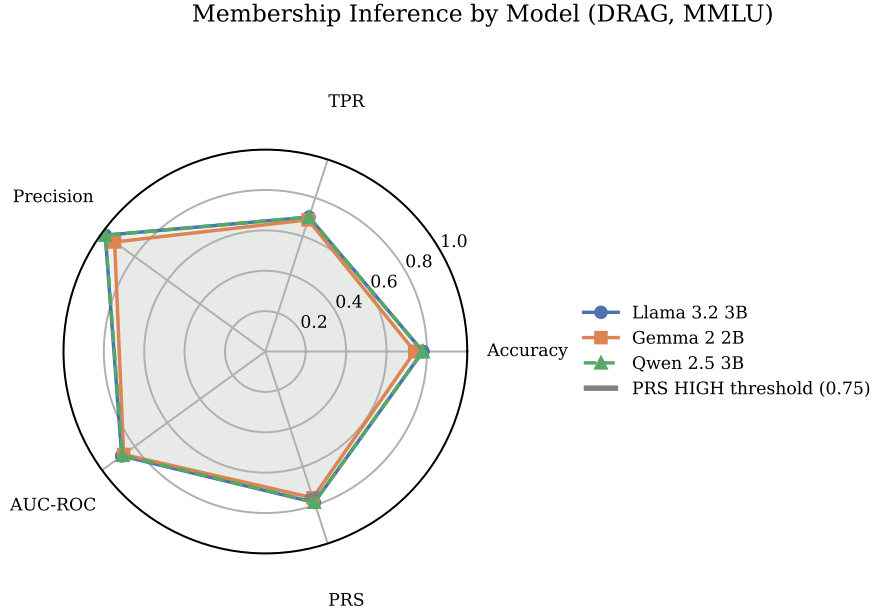


Figure 5.5: Membership inference across five metrics by local model (DRAG, MMLU, undefended). Llama 3.2 3B and Qwen 2.5 3B overlap on every axis.

Table 5.12 and Figure 5.5 place all three models in the HIGH tier, with privacy risk scores between 0.762 and 0.786 and AUC-ROC between 0.8676 and 0.8795. Model choice moves the attack very little at this scale. Llama 3.2 3B and Qwen 2.5 3B return identical confusion matrices (49 true positives, 29 true negatives, 21 false negatives, 1 false positive) and separate only on AUC-ROC, which reads the continuous membership score before the median threshold applies.

That identity follows from how the attack composes its signal. Routing metrics, hop count and message count contribute 30% of the membership score, and those depend on the network rather than the language model, so nearly a third of the discriminative power stays fixed whichever model a peer runs. Membership leakage in this architecture is a property of the retrieval and routing layer at least as much as of the generator, a distinction that centralised RAG evaluations have no way to draw.

Across datasets, MMLU leaks most (AUC-ROC 0.9019), followed by News (0.8324) and Medical (0.6605).

Reliable-dRAG

Table 5.14: Membership Inference Attack Results (Reliable-dRAG, Production Multi-Seed, No Defense)

Seed	Acc.	Precision	Recall	FPR	F1	AUC-ROC	Privacy Risk
0	0.6200	0.5938	0.7600	0.5200	0.6667	0.6200	Medium
42	0.5000	0.5000	1.0000	1.0000	0.6667	0.5800	Low
123	0.5000	0.5000	1.0000	1.0000	0.6667	0.5000	Negligible
Mean (n=3)	0.5400	0.5313	0.9200	0.8400	0.6667	0.5667	Low

Note: PubMedQA-derived corpus, n=25 members and n=25 non-members. We measured these rows under the composite’s earlier weighting, which distributed weight across similarity, certainty and length ratio, and against the deployment before we corrected a stale dataset mount and an outdated model-server build. Table 5.15 carries the current composite and the generalisation estimate. FPR reaching 1.0 at seeds 42 and 123 means the threshold labelled every query a member, and F1 stays at 0.6667 across seeds because recall and FPR move together at those thresholds, so AUC-ROC is the reliable measure here. No defence analogous to DRAG’s Privacy-Preserving Retrieval has been implemented for Reliable-dRAG.

Table 5.15: Membership Inference Composite Selection and Held-Out Confirmation (Reliable-dRAG)

Weighting	Evaluation Set	n	Mean AUC	95% CI	Excludes 0.50
Similarity alone	Diagnostic	10	0.4732	n/a	No (below chance)
Decision 0.55, sim 0.20, certainty 0.10, length 0.15	Diagnostic	10	0.522	[0.472, 0.572]	No
Decision 0.70, length 0.30	Diagnostic	10	0.552	[0.485, 0.618]	No
Decision-match alone	Diagnostic	10	0.568	[0.526, 0.610]	Yes
Decision-match alone (production, weights locked)	Held-out	5	0.620	[0.546, 0.695]	Yes

Note: Diagnostic rows report the ten seeds available during weight selection. Those seeds informed the final choice of weighting and therefore do not constitute held-out evidence, which is why we reserve the last row for the single evaluation we ran after locking the weights. That evaluation used five seeds generated for it and never queried before it, scoring 0.62, 0.68, 0.68, 0.56 and 0.56. The dev-set mean across the 13 search seeds is 0.572, below the held-out mean, which indicates the grid search did not overfit to the seeds it ran on. Restoring certainty at 0.20 lowers three further fresh seeds from 0.6133 to 0.6072, confirming the zero weight on the production formula.

Reliable-dRAG Membership Inference: Weight Selection and Held-Out Result

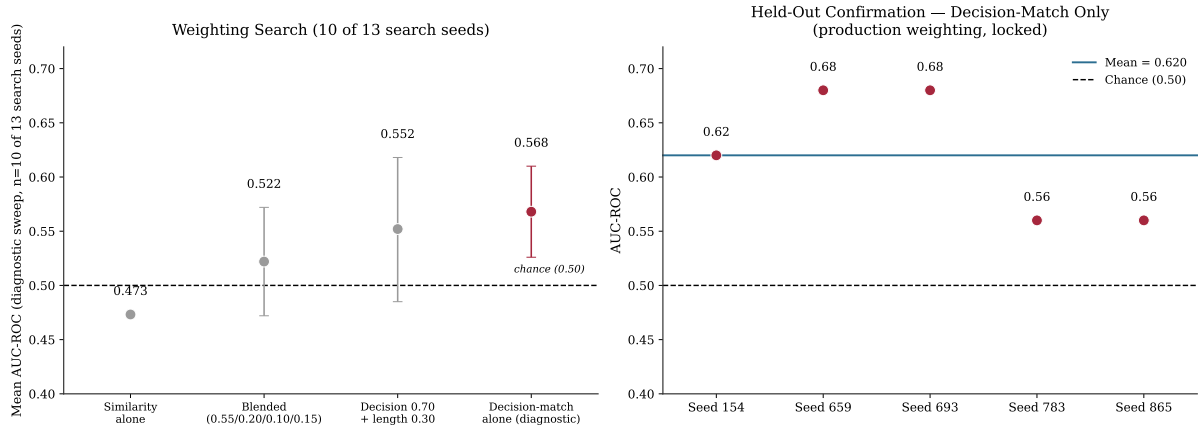


Figure 5.6: Membership inference on Reliable-dRAG: candidate weightings compared across the search seeds, and the locked production weighting evaluated once against five seeds generated after selection.

Reliable-dRAG’s composite reaches mean AUC-ROC 0.620 across five held-out seeds, at a 95% confidence interval of [0.546, 0.695] that excludes chance. The result sits below DRAG’s 0.6605 to 0.9019 range, and the gap follows from signal availability. Reliable-dRAG’s composite rests on a single decision-match term, since an attacker reaches its orchestrator directly and observes no routing behaviour, while DRAG’s composite weights retrieval similarity at 60% and backs it with hop and message counts. Table 5.14 and Figure 5.6 reports the earlier per-seed sweep, which we retain for its accuracy, precision and rate columns rather than as the generalisation estimate, since we measured it under the superseded weighting and against the deployment before we corrected a stale dataset mount and an outdated model-server build.

5.2.5 Availability: Selective Forwarding

Table 5.16: Selective Forwarding Attack Results by Strategy and Ratio (DRAG, No Defense)

Strategy	Ratio	Dropped Queries
Random	0.1	33
Random	0.2	78
Random	0.3	136
Random	0.4	195
Random	0.5	349
High-Connectivity	0.1	76
High-Connectivity	0.2	190
High-Connectivity	0.3	288
High-Connectivity	0.4	432
High-Connectivity	0.5	590

Note: Hit rate holds at 0.95–1.00 across all rows, declining with attack ratio rather than with targeting strategy. Defended counterparts appear in Table 5.31.

Reliable-dRAG

Table 5.17: Selective Forwarding Attack Results by Strategy and Ratio (Reliable-dRAG, Mock Sweep, No Defense)

Strategy	Ratio	Hit Rate	Avg Hops/Query	TTL Exhaustion Rate	Dropped Queries
Baseline	0.0	0.98	2.35	0.02	0
Random	0.1	0.98	2.56	0.02	21
Random	0.2	0.96	2.70	0.04	37
Random	0.3	0.91	3.13	0.09	75
Random	0.4	0.91	3.37	0.09	105
Random	0.5	0.85	3.72	0.15	160
High-Connectivity	0.1	0.90	2.81	0.10	48
High-Connectivity	0.2	0.85	3.25	0.15	121
High-Connectivity	0.3	0.72	3.69	0.28	191
High-Connectivity	0.4	0.65	3.84	0.35	231
High-Connectivity	0.5	0.54	4.41	0.46	311

Note: Mean over 5 attack waves, 100 queries per row. Drop rate is 1.00 for every non-baseline row, so TTL exhaustion rather than drop rate distinguishes severity here.

Table 5.18: Selective Forwarding Attack: Live-Network Validation (Reliable-dRAG, Both Expressible Severities)

Sources Compromised	Ratio	Strategy	Hit Rate (mean $\pm$ std)	Avg Hops/Query (mean $\pm$ std)	TTL Exhaustion (mean $\pm$ std)	Dropped	Queries
1 of 3	0.30	High-Connectivity	1.000 $\pm$ 0.000	2.00 $\pm$ 0.00	0.000 $\pm$ 0.000	200	200
2 of 3	0.67	High-Connectivity	1.000 $\pm$ 0.000	3.00 $\pm$ 0.00	0.000 $\pm$ 0.000	400	200

Note: Each row is the mean over seeds 0, 42 and 123 at 200 questions per seed, run against the live Docker deployment. Sources Compromised rather than Ratio is the severity variable here: `select_targets()` resolves to one compromised source for every ratio from 0.1 to 0.66 on a three-source deployment and to two at 0.67, so these two rows span the full range the architecture can express short of a total blackout. The count was read from each run’s output rather than inferred from the input ratio. Dropped queries scale with the number of compromised sources, at 200 questions times one or two peers visited before the honest source. Average hops is the only metric that moves. Hit rate holds at 1.000 for a structural reason discussed below, and this evaluation exercises no defence: `run_attack.py`’s live mode attaches none, so the redundant-probe mechanism described in Section 4.2.6 is not active in these runs.

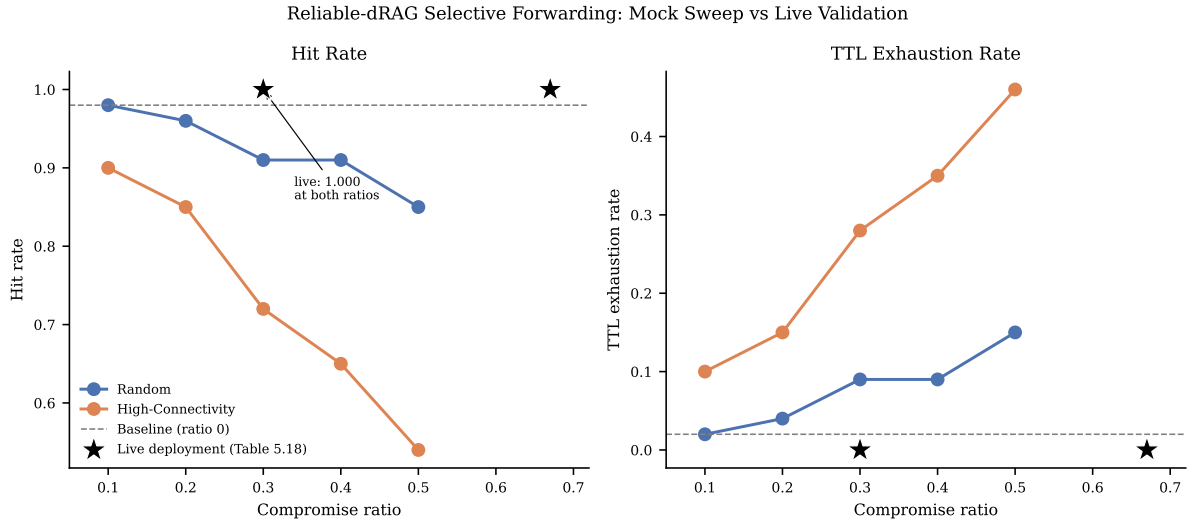


Figure 5.7: Selective forwarding on Reliable-dRAG: simulated sweep across compromise ratio, with the two live measurements overlaid (hit rate and TTL exhaustion, unfenced).

Table 5.18 measures the live deployment at both severities its topology admits, and answer quality holds at every one. What recovers those queries is graph connectivity rather than any countermeasure. Reliable-dRAG’s three sources are fully connected and its hop budget of three reaches every one of them, so a query walks past each compromised source and arrives at an honest one within budget on every attempt. The traversal order is identical across runs, since the router sorts peers by on-chain reliability and all three sit tied at their initial value, and the surviving source returned at least one document for all 600 queries issued across the three seeds. A miss was available only if that source had returned nothing.

This sets the live result against the mock sweep rather than confirming it. The simulation puts hit rate at 0.72 for the same strategy and compromise level, and at 0.54 once

half its twenty peers are compromised, where the live deployment reports 1.000 at both of its own severities. The divergence is now measured at two points instead of one and has a mechanical explanation: a twenty-peer network under a bounded traversal can exhaust its budget before reaching an honest peer, and a three-peer network whose budget covers every peer cannot. Severity for this attack therefore depends on the ratio of hop budget to network size, which is a deployment parameter rather than a property of the attack, and Section 5.6 records what that leaves unresolved.

5.2.6 Availability: Denial of Service

Table 5.19: DoS Attack Results, Mid Intensity (DRAG, All Datasets, No Defense)

Dataset	Strategy	QFR (%)	Avail. (%)	EM	F1	SS	Avg Hops
Medical	Baseline	0.00	100.0	0.7917	0.8913	0.9225	1.96
	Random	35.00	10.0	0.5333	0.6007	0.6301	3.72
	Targeted	37.50	10.0	0.5000	0.5711	0.6024	3.63
	Sequential	36.67	0.0	0.5167	0.5800	0.6089	3.88
MMLU	Baseline	0.00	100.0	0.8750	0.8750	0.9287	1.14
	Random	35.00	10.0	0.5833	0.5833	0.6052	3.18
	Targeted	37.50	10.0	0.5333	0.5333	0.5546	3.19
	Sequential	36.67	0.0	0.5583	0.5583	0.5763	3.43
News	Baseline	0.00	100.0	0.8500	0.8500	0.8500	1.78
	Random	35.00	10.0	0.5667	0.5667	0.5534	3.58
	Targeted	37.50	10.0	0.5583	0.5583	0.5482	3.52
	Sequential	36.67	0.0	0.5500	0.5500	0.5386	3.69

Note: QFR=Query Failure Rate. QFR and Availability are identical across datasets for a given strategy, since congestion state operates on peer indices rather than content; EM, F1 and SS vary because they score drops as zero. Defended counterparts appear in Table 5.32.

Table 5.20: DoS Attack Results, High Intensity (DRAG, All Datasets, No Defense)

Dataset	Strategy	QFR (%)	Avail. (%)	EM	F1	SS	Avg Hops
Medical	Baseline	0.00	100.0	0.7917	0.8913	0.9225	1.96
	Random	48.33	5.0	0.4000	0.4725	0.5078	4.06
	Targeted	49.17	5.0	0.4083	0.4719	0.5047	3.97
	Sequential	53.33	0.0	0.3750	0.4403	0.4739	4.00
MMLU	Baseline	0.00	100.0	0.8750	0.8750	0.9287	1.14
	Random	48.33	5.0	0.4750	0.4750	0.4790	3.77
	Targeted	49.17	5.0	0.4500	0.4500	0.4592	3.63
	Sequential	53.33	0.0	0.4083	0.4083	0.4144	3.67
News	Baseline	0.00	100.0	0.8667	0.8667	0.8667	1.69
	Random	48.33	5.0	0.4583	0.4583	0.4459	4.01
	Targeted	49.17	5.0	0.4417	0.4417	0.4290	3.91
	Sequential	53.33	0.0	0.4250	0.4250	0.4116	3.98

Note: Defended counterparts appear in Table 5.33.

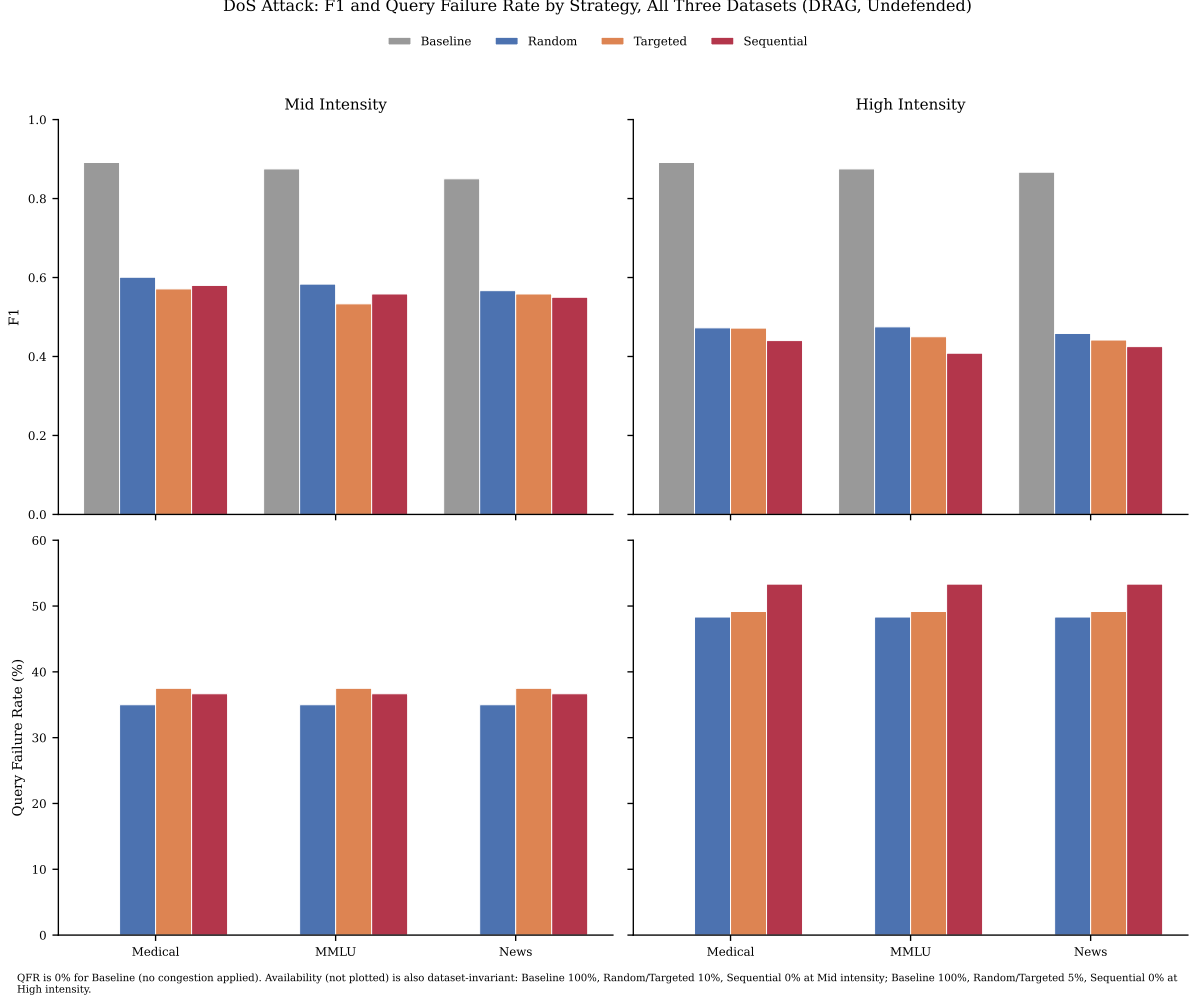


Figure 5.8: Query failure rate and answer quality by targeting strategy and intensity, across all three datasets (DRAG, undefended).

Tables 5.19 and 5.20, plotted in Figure 5.8, place Sequential targeting as the most damaging strategy against DRAG at both intensities, driving availability to 0.0% where Random and Targeted leave 5–10%. A round-robin sweep touches every peer in turn, so the router never finds an uncongested region to fall back on. Answer quality follows the same ordering at high intensity, where Sequential produces the lowest F1 on every dataset. At mid intensity the ordering does not hold: Targeted scores below Sequential on Medical and MMLU, and Sequential’s distinction there is that it congests every peer rather than that it degrades answers furthest.

Reliable-dRAG

Table 5.21: DoS Attack Results by Strategy and Ratio (Reliable-dRAG, Mock Sweep, No Defense)

Strategy	Ratio	Hit Rate	Avail. (%)	Avg Hops/Query	TTL Exhaustion	Down Count	Overloaded
Baseline	0.0	0.960	100.00	2.50	0.040	0.0	0.0
Random	0.1	0.893	81.00	3.07	0.107	3.8	15.5
Random	0.3	0.780	51.00	3.55	0.220	9.8	19.2
Random	0.6	0.670	22.33	4.05	0.330	15.5	20.0
Sequential	0.1	0.843	85.67	3.15	0.157	2.9	16.9
Sequential	0.3	0.677	51.00	3.85	0.323	9.8	19.7
Sequential	0.6	0.587	21.67	4.37	0.413	15.7	20.0
Targeted	0.1	0.897	85.67	2.88	0.103	2.9	15.0
Targeted	0.3	0.763	40.33	3.70	0.237	11.9	19.6
Targeted	0.6	0.610	14.00	4.19	0.390	17.2	20.0

Note: Mean over 5 attack waves and 3 seeds (0, 42, 123).

Table 5.22: DoS Attack Results: Live-Network Answer-Quality Degradation (Reliable-dRAG, No Defense)

Intensity	Condition	EM	F1	SS	Avg Hops/Query	Avg Messages	Query Failure Rate
Low	Baseline	0.60	0.60	0.8315	1.00	2.0	0.00
	Post-Attack	0.40	0.40	0.7782	2.00	4.0	0.00
Mid	Baseline	0.60	0.60	0.8267	1.00	2.0	0.00
	Post-Attack	0.30	0.30	0.6049	3.00	6.0	0.20
High	Baseline	0.60	0.60	0.8267	1.00	2.0	0.00
	Post-Attack	0.00	0.00	0.0448	3.00	6.0	0.90

Note: Seed 42 only. Real concurrent HTTP floods against 1, 2 and 3 data sources for low, mid and high intensity. Average messages equals average hops times 2 by construction rather than as a separate measurement.

Reliable-dRAG DoS: Mock Sweep and Live-Flood Validation

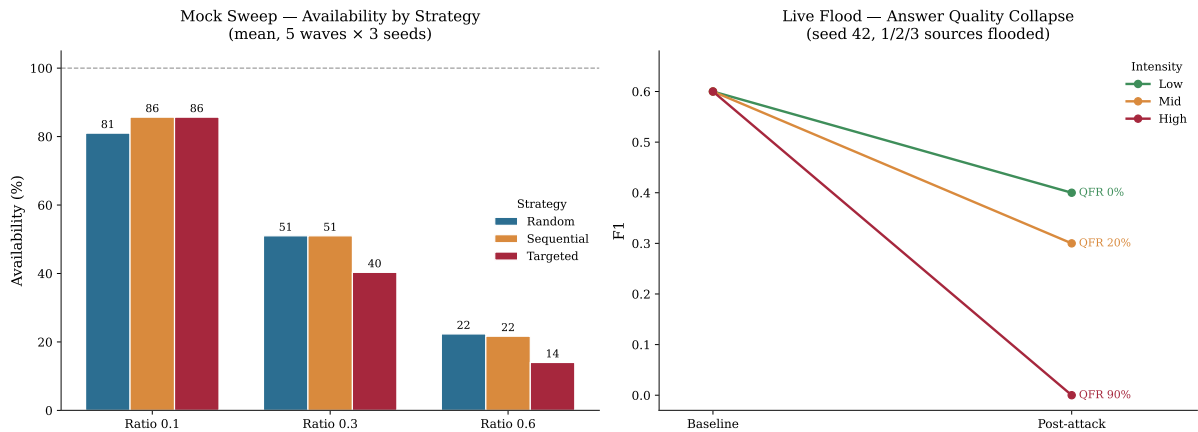


Figure 5.9: Mock sweep availability drop and answer-quality collapse under live flood. Query failure rate is annotated at each endpoint.

Table 5.21 inverts DRAG’s strategy ordering. Targeted selection is the most damaging here, cutting availability to 14.00% at ratio 0.6 against Sequential’s 21.67% and Random’s 22.33%. A 20-node mock topology lets a targeted attacker concentrate damage faster than a round-robin sweep can spread it, while DRAG’s larger peer count rewards the sweep. The live evaluation in Table 5.22 reaches a more severe failure than any simulated condition on either system: flooding all three data sources drives F1 to 0.00 and semantic similarity to 0.0448 at a 90% query failure rate, since three sources constitute the entire retrieval backend and removing them leaves nothing to route to.

5.3 Final Design Adjustments

Each defence in this section answers a specific finding from Section 5.2. We implemented all five on DRAG, which serves as the primary demonstration of how a system consumes the framework’s output. Table 5.23 maps each finding to the adjustment we made and the recovery we measured.

Table 5.23: Evaluation Finding, Design Adjustment, and Measured Result

Finding from Section 5.2	Design Adjustment	Measured Result
Poisoning a hub peer degrades F1 to 66% because the system trusts the first answer it retrieves	Replication (factor 3) plus cross-peer validation with 60% quorum	F1 restored to 80–84% in all 6 cells; messages rise from 6 to 8.2–8.7
TARW trusts advertised topic expertise, so a peer claiming topics it lacks is routed to first	Topic-claim quota at three times the network median, MAD-based	F1 0.68 to 0.80 on MMLU; recovery grows with claim ratio; messages roughly double
Extraction succeeds through topic breadth, not query volume alone	Rate limiter, response perturbation and topic-diversity anomaly detection	Extraction rate to 0.00% in all 4 topologies; query volume cut 82%
Retrieval similarity carries 60% of the membership signal	Laplace noise on retrieval similarity scores	AUC-ROC 0.8795 to 0.7571 at $b = 0.1$, holding 81% task F1
Compromised peers pass liveness checks while dropping queries	Reputation tracking with blacklisting and bounded redundant probes	Dropped queries cut 24–75%, rising with attack severity
Congested peers stay reachable, so the router keeps sending to them	Adaptive routing weight, circuit breaker, query hedging, load shedding	Query failure 35% to 12.5–19.2% at mid intensity

5.3.1 Replication and Cross-Peer Validation

Table 5.24: Data Poisoning Defense Results (DRAG, MMLU Dataset)

PR	Scenario	Defense	F1 (%)	SS	# Messages
20%	Data-Rich	Replication	80.00	0.917	3.69
	Data-Rich	Replication+CPV	83.00	0.927	8.54
	High-Degree	Replication	78.00	0.909	3.53
	High-Degree	Replication+CPV	83.00	0.922	8.27
	Random	Replication	78.00	0.901	3.47
	Random	Replication+CPV	84.00	0.935	8.22
30%	Data-Rich	Replication	77.00	0.896	3.17
	Data-Rich	Replication+CPV	82.00	0.925	8.26
	High-Degree	Replication	74.00	0.887	3.50
	High-Degree	Replication+CPV	81.00	0.909	8.72
	Random	Replication	78.00	0.905	3.26
	Random	Replication+CPV	80.00	0.917	8.22

Note: Replication=(*replication_factor*=3), CPV=Cross-Peer Validation. Compare against Table 5.3, whose clean baseline is 82.00% F1.

Table 5.25: Data Poisoning Defense Results Across Datasets (DRAG, High-Degree Targeting, Replication + Cross-Peer Validation)

Dataset	PR	EM (%)	F1 (%)	SS	# Messages
MMLU	10%	84.00	84.00	0.931	8.45
	20%	83.00	83.00	0.922	8.27
	30%	81.00	81.00	0.909	8.72
News	10%	70.00	77.33	0.788	9.75
	20%	68.00	74.33	0.764	9.96
	30%	67.00	73.49	0.768	9.41
Medical	10%	87.00	96.96	0.978	10.68
	20%	84.00	95.98	0.975	10.53
	30%	82.00	94.84	0.968	10.26

Note: Replication-only is omitted here, since it functions as an ablation step within Table 5.24 rather than as a defence we propose alone. Compare against Table 5.4.

Data Poisoning Attack & Defense: F1 Recovery and Message Cost (DRAG, MMLU)

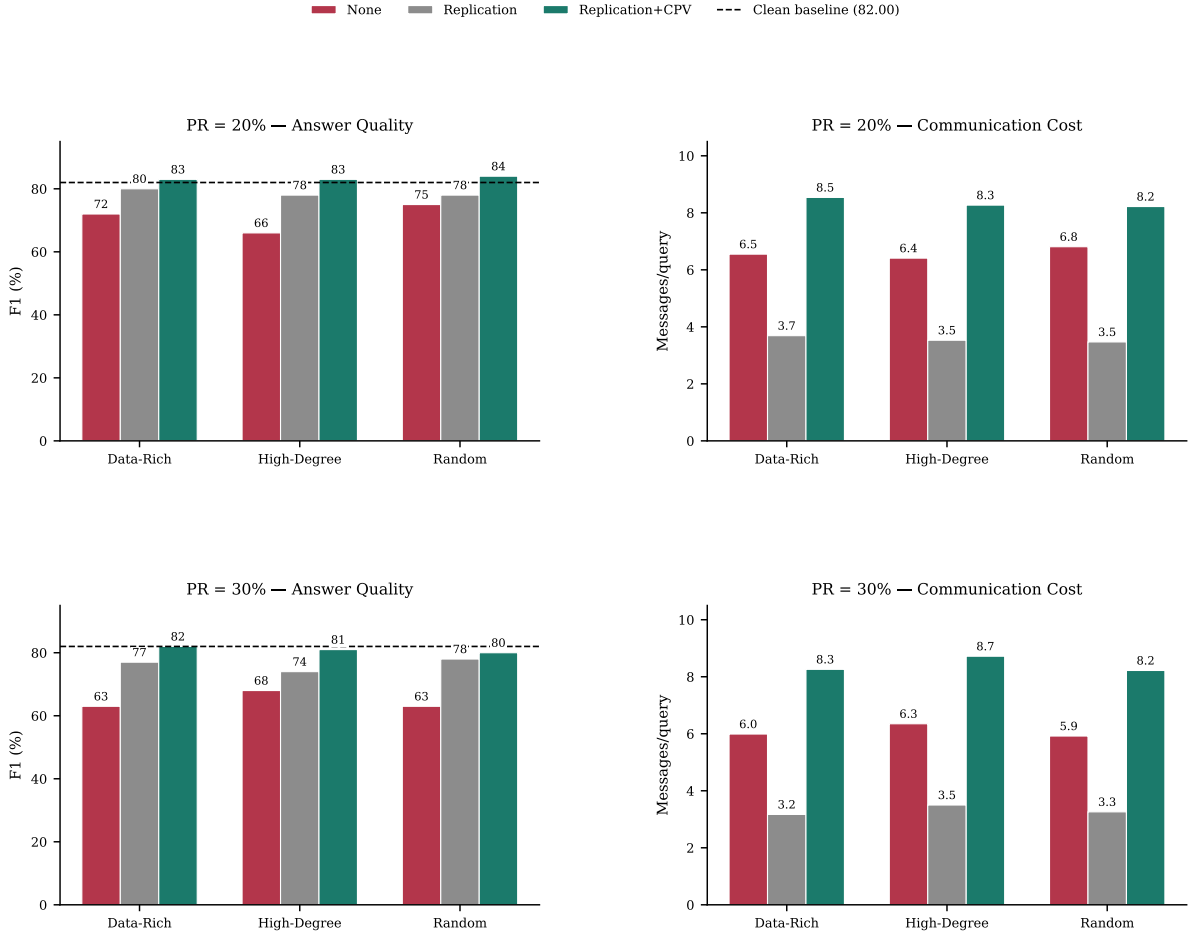


Figure 5.10: Data poisoning defence at both poison ratios: F1 recovery against the clean baseline, and the message cost each mechanism carries (DRAG, MMLU).

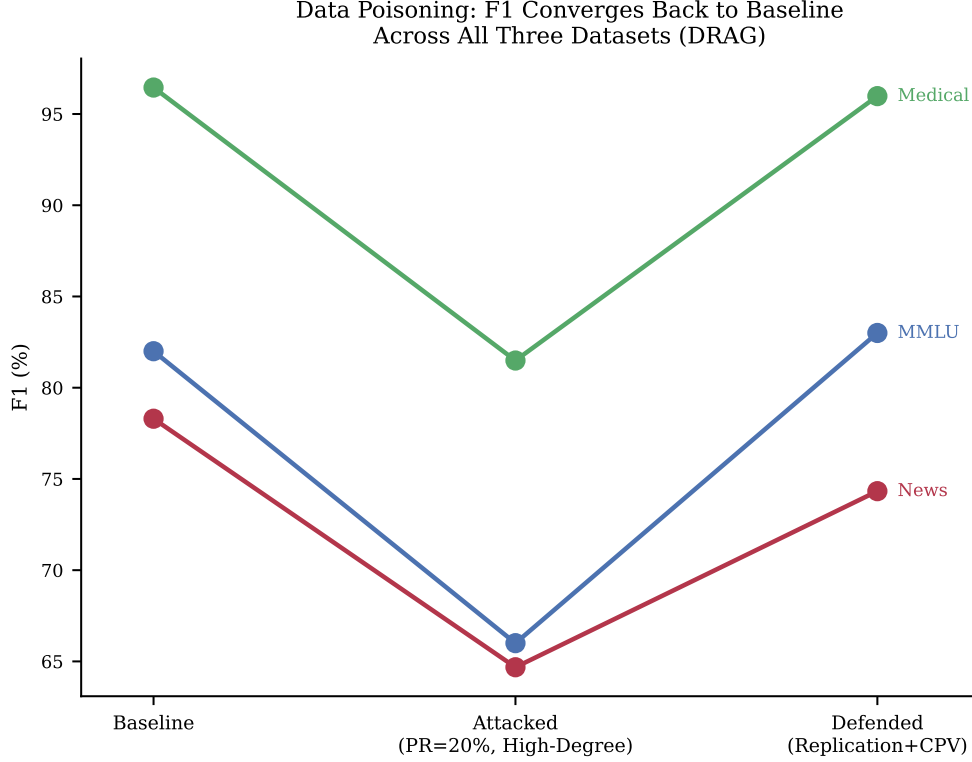


Figure 5.11: Data poisoning under High-Degree targeting across all three datasets: clean baseline, undefended, and defended (DRAG).

Table 5.24 orders the defence cleanly against the undefended baseline in all six strategy and ratio combinations on MMLU, with no reversals, and Table 5.25 holds the same ordering in all nine dataset and ratio cells. Figure 5.10 plots the quality and overhead trade at both ratios, and Figure 5.11 shows the ordering holding on News and Medical.

The two mechanisms pull communication cost in opposite directions. Replication drops messages per query from roughly 6 to 3.2–3.7, since a replicated corpus resolves more queries within the first hop. Cross-peer validation raises it to 8.2–8.7, matching the extra validation round-trips its quorum requires. A deployment adopting both pays around 30% more communication than an undefended network for near-complete recovery of answer quality.

One result deserves flagging. On Medical, defended F1 at 10% and 20% poisoning (96.96% and 95.98%) matches or exceeds the clean, unattacked baseline of 96.45%. Replication and quorum validation therefore deliver a retrieval-quality benefit that exists independently of any attack, which Section 5.6 returns to.

5.3.2 Topic-Claim Quota Enforcement

Routing manipulation succeeds because TARW trusts a peer’s advertised topic expertise, so a peer that claims topics it does not hold receives queries ahead of the peers that do. The defence discounts peers whose advertised claims exceed three times the network median, using a median absolute deviation test over the claim distribution, so a peer

buys routing priority only up to the point where its advertisement stops resembling honest variance.

Table 5.26: Routing Manipulation Defense Results Across Datasets (DRAG, 6 attacker peers, topic claim ratio 0.6)

Dataset	EM	F1	Hit Rate	SS	Avg Hops	Avg Msgs
Medical	0.85	0.9520	0.99	0.9655	1.84	15.52
MMLU	0.80	0.8000	0.93	0.8788	2.03	14.70
News	0.71	0.7597	0.93	0.7682	2.11	14.89

Note: Topics hijacked is fixed by the attack configuration and does not change under defence, since the mechanism discounts a peer’s claims at routing time rather than preventing the peer from making them. Compare against Table 5.6.

Table 5.27: Routing Manipulation Defense: Topic Claim Ratio Sweep (DRAG, MMLU)

Claim Ratio	EM / F1	SS	Hit Rate	Avg Hops	Avg Msgs
0.2	0.77	0.857	0.92	1.90	13.74
0.4	0.79	0.877	0.94	2.12	15.00
0.6	0.80	0.879	0.93	2.03	14.70
0.9	0.81	0.893	0.95	2.04	15.01

Note: Compare against Table 5.7, whose undefended F1 falls 0.77, 0.73, 0.68, 0.59 over the same range.

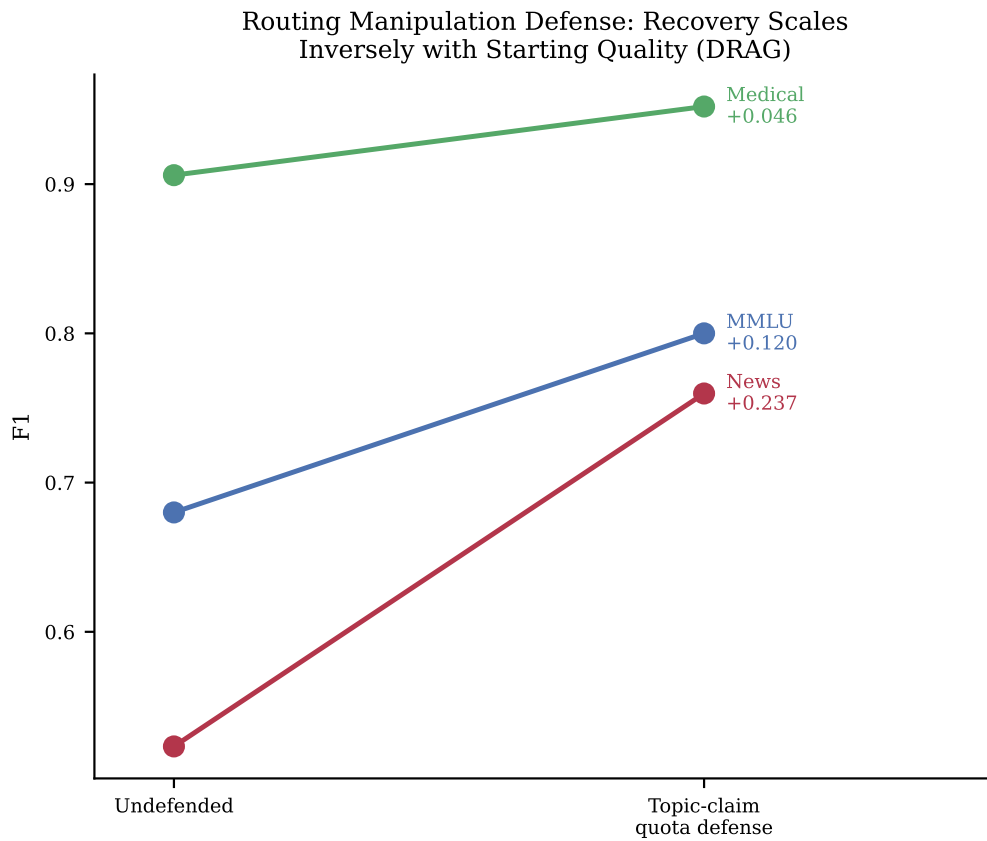


Figure 5.12: Answer quality by dataset, undefended against topic-claim quota enforcement (DRAG). Recovery scales inversely with how much the attack took.

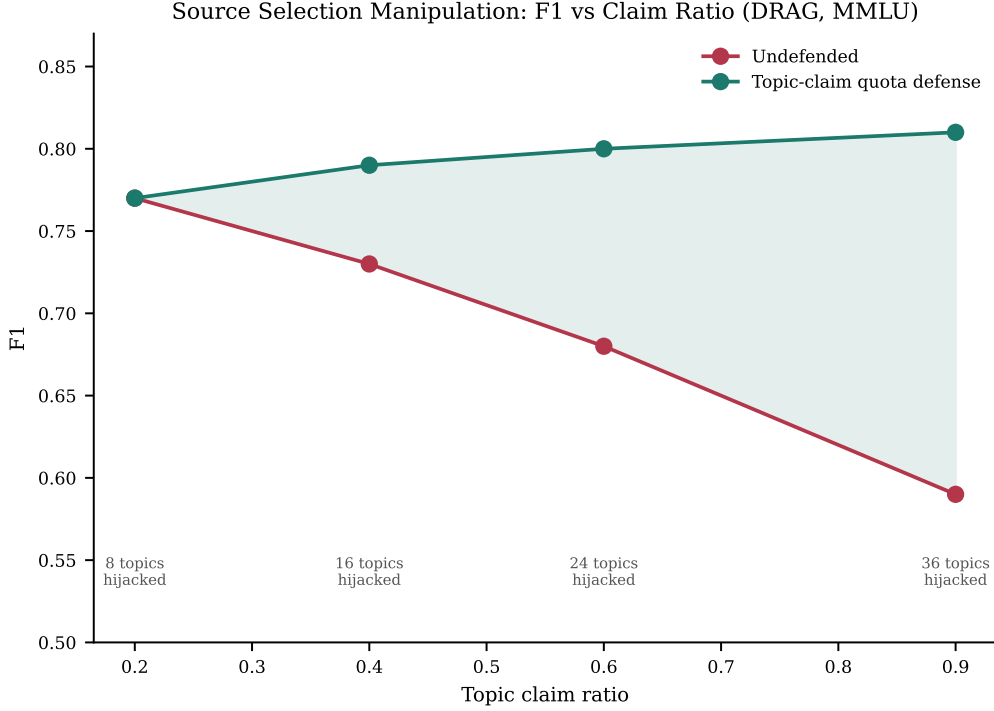


Figure 5.13: Answer quality across the topic claim ratio, undefended against defended (DRAG, MMLU). The shaded region is the recovery, which widens as the attack grows more aggressive.

Table 5.26 and Figure 5.12 show the defence recovering answer quality on all three datasets, with the size of the recovery scaling inversely to what the attack took. News suffers most undefended at 0.5232 F1 and recovers most at +0.2365; Medical suffers least at 0.9060 and recovers +0.0460. Hit rate rises to 0.93 or above on every dataset, against an undefended range of 0.62 to 0.94.

Table 5.27 shows the defence strengthening as the attack grows more aggressive. Defended F1 rises across the claim-ratio sweep (0.77, 0.79, 0.80, 0.81) and hit rate holds between 0.92 and 0.95, while the undefended condition falls from 0.77 to 0.59 over the same range. The quota threshold explains the direction: a peer claiming 20% of topics stays close enough to honest variance to escape discounting some of the time, while a peer claiming 90% clears three times the median by a wide margin and gets discounted on sight. An attacker maximising damage therefore maximises its own detectability, which leaves no claim ratio at which the attack both evades the quota and degrades answers.

The defence costs messages rather than hops. Average hops fall under defence on every dataset (Medical 2.10 to 1.84, MMLU 2.48 to 2.03, News 3.30 to 2.11) while average messages roughly double (9.03 to 15.52, 7.13 to 14.70, 7.19 to 14.89). Discounting fabricated claims lets TARW reach legitimate holders in fewer hops, and the validation round-trips that support the discounting are what raise the message count. Message overhead stays flat at 13.7 to 15.0 across the whole claim-ratio sweep, so the cost is a fixed per-query charge rather than one that scales with attack severity. A deployment adopting this defence should budget bandwidth, not latency.

5.3.3 Layered Extraction Prevention

Table 5.28: Knowledge Extraction Defense Results by Network Topology and Dataset (DRAG, Layered Defense)

Dataset	Peers	Conn.	ER (%)	CRR (%)	SS	EED	TC (%)
MMLU	10	Low	0.00	7.00	0.9984	0.0031	12.20
		High	0.00	9.00	0.9976	0.0033	21.95
	20	Low	0.00	8.00	0.9970	0.0034	19.51
		High	0.00	9.00	0.9978	0.0032	21.95
News	10	Low	0.00	0.09	0.8957	0.0442	16.67
		High	0.01	0.09	0.9201	0.0361	19.05
	20	Low	0.02	0.09	0.8989	0.0329	7.14
		High	0.01	0.09	0.8738	0.0458	19.05
Medical	10	Low	0.00	0.10	0.8568	0.0912	8.00
		High	0.00	0.10	0.8715	0.0911	7.00
	20	Low	0.00	0.10	0.8729	0.0944	9.00
		High	0.00	0.09	0.8399	0.0941	6.00

Note: Layered defence = query rate limiter (20 queries per 300s window), response perturbation (15% character-level noise) and extraction anomaly detector. Query Efficiency is omitted because it reads 18.00% in every row: the anomaly detector flags the attacker after 10 queries and the rate limiter blocks the remainder of the window, so the same fixed cap binds in every topology and on every dataset. SS and EED rise under defence and should not be read as preserved answer quality; see the discussion below. Compare against Table 5.9.

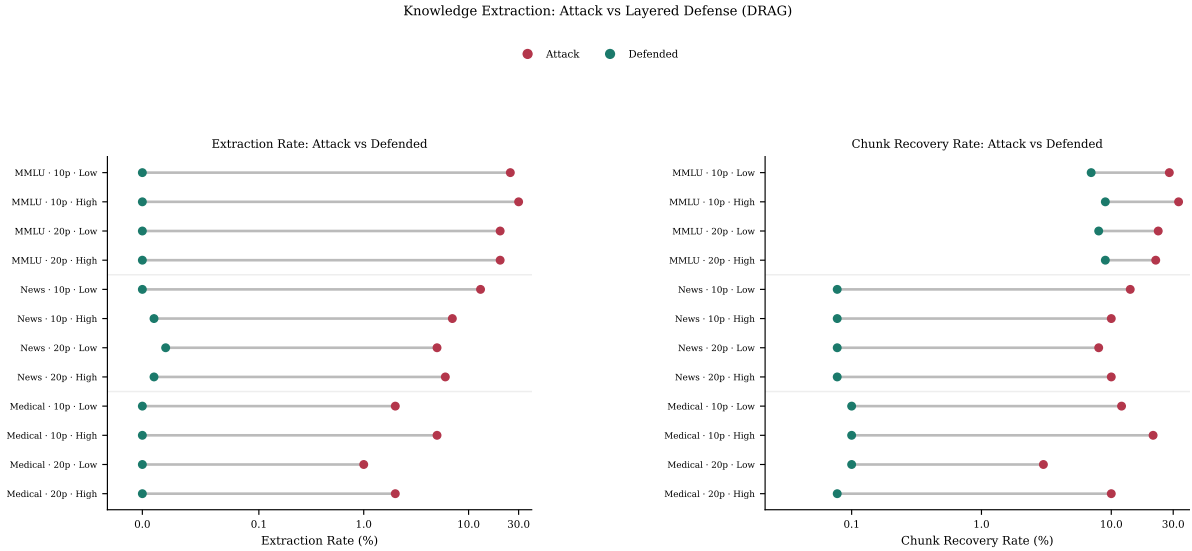


Figure 5.14: Extraction rate and chunk recovery, undefended against the layered defence, for every topology and dataset tested (DRAG). Horizontal axes are symmetric-logarithmic.

Table 5.28 records the layered defence suppressing extraction to at most 0.02% in all twelve conditions, across both network sizes, both connectivity settings and all three

datasets. Two mechanisms produce that result, and separating them matters for reading the table. Response perturbation guarantees the collapse of exact extraction, since the metric requires an exact question and answer match and any character substitution defeats it. Volume suppression accounts for the fall in chunk recovery: the anomaly detector flags the attacker after 10 queries and the rate limiter blocks the remainder of the window, so 18% of planned queries land in every condition. Topic coverage falls with it, from 61–82% undefended to 6–22% defended.

Chunk recovery is suppressed harder on free text than on multiple choice. News and Medical fall from 3–21% to roughly 0.1%, while MMLU falls from 22–33% to 7–9%. MMLU’s single-character answers clear the partial-credit similarity threshold almost automatically once perturbed, so partial credit survives there and vanishes elsewhere.

Semantic similarity and edit distance move against the defence throughout, rising to 0.84–0.998 and falling to 0.003–0.094. Reading that as preserved answer quality would be wrong. Perturbing a one-character MMLU answer yields another single character, and a debug sample records ground truth 3 perturbed to 1 scoring semantic similarity 1.0000 with edit distance 0.0006, because single-character strings cluster together in the sentence embedding space whatever they contain. The same rise appears on News and Medical, which rules out the single-character explanation as the whole account and points instead at the sentence embedding model tolerating 15% character noise across answer lengths. Extraction rate and query efficiency are the two metrics that track this defence on every dataset and topology tested.

5.3.4 Privacy-Preserving Retrieval

Table 5.29: Membership Inference Defense Results: Privacy-Preserving Retrieval Noise Sweep (DRAG, MMLU, Llama 3.2 3B)

Noise Scale b	Attack Acc.	AUC-ROC	Privacy Risk	Task F1 (%)	Task SS
0.0 (no defense)	0.78	0.8795	High	82.00	0.909
0.05	0.73	0.8540	Medium	85.00	0.929
0.1	0.64	0.7571	Medium	81.00	0.893
0.2	0.52	0.5788	Low	51.00	0.795
0.5	0.54	0.5152	Low	29.00	0.694

Note: Privacy-Preserving Retrieval adds Laplace noise at scale b to each peer’s retrieval similarity scores. Attack Acc. and AUC-ROC measure the adversary; Task F1 and Task SS measure normal question-answering utility on the same noisy network within the same run. The five tiers are the values documented in `config/defense.yaml`. The $b = 0.0$ row reuses the Llama leg of Table 5.12.

Table 5.30: Membership Inference Defense Results by Dataset (DRAG, Privacy-Preserving Retrieval, $b = 0.2$)

Dataset	Acc.	TPR	FPR	Precision	AUC-ROC	Privacy Risk
Medical	0.44	0.4571	0.6000	0.640	0.4771	Negligible
MMLU	0.63	0.6143	0.3333	0.8113	0.6643	Medium
News	0.60	0.5714	0.3333	0.800	0.6962	Medium

Note: Compare against Table 5.13.

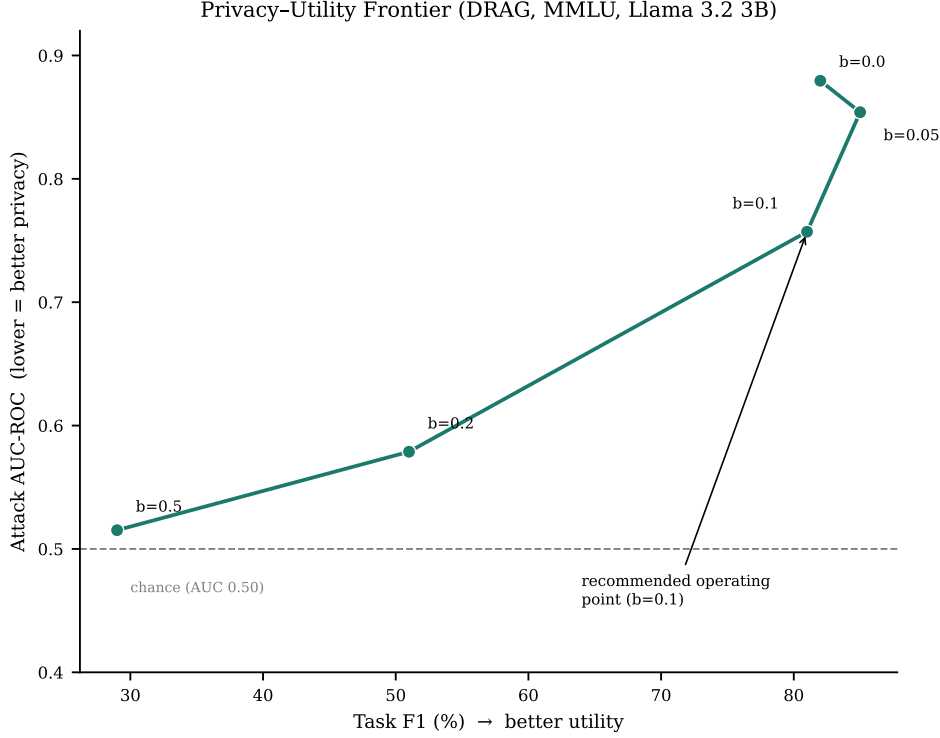


Figure 5.15: Privacy-utility frontier across five noise scales (DRAG, MMLU, Llama 3.2 3B). Each point is one measured setting; better lies down and to the right.

Table 5.29 and Figure 5.15 trace the privacy and utility axes together across five noise settings and give the privacy-utility frontier promised in Section 1.4. Both axes move monotonically. Privacy risk descends from HIGH to LOW as AUC-ROC falls from 0.8795 to 0.5152, and task F1 falls from 82.00% to 29.00% over the same range.

The frontier is not linear, which matters for choosing an operating point. Between $b = 0.05$ and $b = 0.1$ the defence buys a 0.097 reduction in AUC-ROC for 4 points of F1. The next step to $b = 0.2$ costs 30 points of F1 for a 0.178 reduction, and the step beyond that buys almost no further privacy (0.5788 to 0.5152) while removing another 22 points of utility. A deployment tuning this defense should sit at $b = 0.1$, holding the attack to MEDIUM risk while retaining 81% F1. Driving the attack to chance costs more than half the system’s answer quality.

Across datasets at $b = 0.2$, Medical falls furthest in relative terms, from AUC-ROC 0.6605 to 0.4771, crossing below the chance line and dropping from LOW to NEGLIGIBLE. MMLU and News start stronger and retain a MEDIUM residual after defense. Residual risk scales with how strong the underlying attack was.

5.3.5 Reputation-Based Forwarding Defense

Table 5.31: Selective Forwarding Defense Results by Strategy and Ratio (DRAG, Reputation-Based Defense)

Strategy	Ratio	Dropped Queries	Reduction	Blacklisted	Bypasses
Random	0.1	25	24.2%	1	8
Random	0.2	55	29.5%	3	23
Random	0.3	88	35.3%	6	108
Random	0.4	112	42.6%	8	155
Random	0.5	143	59.0%	9	206
High-Connectivity	0.1	30	60.5%	2	46
High-Connectivity	0.2	60	68.4%	4	130
High-Connectivity	0.3	90	68.8%	6	198
High-Connectivity	0.4	120	72.2%	8	312
High-Connectivity	0.5	150	74.6%	10	440

Note: Reduction is computed against Table 5.16. Hit rate is identical between attack-only and defended conditions at every row.

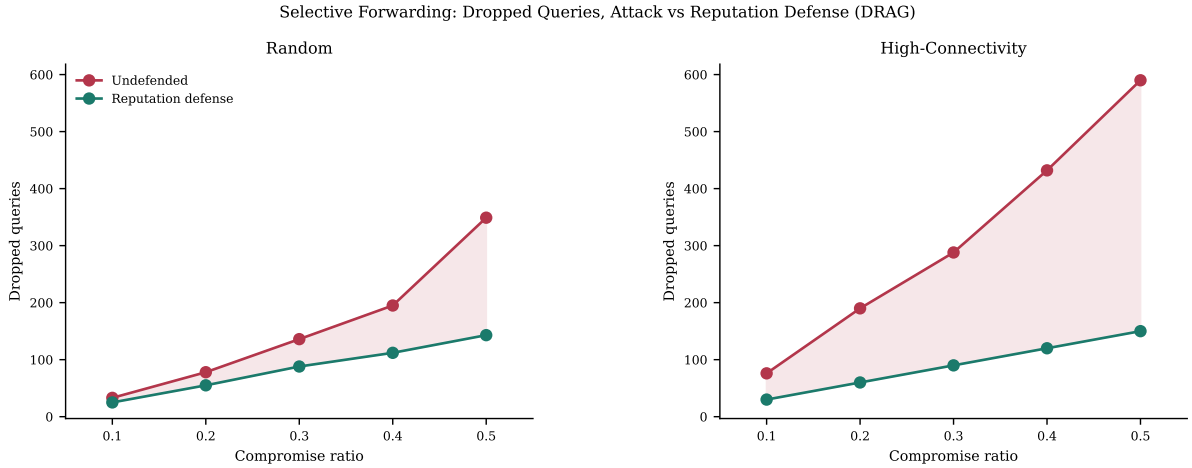


Figure 5.16: Selective forwarding on DRAG: dropped queries and hit rate, undefended against the reputation defence, by targeting strategy and compromise ratio.

Table 5.31 and Figure 5.16 show the reputation defence’s benefit growing with attack severity. Under Random targeting, dropped queries fall from 33 to 25 at ratio 0.1, a 24.2% reduction, rising to 59.0% by ratio 0.5. Under High-Connectivity targeting the same pattern holds at larger scale, from 60.5% at ratio 0.1 to 74.6% at ratio 0.5. Peers blacklisted rise from 2 to 10 and bypasses from 46 to 440 over that range, so a more aggressive attacker supplies the reputation system with more evidence to act on.

Hit rate is identical between defended and undefended conditions at every ratio. Topic replication already provides enough redundant paths for the undefended router to recover within its hop budget at this network size, so the defense contributes routing efficiency rather than availability.

5.3.6 Layered Congestion Defense

Table 5.32: DoS Defense Results, Mid Intensity (DRAG, All Datasets, Layered Defense)

Dataset	Strategy	QFR (%)	Avail. (%)	EM	F1	SS	Avg Hops
Medical	Random	19.17	75.0	0.6583	0.7424	0.7729	2.73
	Targeted	15.00	70.0	0.6667	0.7828	0.8177	2.24
	Sequential	12.50	65.0	0.6750	0.7813	0.8166	2.55
MMLU	Random	19.17	75.0	0.7000	0.7000	0.7349	2.18
	Targeted	15.00	70.0	0.7417	0.7417	0.7830	1.86
	Sequential	12.50	65.0	0.7750	0.7750	0.8165	1.93
News	Random	19.17	75.0	0.7083	0.7083	0.7051	2.59
	Targeted	15.00	70.0	0.7333	0.7333	0.7282	2.42
	Sequential	12.50	65.0	0.7583	0.7583	0.7539	2.49

Note: Layered defence = adaptive routing weight, circuit breaker, query hedging and load-shedding mitigation. Compare against Table 5.19.

Table 5.33: DoS Defense Results, High Intensity (DRAG, All Datasets, Layered Defense)

Dataset	Strategy	QFR (%)	Avail. (%)	EM	F1	SS	Avg Hops
Medical	Random	21.67	30.0	0.6000	0.7013	0.7336	2.89
	Targeted	24.17	40.0	0.5833	0.6891	0.7218	3.02
	Sequential	40.83	20.0	0.4917	0.5544	0.5822	3.58
MMLU	Random	21.67	30.0	0.6750	0.6750	0.7147	2.27
	Targeted	24.17	40.0	0.6500	0.6500	0.6876	2.51
	Sequential	40.83	20.0	0.5167	0.5167	0.5363	3.25
News	Random	21.67	30.0	0.6583	0.6583	0.6525	2.68
	Targeted	24.17	40.0	0.6833	0.6750	0.6696	2.92
	Sequential	40.83	20.0	0.5167	0.5167	0.5037	3.48

Note: Compare against Table 5.20.

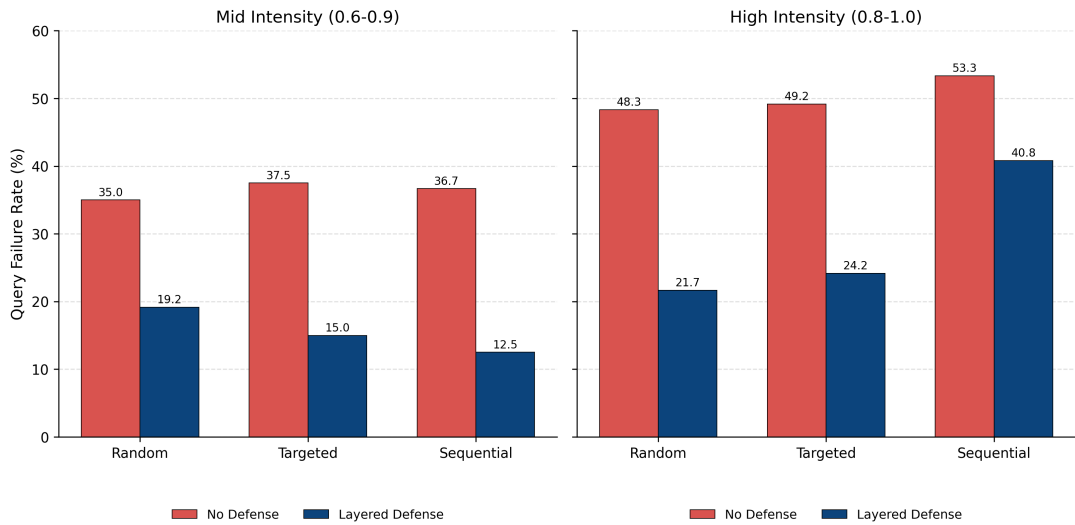


Figure 5.17: Query failure rate by strategy and intensity, with and without layered defense (DRAG)

At mid intensity, Table 5.32 cuts query failure from 35.00% to 19.17% under Random targeting and from 36.67% to 12.50% under Sequential, and lifts availability from 0–10% to 65–75%. In Table 5.33, plotted alongside the undefended condition in Figure 5.17, the recovery narrows and becomes strategy dependent. Random holds a 55.2% relative reduction (48.33% to 21.67%) while Sequential falls to 23.4% (53.33% to 40.83%) and recovers only 20.0% availability, the worst defended outcome across all six strategy and intensity combinations. A round-robin sweep leaves the circuit breaker and hedging layers the least uncongested capacity to redirect toward, so the defense erodes precisely against the strategy that spreads damage most evenly.

5.3.7 Reliable-dRAG Defenses

We scoped defense work to DRAG, which serves as the framework’s demonstration of the feedback loop from evaluation to design change. Section 5.2’s Reliable-dRAG results are therefore reported undefended throughout, and Section 5.6 records the resulting asymmetry as a limitation.

5.4 Statistical Analysis

5.4.1 Sample Sizes and Repetition

Every DRAG run in this chapter evaluates 100 questions. We verified the count per run from the simulator’s own log line and from a record count over the saved test cases rather than from a raw line count, since several MMLU items contain embedded newlines inside quoted CSV fields. Membership inference splits its 100 items 70 members to 30 non-members. Knowledge extraction issues 5 queries per topic over 10 topics, which caps the reachable extraction rate at 50% and should be read as the ceiling for those tables rather than 100%.

Reliable-dRAG results report the mean and standard deviation over seeds 0, 42 and 123 wherever the attack permits repetition, except for membership inference, whose held-out evaluation uses five seeds generated for that purpose. Knowledge extraction uses 54 probes per seed per source. Data poisoning evaluates 7 questions, so each question carries 14.3 accuracy points and the standard deviations in Table 5.5 are correspondingly wide. The membership inference composite was evaluated against five seeds generated after its weights were locked, reported in Section 5.4.3. Table 5.15 records the weightings we tested and rejected, and its seeds sit inside the search set rather than outside it.

5.4.2 Sweep Design and Monotonicity

We treat monotonicity across a parameter sweep as the acceptance criterion for DRAG results, in place of the confidence intervals a seed-varied design would produce. A degradation curve that rises with attack strength, and a defense ordering that holds at every point on that curve, constitute stronger evidence than a single point with error bars, because they test the claim at several operating conditions rather than one.

Three sweeps satisfy the criterion without exception. Cross-peer validation orders above replication, which orders above no defence, in all 6 MMLU cells and all 9 cross-dataset

cells. Routing manipulation degrades monotonically across 4 claim ratios, with topics hijacked tracking the ratio linearly. Privacy-preserving retrieval moves both privacy and utility monotonically across 5 noise settings.

Two sweeps contain a reversal, and we report both rather than smoothing them. High-Degree poisoning rises from 66.00% to 68.00% F1 between the 20% and 30% ratios, which Section 5.2.1 attributes to overlap between the Data-Rich and High-Degree peer sets at 30% in a 20-peer graph. News’s undefended poisoning sequence puts 10% ahead of 20% in damage, which we attribute to topic sampling rather than to the poison ratio.

5.4.3 Held-Out Validation

Reliable-dRAG’s membership inference composite is the one result in this chapter tuned against data rather than specified in advance, so it carries a corresponding validation requirement. We searched candidate signal weightings over the 13 seeds the project had already observed, locked the weights, then evaluated once against five seeds generated for that evaluation and never queried before it. Table 5.15 reports mean AUC-ROC 0.620 at a 95% confidence interval of [0.546, 0.695], which excludes chance. The held-out mean sits above the 0.572 the same weighting scores on the 13 search seeds, so the search did not overfit to the data it ran on. Every one of the five held-out seeds scored above 0.50 and none inverted. Similarity alone scores 0.4732, below chance, which is why the production composite zeroes it, and restoring certainty at 0.20 lowers three further fresh seeds from 0.6133 to 0.6072.

5.4.4 Degenerate Thresholds

Three tables contain rows where a classifier threshold labelled every query a member, producing TPR and FPR both equal to 1.0: DRAG’s News row in Table 5.13, and Reliable-dRAG’s seeds 42 and 123 in Table 5.14. Table 5.15 records the weightings we tested and rejected alongside the production composite.

5.5 Comparisons and Relationships

5.5.1 Attack Classes Across Two Architectures

Table 5.34 places each attack class beside the mechanism it exploits in each system. The framework specifies each class independently of mechanism; applying it to a new architecture means identifying which component carries the corresponding trust assumption and instantiating against that.

Table 5.34: Attack Class Instantiation and Severity Across Both Systems

Attack Class	DRAG Mechanism	Reliable-dRAG Mechanism	DRAG Severity	Reliable-dRAG Severity
Data Poisoning	In-memory peer knowledge base	/poison write path into live FAISS index	Medium (30.7% F1 loss, News)	High (66.7% degradation)
Source Selection Manipulation	TARW topic advertisement	On-chain grounding-farming; orchestrator key forgery	Medium (28.0% F1 loss at claim ratio 0.9)	Low (9.0–12.0 pp accuracy drop); see note
Knowledge Extraction	Multi-hop TARW routing	Direct per-source /query HTTP	Medium (30% ER, 10 peers, MMLU)	Medium (41.9% ER, gray-box)
Membership Inference	Similarity plus routing composite	Decision-match on single response	High (PRS 0.786)	Low–Medium (AUC 0.620, CI [0.546, 0.695])
Selective Forwarding	TARW with topic replication	TTL-bounded breadth-first search	Negligible (hit rate 1.00 to 0.95)	Medium (hit rate 0.98 to 0.54, simulated)
Denial of Service	Wave congestion over 20 peers	Wave congestion; live HTTP flood on 3 sources	High (53.3% F1 loss, 53.3% query failure)	Critical (100% F1 loss, 90% query failure)

Note: We assign severity with the rubric in Table 5.2, taking each attack at its most damaging tested configuration. Integrity and availability rows report degradation relative to that configuration’s own clean baseline rather than in absolute metric points. DRAG severities are undefended. Two rows carry qualifications. The rubric scores user-visible answer quality, so it does not capture the on-chain reliability corruption that Source Selection Manipulation inflicts on Reliable-dRAG, where Grounding-Farming drives the target source’s reliability score by +680 and Key-Forgery by +20,000 while answer accuracy falls by 9.0 to 12.0 points; Section 5.6 records this as a coverage gap in the rubric. Both selective forwarding rows rest on a metric choice we should state. Reliable-dRAG’s Medium comes from the simulated network model, and the live deployment, measured at both severities its three-source topology can express, shows no degradation at either; we attribute that disagreement to the ratio between hop budget and network size rather than to either measurement being wrong, and Section 5.6 records what it leaves open. We assign DRAG’s Negligible by hit rate, which falls from 1.00 to 0.95, while the same conditions drop as many as 590 queries; the rubric follows hit rate because it measures what reaches a user.

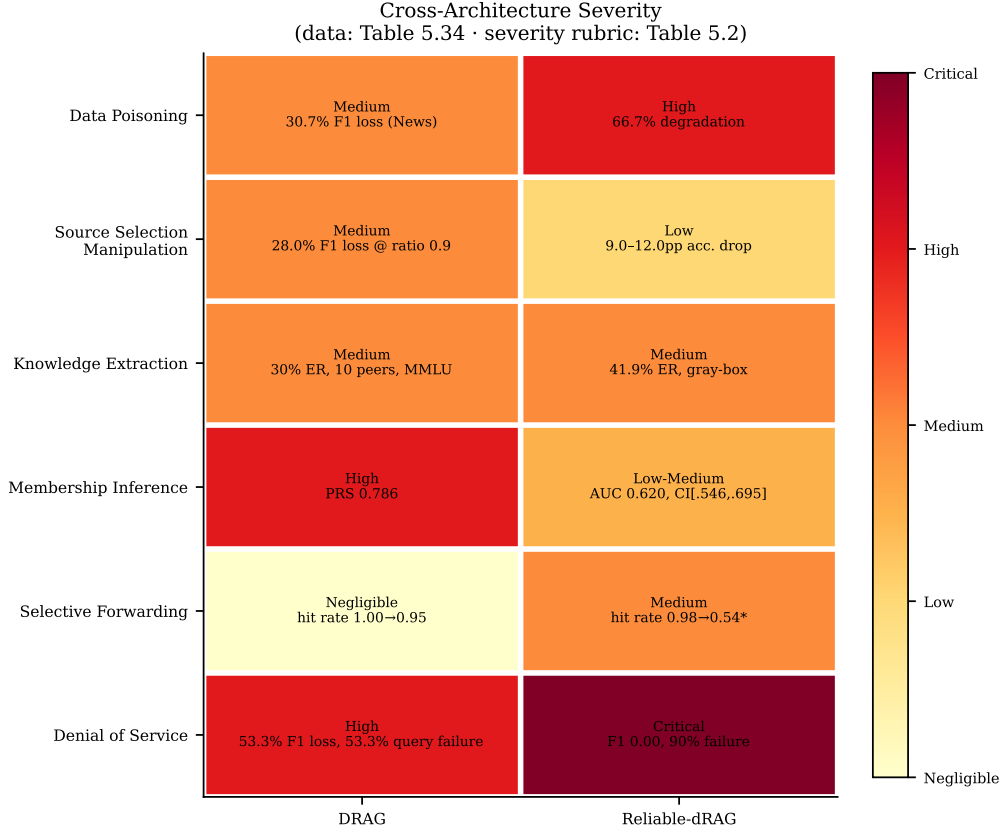


Figure 5.18: Severity by attack class and system, assigned with the rubric of Table 5.2. Cell annotations give the measurement behind each tier.

Three relationships come out of the comparison.

Source Selection Manipulation instantiates differently in each system and exists in neither centralised RAG nor in either system’s published threat model. DRAG’s routing algorithm trusts a peer’s advertised topic expertise, so the attack falsifies advertisements. Reliable-dRAG’s reranker trusts an on-chain reliability score derived from a grounding check, so the attack farms that check with filler content that satisfies a literal substring test. We derived the second instantiation by applying the framework’s mapping step to Reliable-dRAG’s reliability mechanism, not by porting DRAG’s attack, and the deployed on-chain defence turned out to be structurally unable to observe it: the defence validates caller identity, transaction rate, delta size and bounds, and Grounding-Farming submits no transaction that violates any of those.

Reliable-dRAG suffers more than DRAG on both availability classes, and redundancy explains the gap in each. Selective forwarding costs DRAG almost nothing (hit rate 0.95–1.00) because topic replication leaves redundant paths inside the hop budget, while it drives Reliable-dRAG’s hit rate to 0.54 in simulation, though the live deployment shows no degradation at either severity it can express (Section 5.2.5), because a TTL-bounded breadth-first search spends a hop on every visit to a silent peer. Denial of service reaches a worse outcome on Reliable-dRAG (F1 0.00) than on DRAG (F1 0.41) because three data sources constitute the whole retrieval backend, while 20 peers retain partial routing

paths. Redundancy, not defence quality, decides both outcomes.

Membership inference is the one class where DRAG leaks more. Its composite draws 60% of its weight from retrieval similarity and 30% from routing behaviour, and a multi-hop network exposes routing behaviour that a directly-queried orchestrator does not. The signal the attack needs is a byproduct of the routing layer, so the architecture that routes more leaks more.

5.5.2 Comparison with Existing Work

Published RAG security evaluations target centralised, single-knowledge-base deployments. SafeRAG benchmarks attack surfaces against one retrieval corpus; RAGLeak and DCMI both study membership inference against a single vector store; Zeng et al. characterise extraction and leakage in a monolithic RAG pipeline. None models source selection, multi-party trust, or knowledge partitioned across independently operated participants, because those components do not exist in the architectures they evaluate.

Mu & Li [23] are the exception. They attack the routing stage of federated RAG across three router families and defend it with TASR, so source-selection manipulation in distributed retrieval is measured ground. Our DRAG results extend that ground to an unmediated gossip topology, where no server exists to verify a profile and a false topic claim propagates across the overlay. Grounding-Farming sits outside it. TASR scores a source on the relevance of the evidence it returns and that evidence’s consistency with the profile it registered, and its authors record one adversary the design does not cover, a source returning content that reads as relevant while supporting no correct answer. That is the adversary our Reliable-dRAG results measure against a deployed mechanism: full escalation in three of six trials, with +680 reliability and +1,574 usefulness accrued without a single on-chain transaction the system’s own defense could flag.

Three of our six attack classes have no centralised analogue. Source Selection Manipulation requires more than one source to choose between. Selective Forwarding requires a participant that can be reached but decline to answer honestly. Membership inference at the partition level, revealing which source holds a record rather than whether the system holds it, requires knowledge to be partitioned in the first place. The remaining three classes exist centrally but change character under distribution: poisoning scales its attack surface with source count and rewards strategic source selection, extraction gains a topology dependence we measured directly in Table 5.9, and denial of service acquires a redundancy dimension absent from a single-server deployment.

Our extraction results also qualify a figure that recurs in this literature. Reported extraction rates in the 20–60% range come predominantly from multiple-choice corpora, and Table 5.9 shows the same attack recovering 1–6% on free-text corpora at identical topology and query budget. Extraction severity is a function of answer structure as much as of retrieval design.

5.6 Discussions

5.6.1 Interpretation

Applying the framework twice produced a result we did not anticipate: defence effectiveness rises with attack severity in three of the five DRAG defences. Cross-peer validation holds its ordering at every poison ratio and recovers most where the attack takes most. Cross-peer validation against routing manipulation improves as the claim ratio rises, because the outlier detector’s quota of three times the median grows easier to clear as an attacker claims more topics. The reputation defence against selective forwarding cuts 24.2% of drops at ratio 0.1 and 74.6% at ratio 0.5, because a more aggressive attacker supplies more evidence. Measuring any of these at one attack strength would have hidden the property, which argues for parameter sweeps over single-point evaluation when assessing a defence rather than an attack.

The layered congestion defence runs the other way, eroding from a 65.9% relative reduction at mid intensity to 23.4% at high intensity under Sequential targeting. The distinction is whether a defence needs surviving capacity to work. Reputation and quorum mechanisms need evidence, which an aggressive attacker supplies. Rerouting mechanisms need somewhere to reroute to, which an aggressive attacker removes.

5.6.2 Implications for System Design

Four design conclusions follow from the measurements.

Network size resists extraction better than connectivity tuning does. Doubling peers from 10 to 20 cut extraction from 25–30% to 20%, while connectivity mattered only in the smaller network.

Replication carries a retrieval benefit independent of security. Medical’s defended F1 at 10% and 20% poisoning matched or exceeded its clean baseline, so a deployment adopting replication for integrity reasons should expect answer quality to improve rather than to degrade.

Privacy-preserving retrieval has a usable operating point and an unusable one. At $b = 0.1$ the defence holds membership inference to MEDIUM while retaining 81% task F1; beyond $b = 0.2$ it buys almost no additional privacy for large utility losses.

A defence aimed at the wrong layer provides no protection regardless of its own quality. Reliable-dRAG’s on-chain caps stop forged and oversized transactions, and Grounding-Farming submits neither, so a well-implemented defence sits beside an unmitigated attack. Our framework surfaced that gap by asking which signal source selection trusts rather than which transactions the defence inspects.

5.6.3 Limitations

We evaluated two systems. Both are distributed RAG deployments with independently operated sources, and they differ enough architecturally, gossip peer-to-peer routing

against a blockchain-reranked orchestrator, to test whether the attack classes transfer. Two systems demonstrate portability rather than establishing generality, and we present the framework as a proof of concept on that basis.

DRAG results come from a single random seed with parameters swept, so we report monotonicity rather than confidence intervals and cannot separate small non-monotonic movements from sampling variation. The High-Degree reversal at 30% poisoning and News’s 10% against 20% reversal both fall into that gap.

Reliable-dRAG’s live selective forwarding evaluation covers both severities its topology admits, at three seeds each, and reports no degradation at either. That result establishes less than its consistency suggests. A three-source deployment whose hop budget reaches every source cannot express a graduated attack: the compromise count takes only the values one, two and three, and the third is a total blackout. The mock sweep supplies the dose-response curve for this attack class, and the two models disagree by a wide margin at matched severity, which we attribute to the ratio between hop budget and network size rather than to either model being wrong. We have not tested that attribution directly, and doing so would require either a larger live deployment or a mock configuration matched to the live topology. The live DoS evaluation runs at a single seed, though its effect size and monotonic dose-response across three severity levels leave less room for a sampling explanation.

We implemented defenses on DRAG only, so Section 5.3 demonstrates the feedback loop on one architecture rather than two. Table 5.34 reports both systems undefended for that reason, so it measures exposure rather than residual risk after mitigation, and we make no claim about how Reliable-dRAG would fare with equivalent defences in place.

Grounding-Farming escalated in three of six independent trials, and one seed produced opposite outcomes on two runs at identical configuration. We report it as an exposure with a measured escalation rate near one in two rather than as a reliable attack, and six trials cannot separate that rate from the self-reinforcing dynamic Section 4.2.2 describes. Wider replication is the first item in Section 6.3.

Our severity rubric scores user-visible answer quality, which under-reports Grounding-Farming. The attack drives the farming source’s on-chain reliability by +680 and usefulness by +1,574 while costing end users 9.0 percentage points of accuracy, so the rubric assigns a Low tier to an attack that corrupts the mechanism outright. We report the on-chain deltas beside the tier and identify a mechanism-corruption dimension as the rubric’s clearest missing axis, rather than rescoring six attack classes against a dimension introduced after the measurements were taken.

Two threat models assume more than an external attacker holds. Data poisoning assumes write access to at least one source’s knowledge store, an insider or compromised-participant position we treat as a worst-case upper bound; the informative variable is which source an attacker chooses rather than whether the attack succeeds. Knowledge extraction’s gray-box condition pre-filters probes to guarantee a hit, measuring post-reconnaissance efficiency rather than discovery cost, which is why we report the blind condition beside it.

Query efficiency reads 100% in every undefended extraction row because the attack issues network queries with a zero confidence threshold, so any non-empty response counts as a hit. The metric is informative as a ratio against the defended rows and misleading as an absolute measure of attack precision.

Six attack classes do not exhaust the space. Sybil attacks and collusion between sources compose over the single-adversary primitives we measured rather than introducing new trust assumptions, and cross-source correlation, inferring from several sources jointly what none reveals alone, is the clearest class our framework does not yet cover.

Chapter 6

Conclusion

6.1 Summary of Findings

This thesis developed a seven-step evaluation procedure for the security of distributed RAG systems and applied it to two architecturally distinct deployments, DRAG [9] and Reliable-dRAG [21], evaluating all six attack classes the procedure derives, two per property of the CIA triad, against both.

The undefended results show that distribution changes what a given attack class costs an adversary without making any of the six classes go away. Data poisoning cost DRAG up to 24 points of F1, on the News corpus at a 10% poison ratio, and 16 points on MMLU at 20%. On Reliable-dRAG it reached 100% attack success at 66.7% mean degradation once we made the poison text plausible instead of generic. Knowledge extraction recovered 20–30% of DRAG’s multiple-choice corpus but only 1–6% of its free-text corpora, and reached 37–42% of Reliable-dRAG’s document collection through nothing more than its own legitimate query interface. Membership inference reached AUC-ROC 0.90 on DRAG, HIGH risk by our own rubric, but only 0.62 on Reliable-dRAG, a difference traced directly to which architectural layer each system exposes to an attacker: DRAG’s multi-hop routing leaks a signal a directly-queried orchestrator does not. Selective forwarding cost DRAG almost nothing in simulation, hit rate held at 0.95 or above, because topic replication supplied redundant paths, while the same withholding behaviour drove Reliable-dRAG’s simulated hit rate to 0.54. Both figures come from network models rather than live deployments, and the single live run we performed against Reliable-dRAG recovered every dropped query, which Section 5.6 records as unresolved. Denial of service was the most severe class on both systems, driving availability to 0% on DRAG under sequential targeting and F1 to 0.00 on Reliable-dRAG once all three of its data sources were flooded. Source selection manipulation produced the thesis’s clearest architectural finding: the same class falsifies a peer’s advertised topic expertise on DRAG, but on Reliable-dRAG it farms an on-chain grounding check, an attack, Grounding-Farming, that required no privileged access, was not adapted from any attack in the reviewed literature, and evaded a defense the system’s own designers had already built for a different kind of manipulation.

Of the six defenses designed against these findings, all against DRAG, three held their effectiveness or strengthened it as attack severity rose: cross-peer validation restored data-poisoning F1 to 80–84% against a clean baseline of 82% in every tested cell, the topic-claim quota defense recovered further as the claim ratio grew because a more aggressive claim is also a more detectable one, and the reputation-based forwarding defense cut dropped queries by up to 74.6% for the same reason. Privacy-preserving retrieval traded membership-inference AUC-ROC for task F1 along a non-linear frontier, with $b = 0.1$ identified as the operating point that holds risk to MEDIUM while retaining 81%

of clean F1. The layered congestion defense ran the other way, its relative benefit eroding from 65.9% at mid intensity to 23.4% at high intensity under sequential targeting, since rerouting mechanisms need surviving capacity to redirect toward and an aggressive, evenly-spread attacker removes exactly that. Reliable-dRAG’s results are reported undefended throughout, an asymmetry the thesis states as a limitation rather than obscures.

6.2 Contributions to the Field

The contributions of this thesis are summarised below.

1. **A seven-step, architecture-agnostic evaluation procedure.** A structured method, in the sense STRIDE is a method rather than a tool, by which an evaluator characterises a target system’s trust assumptions, maps a derived attack set onto them, and converts the result into comparable, severity-ranked findings without prior familiarity with the system’s internals.
2. **A CIA-triad-derived, six-class attack taxonomy for distributed RAG.** Data Poisoning, Source Selection Manipulation, Knowledge Extraction, Membership Inference, Selective Forwarding, and Denial of Service, derived directly from the architectural components a distributed deployment introduces beyond a centralised one, rather than assembled from a survey of previously reported attacks.
3. **The first cross-architecture empirical security comparison in this space.** Prior attack evaluation against distributed retrieval varies the routing mechanism inside a single server-mediated federated architecture; this work is the first, to our knowledge, to evaluate the same attack classes under one metric set against two systems that differ in trust topology: a peer-to-peer gossip overlay with no central coordinator, and a blockchain-reranked orchestrator.
4. **Grounding-Farming: an unprivileged attack the deployed defense cannot observe.** An attack against Reliable-dRAG’s on-chain reliability mechanism, obtained from the evaluation procedure’s mapping step rather than adapted from the literature, that requires no privileged access and evades the system’s own deployed defense against reliability-score manipulation.
5. **Six defense mechanisms with independently measured cost and recovery.** Each mechanism was evaluated against the attack class it primarily answers, with no other defense active, so its marginal contribution and its cost in messages, hops, or task utility can be attributed to that mechanism alone rather than to a bundled security layer.
6. **A reproducibility protocol matched to what each system exposes.** Parameter sweeps and monotonicity checks on DRAG’s controllable topology; multi-seed repetition, including a held-out generalisation test for the tuned membership-inference composite, on Reliable-dRAG’s fixed deployment.
7. **A documented record of measurement corrections.** Verified corruption levels that diverged from Reliable-dRAG’s own source labels, a replaced targeting strategy after an unreachable blockchain signal was identified, a corpus-drift bug that had silently zeroed an extraction attack, and a configuration fault that had left a

replication defense unreplicated, each caught, corrected, and reported rather than left in the results.

8. **A shared five-tier severity rubric across confidentiality, integrity, and availability.** A common scale onto which an extraction rate, an AUC-ROC, and a query-failure rate can all be mapped, making results from different attack classes and different systems directly comparable rather than only individually interpretable.

6.3 Recommendations for Future Work

6.3.1 Cross-Source Correlation Attacks

The framework’s own derivation identifies cross-source correlation, an adversary inferring from the combined responses of several sources what none discloses individually, as the attack class within the distribution layer that neither system’s evaluation presently covers. Its derivation would follow the same seven steps used throughout this thesis: characterising how each system aggregates responses across sources is the natural starting point, since that aggregation step is exactly where an individually-innocuous set of responses could jointly leak more than any one of them.

6.3.2 Closing the Defense Gap on Reliable-dRAG

Every defense in this thesis was designed and evaluated against DRAG; Reliable-dRAG’s results remain undefended throughout. Layered extraction prevention and privacy-preserving retrieval are the two most directly portable candidates, since Reliable-dRAG already exposes a comparable rate-limitable query surface and a comparable retrieval-similarity signal for membership inference; a defense against Grounding-Farming specifically would need to target the grounding check itself rather than the transaction-level properties the system’s existing on-chain defense inspects.

6.3.3 Formal Privacy Accounting for Privacy-Preserving Retrieval

Privacy-preserving retrieval’s Laplace noise gives a per-query differential-privacy guarantee, but the thesis does not implement a privacy budget or accountant to bound how that guarantee degrades under repeated querying of the same peer. Adding a per-peer query budget, or a Rényi-DP accountant tracking cumulative exposure across a session, would convert the defense from an empirically validated noise injection into a provably bounded privacy mechanism, closing the gap between the $b = 0.1$ operating point this thesis recommends and a guarantee that holds under sustained adversarial querying rather than a single measured run.

6.3.4 Statistical Replication for DRAG

DRAG’s results come from a single random seed with parameters swept, so the thesis reports monotonicity across a sweep rather than a confidence interval at any one

point, and two reversals, the High-Degree poisoning crossover at 30% and News’s non-monotonic 10%-to-20% sequence, cannot presently be separated from sampling variation. Re-running the swept configurations across multiple seeds, in the same manner already applied to Reliable-dRAG, would let both reversals be characterised as either real or noise rather than left as an open question.

6.3.5 Live-Deployment Validation Beyond a Single Configuration

Reliable-dRAG’s live denial-of-service run is the strongest evidence in this thesis that a simulated condition translates to a real deployment, and it rests on a single seed and configuration. Its live selective-forwarding counterpart points the other way: the compromised source dropped every query routed to it, yet redundant probing recovered all of them, an outcome the simulated sweep does not predict. The two disagree, and the disagreement is unresolved. Extending the live evaluation across the ratio and intensity sweeps already run in simulation would settle whether the simulated selective-forwarding result overstates the damage or whether that one configuration is unrepresentative.

6.3.6 Extending the Framework to Federated Architectures

FedRAG was considered during early planning and explicitly descoped to protect the depth of evaluation on the two systems this thesis integrates. Since the seven-step procedure characterises a system by its trust assumptions rather than by its topology, applying it to a federated aggregation architecture, where the shared trust assumption is that a participant’s model update reflects honest local training rather than that a query response reflects honest local data, is a natural next application and would test whether the same six-class derivation, or a modified one, transfers to a third architectural family.

6.4 Closing Remarks

This thesis set out to answer a question the literature reviewed in Chapter 2 leaves open: whether a single evaluation procedure, derived once from first principles rather than assembled from prior attacks, could be applied to more than one distributed RAG architecture and still produce comparable, actionable findings. Applied twice, to a peer-to-peer overlay and to a blockchain-reranked orchestrator that share almost nothing else architecturally, it did: the same six attack classes manifest in both, at severities that diverge in ways the architectures themselves explain, and the procedure’s final step converted six of DRAG’s own findings directly into measured defenses. The clearest evidence that the derivation performs real work rather than restating known attacks is Grounding-Farming, an exposure in a system whose designers had already built a defense, just aimed at a different manipulation than the one their own reliability mechanism turned out to permit. Two systems demonstrate that a procedure transfers, not that it is universal, and the recommendations above are aimed squarely at that distinction: wider defense coverage, formal privacy accounting, statistical replication, and a third architectural family are what would move this framework from a validated proof of concept to a general-purpose evaluation standard for distributed RAG deployments.

Bibliography

- [1] A. Chakraborty, C. Dahal, and V. Gupta, *Federated Retrieval-Augmented Generation: A Systematic Mapping Study*, 2025. arXiv:2505.18906 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2505.18906>
- [2] P. Addison, M.-T. H. Nguyen, T. Medan, J. Shah, M. T. Manzari, B. McElrone, L. Lalwani, A. More, S. Sharma, H. R. Roth, I. Yang, C. Chen, D. Xu, Y. Cheng, A. Feng, and Z. Xu, *C-FedRAG: A Confidential Federated Retrieval-Augmented Generation System*, 2024. arXiv:2412.13163 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2412.13163>
- [3] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, and J. Tang, *The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)*, 2024. arXiv:2402.16893 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2402.16893>
- [4] V. Rathod, S. Nabavirazavi, S. Zad, and S. S. Iyengar, *Privacy and Security Challenges in Large Language Models*, in Proc. 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC), pp. 746–752, 2025. doi: 10.1109/CCWC62904.2025.10903912
- [5] A. Arzanipour, R. Behnia, R. Ebrahimi, and K. Dutta, *RAG Security and Privacy: Formalizing the Threat Model and Attack Surface*, 2025. arXiv:2509.20324 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2509.20324>
- [6] Q. Mao, Q. Zhang, H. Hao, Z. Han, R. Xu, W. Jiang, Q. Hu, Z. Chen, T. Zhou, B. Li, Y. Song, J. Dong, J. Li, and P. S. Yu, *Privacy-Preserving Federated Embedding Learning for Localized Retrieval-Augmented Generation*, 2025. arXiv:2504.19101 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2504.19101>
- [7] C. Jiang, X. Pan, G. Hong, C. Bao, Y. Chen, and M. Yang, *Feedback-Guided Extraction of Knowledge Base from Retrieval-Augmented LLM Applications*, 2025. arXiv:2411.14110 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2411.14110>
- [8] J. Li, C. Xu, L. Jia, F. Wang, C. Zhang, and J. Liu, *EACO-RAG: Towards Distributed Tiered LLM Deployment using Edge-Assisted and Collaborative RAG with Adaptive Knowledge Update*, 2025. arXiv:2410.20299 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2410.20299>
- [9] C. Xu, L. Gao, Y. Miao, and X. Zheng, *Distributed Retrieval-Augmented Generation*, 2025. arXiv:2505.00443 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2505.00443>
- [10] J. Yu and H. Sato, *DeRAG: Decentralized Multi-Source RAG System with Optimized Pyth Network*, in Proc. 2024 IEEE Int. Symp. on Parallel and Distributed Processing with Applications (ISPA), pp. 106–115, 2024. doi: 10.1109/ISPA63168.2024.00022

- [11] P. Zhou, Y. Feng, and Z. Yang, *Privacy-Aware RAG: Secure and Isolated Knowledge Retrieval*, 2025. arXiv:2503.15548 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2503.15548>
- [12] N. Grislain, *RAG with Differential Privacy*, in Proc. 2025 IEEE Conference on Artificial Intelligence (CAI), pp. 847–852, 2025. doi: 10.1109/CAI64502.2025.00150
- [13] L. Ammann, S. Ott, C. R. Landolt, and M. P. Lehmann, *Securing RAG: A Risk Assessment and Mitigation Framework*, 2025. arXiv:2505.08728 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2505.08728>
- [14] X. Liang, S. Niu, Z. Li, S. Zhang, H. Wang, F. Xiong, J. Z. Fan, B. Tang, S. Song, M. Wang, and J. Yang, *SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model*, 2025. arXiv:2501.18636 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2501.18636>
- [15] X. Gao, X. Meng, Y. Dong, Z. Li, and S. Guo, *DCMI: A Differential Calibration Membership Inference Attack Against Retrieval-Augmented Generation*, 2025. arXiv:2509.06026 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2509.06026>
- [16] A. Muhamed, M. Diab, and V. Smith, *CoRAG: Collaborative Retrieval-Augmented Generation*, 2025. arXiv:2504.01883 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2504.01883>
- [17] S. Liu, Z. Zheng, X. Huang, F. Wu, G. Chen, and J. Wu, *Efficient Distributed Retrieval-Augmented Generation for Enhancing Language Model Performance*, 2025. arXiv:2504.11197 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2504.11197>
- [18] D. Zhao, *FRAG: Toward Federated Vector Database Management for Collaborative and Secure Retrieval-Augmented Generation*, 2024. arXiv:2410.13272 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2410.13272>
- [19] X. Fang, L. Qiao, J. Shi, and H. An, *Guardian Angel: A Secure and Efficient Retrieval-Augmented Generation Framework*, in Proc. 2025 5th Int. Conf. on Artificial Intelligence and Industrial Technology Applications (AIITA), pp. 1773–1777, 2025. doi: 10.1109/AIITA65135.2025.11047845
- [20] K. Feng, W. Susilo, and J. Pieprzyk (eds.), *RAGLeak: Membership Inference Attacks on RAG-Based Large Language Models*, in Proc. ACISP 2025 – Information Security and Privacy, Lecture Notes in Computer Science, vol. 15660, Springer, Singapore, 2025. doi: 10.1007/978-981-96-9101-2_8
- [21] Y. Lu, W. Tang, M. Johnson, T. Jung, and M. Jiang, *A Decentralized Retrieval Augmented Generation System with Source Reliabilities Secured on Blockchain*, 2025. arXiv:2511.07577 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2511.07577>
- [22] C. Karlof and D. Wagner, *Secure Routing in Wireless Sensor Networks: Attacks and Countermeasures*, Ad Hoc Networks, vol. 1, no. 2–3, pp. 293–315, 2003. doi: 10.1016/S1570-8705(03)00008-8
- [23] J. Mu and Q. Li, *A Wolf in Sheep’s Clothing: Targeted Routing Hijacking in Federated RAG*, 2026. arXiv:2605.28112 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2605.28112>

- [24] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, *The EigenTrust Algorithm for Reputation Management in P2P Networks*, in Proc. 12th Int. Conf. on World Wide Web (WWW '03), pp. 640–651, 2003. doi: 10.1145/775152.775242
- [25] J. R. Douceur, *The Sybil Attack*, in Peer-to-Peer Systems (IPTPS 2002), Lecture Notes in Computer Science, vol. 2429, Springer, pp. 251–260, 2002. doi: 10.1007/3-540-45748-8_24
- [26] M. Castro and B. Liskov, *Practical Byzantine Fault Tolerance*, in Proc. 3rd Symp. on Operating Systems Design and Implementation (OSDI '99), pp. 173–186, 1999.